

Package ‘geo’

January 28, 2026

Version 1.5-1

Date 2022-11-14

Title Draw and Annotate Maps, Especially Charts of the North Atlantic

Author Hoskuldur Bjornsson, Sigurdur Thor Jonsson, Arni Magnusson, and Bjarki Thor Elvarsson

Maintainer Bjarki Elvarsson <bjarki.elvarsson@hafogvatn.is>

Depends maps, mapdata, R (>= 2.10.0)

Imports splines, methods, utils, grDevices, graphics, stats, polyclip, isoband

LazyData yes

LazyDataCompression gzip

Description Used by Hafro staff to draw maps showing the distribution of fishing intensity and catches, and of survey data for Icelandic fish stocks. Potentially useful for others.

License GPL (>= 2)

Encoding UTF-8

RoxygenNote 7.2.2

Suggests testthat (>= 3.0.0), knitr, rmarkdown

Config/testthat/edition 3

VignetteBuilder knitr

NeedsCompilation no

Contents

geo-package	4
adapt	5
adjust.grd	5
apply.shrink	6
apply.shrink.dataframe	7
arcdist	8
Arrow	8
bisland	9
bua	9
bwps	10

circle.one.point	11
Closed.curve	12
colps	12
colsymbol	13
combine.rt	15
COUNTRY.DEFAULT	17
currentarrows	17
cut_box.1	18
cut_box.2	18
cut_multipoly	19
d2ir	20
deg2rect	21
depthloc	23
Elimcomp	23
extract	24
eyjar	24
faeroes	25
fill.matrix	26
fill.outside.border	26
fill.points	27
find.hnit	28
findcut	28
findline	29
fitspher.aut.1	30
frame2gpx	30
gbdypi	31
gbdypi.100	32
gbdypi.1000	32
gbdypi.200	33
gbdypi.400	33
gbdypi.500	33
gbdypi.800	34
gbdypif.200	34
gbdypif.400	35
gbdypif.500	35
gbplot	36
geo.Split.poly	37
geoarea	37
geoarea.old	38
geoaxis	39
geochull	40
geocontour	41
geocontour.fill	44
geoconvert	50
geoconvert.1	51
geoconvert.2	51
geocurve	52
geodefine	53
geodezoom	54
geoexpand	54
geogrid	55
geoidentify	56

geoinside	57
geointersect	59
geolegend	60
geolines	61
geolines.with.arrows	62
geolocator	63
geopar	63
geopar.std	63
geoplot	64
geoplotbwpalette	70
geoplotpalette	71
geopoints	71
geopolygon	72
geoSR	74
geosubplot	76
geosymbols	77
geotext	82
geotows	84
geoworld	85
geozoom	87
gepco500	88
giveborder	88
glaciers	89
greenland	89
gridaxes	90
gridaxes.Lambert	91
gridpoints	92
icelanddrivers	93
ices	94
init	95
inside	97
inside.reg.bc	98
inside.reg.bc1	98
inside.reg.lump	99
intra.point.dist	101
invlambert	102
invmerc	103
invProj	104
island	105
labels1	105
labels2	106
labels_line	107
labels_size	108
lambert	109
litir	110
locdist	110
mercator	111
nonsetpar	112
Open.curve	112
orthproj	113
paint.window	113
paint.window.x	114

pdist	115
pdistx	115
plot_nogrid	116
plot_reitnr	116
pltgrid	117
plvar	118
pointkriging	118
postcol	120
pre2s	121
prepare.line	121
Proj	122
ps	123
rect2deg	124
rectArea	125
rectPeri	126
reg.bc	127
reg.lump	128
reitaplott	128
Reitur2Svaedi1to10	130
rGrid	131
Rlitir	132
s2pre	133
SegmentWithArrow	134
selectedpar	135
selpos	135
Set.grd.and.z	136
setgrid	137
shadeborder	138
shading1	139
skipta.texta	140
SMB.std.background	141
spherical	142
subplot	142
twohmls	143
variofit	143
variogram	144

Index**146**

geo-package

*Draw and Annotate Maps, Especially Charts of the North Atlantic***Description**

Used by Hafro staff to draw maps showing the distribution of fishing intensity and catches, and of survey data for Icelandic fish stocks. Potentially useful for others.

Author(s)

Höskuldur Björnsson

adapt	<i>Adapts geographical positions</i>
-------	--------------------------------------

Description

Adapts geographical positions for further geo-use.

Usage

```
adapt(reg.lat, reg.lon, projection = "Mercator")
```

Arguments

reg.lat	Latitude or y-coordinate
reg.lon	Longitude or x-coordinate
projection	Projection, default "Mercator", "none" denotes x/y coordinates

Value

Returns a list of either:

x, y	x- and y-positions
------	--------------------

or:

lat, lon	Latitude and longitude
----------	------------------------

and:

lxv	index of uninterrupted/contiguous positions
-----	---

Note

Needs further elaboration, this function is called by `geoarea.old`, `geoinside`, `inside` and `pointkriging`.

adjust.grd	<i>Adjusts grid for plot in Lambert projection.</i>
------------	---

Description

Adjusts grid for plot in Lambert projection.

Usage

```
adjust.grd(ply, rat = 0.025)
```

Arguments

ply	gridline??
rat	curvature or slant???

Value

Returns diagonal or curved lines for use in grid on Lambert projected plots.

Note

Needs elaboration. Utility, internal, might be hidden?

See Also

[findline](#), [gridaxes.Lambert](#)

apply.shrink

Apply a function to a vector for a combination of categories.

Description

Apply a function to a vector for a combination of categories.

Usage

```
apply.shrink(X, INDICES, FUN = NULL, names, ...)
```

Arguments

X	Input data to FUN
INDICES	list of categories to be combined
FUN	Function to be applied
names	Column names for the resulting dataframe
...	Additional arguments to FUN

Value

Dataframe of outcomes applying FUN to X for the combination of categories in INDICES

Note

Needs elaboration, or could be dropped/hidden, use merge instead.

See Also

[apply.shrink.dataframe](#), [merge](#)

Examples

```
## stupid example, showing naming of results
names(apply.shrink(depthloc$z,
  list(a=round(depthloc$lat), b=round(depthloc$lon)), mean))
names(apply.shrink(depthloc$z, list(round(depthloc$lat),
  round(depthloc$lon)), mean, names = c("a", "b", "z")))
```

`apply.shrink.dataframe`*Apply functions to columns in a dataframe*

Description

`apply.shrink` for a dataframe, different functions can be applied to different columns, one for each column (?).

Usage

```
apply.shrink.dataframe(  
  data,  
  name.x,  
  name.ind,  
  FUNS = NULL,  
  NA.rm = FALSE,  
  resp.name = NULL,  
  full.data.frame = FALSE,  
  Set = NA,  
  name.res,  
  ...  
)
```

Arguments

<code>data</code>	Input dataframe
<code>name.x</code>	Input value columns
<code>name.ind</code>	Category columns
<code>FUNS</code>	Functions to apply
<code>NA.rm</code>	na-action, default FALSE
<code>resp.name</code>	???
<code>full.data.frame</code>	???
<code>Set</code>	?
<code>name.res</code>	User selected values for the results columns
<code>...</code>	Additional arguments (to what???)

Value

Dataframe of variouse outcomes.

Note

Needs elaboration, although ... are included in the arguments list they don't seem to be used anywhere.

See Also

[apply.shrink](#), [merge](#)

arcdist

Geographical distance computations

Description

Computes distances between lat/lon data points.

Usage

```
arcdist(lat, lon, lat1 = NULL, lon1 = NULL, scale = "nmi")
```

Arguments

lat	Latitude of first coordinate or list with lat, lon of first coordinate.
lon	Longitude of first coordinate or list with lat, lon of second coordinate.
lat1, lon1	If lat and lon are vectors of lat,lon positions, then lat1 and lon1 must be given as the second set of positions.
scale	nmi returns value in nautical miles, any other value in kilometers

Value

A single vector of distances between pairs of points is returned

See Also

[geoplot](#), [geotext](#), [selpos](#),

Examples

```
pos1 <- list(lat = c(65, 66), lon = c(-19, -20))
pos2 <- list(lat = c(64, 65), lon = c(-19, -20))
dists <- arcdist(pos1, pos2)          # pos1 and pos2 are lists of coordinates.
lat <- c(65, 66)
lon <- c(-19, -20)
lat1 <- c(64, 65)
lon1 <- c(-19, -20)
dists <- arcdist(lat, lon, lat1, lon1) # Input in vector format.
```

Arrow

Add arrow to plot.

Description

Adds arrow to a plot, given base and point position along with half angle.

Usage

```
Arrow(pos, angle = 15, col = 2)
```

Arguments

pos	List with components <code>lat</code> , <code>lon</code> of length 2 or more (only first two used).
angle	Angle (< 90) determining arrow sharpness
col	Arrow color, defaults to color 2 of palette in effect.

Value

Mostly used for plotting, but the function returns the arrow polygon invisibly.

See Also

[geopolygon](#), [invProj](#)

Examples

```
geoplot()
Arrow(list(lat=c(65,65.5),lon=c(-19, -18)),angle=60,col="brown")
Arrow(list(lat=c(65,65.5),lon=c(-19, -18)),angle=45,col="red")
Arrow(list(lat=c(65,65.5),lon=c(-19, -18)),angle=30,col="green")
Arrow(list(lat=c(65,65.5),lon=c(-19, -18)),angle=30,col="blue")
```

bisland

Icelandic coastline, hi-resolution

Description

Icelandic coastline, hi-resolution, approx. 10 times that of island.

Format

A data frame with 19841 observations on the following 2 variables.

lat a numeric vector

lon a numeric vector

bua

Makes a list used in pointkriging

Description

Makes a list of four components sequences based on input number for use in pointkriging.

Usage

```
bua(nm = 10)
```

Arguments

nm A number

Value

Returns a list with components:

rrt	sequence 1 of length $4 \times nm^2$
crt	sequence 2, same length as rrt
rrt	sequence 3, same length as rrt
crt	sequence 4 of length $nm + 1$

for use in `pointkriging`.

Note

Needs further elaboration.

See Also

[pointkriging](#)

bwps	<i>Open a postscript device with the color scheme given by <code>geoplotbw-palette</code>. (black and white)</i>
------	--

Description

The function starts a postscript device and the arguments are the same as the arguments to `postscript` (height, width, file, bg etc) . The color scheme is 1 black, 2-155 white to black in even steps.

Usage

```
bwps(...)
```

Arguments

... The arguments to `postscript` are optional. See `help postscript`.

Details

<!--explain details here-->

Value

No value returned.

Side Effects

A graphics device is opened (often a file). It must be closed again by `dev.off()`

See Also

`postscript`, `geoplotbwpalette`, `litir`

Examples

```
## Not run:
bwps(file="map1.ps",height=6,width=5)
geoplot(xlim=c(-28,-10),ylim=c(64,69))
geosymbols(data,z=data$value,circles=0.2,sqrt=T)
geopolygon(island,col="white")# paint white over the symbols
geolines(island) # that are inside the country. (island)
dev.off()

# same example in a different way.
bwps(file="map1.ps",height=6,width=5,bg="white")
geoplot(xlim=c(-28,-10),ylim=c(64,69))
geosymbols(data,z=data$value,circles=0.2,sqrt=T)
geopolygon(island,col=0)#col 0 is now white
geolines(island) # was transparent earlier
dev.off()
## End(Not run)
```

circle.one.point

Find a circle around a point

Description

Through simple approximate geometry the perimeter of a circle around a point given in lat and lon is found

Usage

```
circle.one.point(lat, lon = NULL, rad, n = 10)
```

Arguments

lat, lon	Latitude and longitude in decimal degrees.
rad	radius of the circle in nmi
n	number of points on the circle perimeter

Examples

```
## draws a circle around Iceland
geoplot()
geolines(circle.one.point(65,-19,150,n=50),lwd=3,col="red")
```

Closed.curve	<i>Closed smooth curve through positions</i>
--------------	--

Description

A smooth curve is fitted to the positions with periodic spline as implemented in function `ps`.

Usage

```
Closed.curve(gogn, df = round(nrow(gogn)/2), n = 10)
```

Arguments

<code>gogn</code>	Data frame of positions to be smoothed
<code>df</code>	Degrees of freedom
<code>n</code>	Number of points returned per data point

Value

Data frame of positions with components

<code>lat, lon</code>	in decimal degrees
-----------------------	--------------------

Note

Missing function `spline.des` is required for [ps](#) to work.

See Also

[ps](#)

<code>colps</code>	<i>Open a postscript device with the color scheme given by <code>geoplot-palette</code>.</i>
--------------------	--

Description

The function starts a postscript device and the arguments are the same as the arguments to `postscript` (height, width, file, bg etc).

Usage

```
colps(...)
```

Arguments

<code>...</code>	The arguments to <code>postscript</code> are optional. See <code>help postscript</code> .
------------------	---

Details

<!--explain details here-->

Value

No value returned.

Side Effects

A graphics device is opened (often a file). It must be closed again by dev.off()

See Also

postscript, geoplotpalette, litir

Examples

```
## Not run:
colps(file="map1.ps",height=6,width=5)
geoplot(xlim=c(-28,-10),ylim=c(64,69))
geosymbols(data,z=data$value,circles=0.2,sqrt=T)
geopolygon(island,col="white")# paint white over the symbols
geolines(island) # that are inside the country. (island)
dev.off()

# same example in a different way.
colps(file="map1.ps",height=6,width=5,bg="white")
geoplot(xlim=c(-28,-10),ylim=c(64,69))
geosymbols(data,z=data$value,circles=0.2,sqrt=T)
geopolygon(island,col=0)#col 0 is now white
geolines(island) # was transparent earlier
dev.off()

## End(Not run)
```

colsymbol

Plot colored symbols

Description

Adds colored symbols in a variety of shapes to a geo-plot.

Usage

```
colsymbol(
  lat,
  lon,
  z,
  circles,
  squares,
  rectangles,
  hbars,
  vbars,
  perbars,
  parbars,
  levels,
```

```

nlevels,
colors,
white,
n,
maxcol,
digits,
label.location,
labels,
fill.circles,
density,
angle,
rotate,
minsym = "<",
label.resolution = 0,
col = 1,
labels.only = F,
open.circles,
lwd,
border = F,
bordercol = 0
)

```

Arguments

lat	Latitude
lon	Longitude
z	Value
circles	If not zero, use circles of this size.
squares	If not zero, use circles of this size
rectangles	If not zero, use circles of this size
hbars	If not zero, use circles of this size
vbars	If not zero, use circles of this size
perbars	If not zero, use circles of this size
parbars	If not zero, use circles of this size
levels	Levels used for determining symbols size
nlevels	Number of levels
colors	Colors to use
white	Logical, use white for lowest level if TRUE
n	Number of points used to make a circle (?)
maxcol	maxcolor?
digits	??
label.location	Where to put legend
labels	Labels for legend
fill.circles	Should circles be filled?
density	Density of shading when applicable
angle	Slant of shading

rotate	Should text (??) be rotated?
minsym	minimum value for a symbol to be drawn?
label.resolution	Number of digits in label???
col	Colors to use
labels.only	TRUE when labels/legend is added in a sperate call
open.circles	Should circles be open??
lwd	Line width of symbols
border	Should a border be drawn on the symbol
bordercol	Color for border if drawn

Value

No value returned, utility lies in side effect off adding colored symbols to existing plot, generally used as internal function in geosymbols.

Note

Needs further elaboration, see documentation for geosymbols.

See Also

Called by [geosymbols](#), calls [Proj](#), [geolocator](#), [labels_size](#), [labels1](#), [labels2](#), [shading1](#), [paint.window](#)

 combine.rt

Aggregate data on square grid, pad with zeros. Faster than tapply.

Description

Function that smooths data so data. In each given square where the number of points exceeds certain minimum the program gives the mean, sum, variance or median of the points within the square as well as the number of points and indication if the minimum number of points was reached. For each datapoint latitude, longitude and z has to be given. The program returns the mean and mean longitude for points within a square, as well as the number of points behind that point. The program also has the possibility to remove outliers within each square. This program can be used to prepare data for the program pointkriging if the data is so dense in certain places that it disturbs the neighbourhood search or if there are +10000 datapoints.

Usage

```
combine.rt(
  lat,
  lon,
  z,
  grlat,
  grlon = 0,
  fun,
  fill = F,
```

```

    reg = 0,
    minnumber = 2,
    wsp = 0,
    wz = 0,
    wlat = 0,
    xy = F,
    rat = 0.2,
    type
)

```

Arguments

lat	Latitude of datapoints. (or x coordinates)
lon	Longitude of datapoints. (or y coordinate)
z	Values at datapoints.
grlat	Latitude of defined grid.
grlon	Longitude of defined grid. If 0 grlat is a list with components lat and lon.
fun	Function (?).
fill	If fill is T the center points of squares where the number of points does not reach the minimum is set to zero. Good for data like trawlers report where it can be assumed that there is little fish where they do not try.
reg	List with two components lat and lon. Points outside region are not returned. The list is typically output from the program define.area.
minnumber	Minimum number of points needed for the square to be valid. Default value is 2.
wsp	Workspace. Not of interest.
wz	Weighing for the z values. If one is trying to find the mean tons/hour in an area is it reasonable to weigh tons/hour by the catch. If the weight is zero within a square unweighed mean is used.
wlat	Weighing to find the mean latitude and mean longitude. It is often reasonable to weigh the latitude and the longitude by the catch.
xy	Projected? Default unprojected (lat/lon) values, FALSE.
rat	Ratio for "rm.outliers". If rat is for example 0.2 20% of the points on each side are moved. i.e. the middle 60% are kept. Default value is 0.6.
type	Only value with description rm.outliers.

Details

It can also be used to find sums or averages of points within squares.

The program works quite fast even for very large datasets.

Value

<s-example> List with components lat,lon,z,n and fill. lat mean latitude of points within each square. lon mean longitude of points within each square z sum, mean, variance or median of data n Number of points behind point.

If type = "rm.outliers" n this component stores the number of the square corresponding to each point.
 fill If 0 there are datapoints behind the points else it is an artificial point. (fill = T) </s-example>

See Also

[variogram](#), [variofit](#), [pointkriging](#), [grid](#), [geocontour.fill](#).

COUNTRY.DEFAULT	<i>Default country for geo.</i>
-----------------	---------------------------------

Description

Default country, island, given as a character string.

Format

The format is: chr "island"

currentarrows	<i>Plot arrows and segments showing the size and direction of currents.</i>
---------------	---

Description

Plot arrows and segments showing the size and direction of currents.

Usage

```
currentarrows(
  data,
  maxsize = 0.5,
  maxn,
  col = "blue",
  lwd = 2,
  arrowsize = 0.2,
  center = T
)
```

Arguments

data	Data in a list with components lat and lon with decimal degrees, and current with the current magnitude.
maxsize	Maximum current segment size.
maxn	Current given with maxsize, defaults to max(data\$current).
col	Color of current arrows and segments.
lwd	Line width of the segments showing current.
arrowsize	Arrow size.
center	Whether or not to center the arrow, defaults to TRUE.

Note

Needs further checking and elaboration.

cut_box.1	<i>Cut box ?</i>
-----------	------------------

Description

Cut box in connection with overlaying a Lambert projected plot with a grid.

Usage

cut_box.1(x, y, xb, yb)

Arguments

- x, y Longitude and latitude of gridlines
- xb, yb Longitude and latitude limits

Value

List with compoents:

- x Longitudes??
- y Latitudes??
- ind Indices??

Note

Internal in gridaxes.Lambert, needs elaboration.

See Also

[gridaxes.Lambert](#)

cut_box.2	<i>Cut box type 2???</i>
-----------	--------------------------

Description

Cut box type 2???.

Usage

cut_box.2(x, y, xb, yb)

Arguments

- x Longtude ?
- y Latitude ?
- xb Limits of longitude?
- yb Limits of latitude?

Value

List with components:

x, y	Longitude and latitude of ???
x1, y1	Some other Longitude and latitude of ???

Note

Internal to gridaxes.Lambert, needs elaboration.

See Also

[gridaxes.Lambert](#)

cut_multipoly	<i>Intersect or take complement (?) of polygons</i>
---------------	---

Description

Intersect or take complement (?) of one or many polygons with a single one.

Usage

```
cut_multipoly(x, xb, in.or.out = 0)
```

Arguments

x	Polygon or polygons, seperated with NAs, hence 'multipoly'
xb	Polygon to intersect with/complement from
in.or.out	Whether to take the intersect (0) or complement of x in xb (1). Default 0

Value

List with compents:

x, y	with coordinates of intersection or complement (?)
------	--

Note

Check use of in.or.out=1, when there are many polygons in x, how is the complement with xb taken? Needs elaboration. Possibly drop in.or.out from argument list and fix it at 0 in call to findcut.

See Also

Called by [geopolygon](#), calls [findcut](#) and [prepare.line](#)

d2ir

*Convert between Geographic Coordinates and ICES Rectangles***Description**

This function converts geographic coordinates to ICES North Atlantic statistical rectangles and rectangle codes to rectangle center coordinates.

Usage

```
d2ir(lat, lon = NULL, useI = FALSE)
```

```
ir2d(ir, useI = FALSE)
```

Arguments

lat	vector of latitudes, or a list containing lat and lon.
lon	vector of longitudes (ignored if lat is a list).
useI	whether to use the letter “I” in statistical rectangle codes.
ir	ICES rectangle code, e.g. ‘37F3’

Details

The default useI=FALSE is in accordance with the prescription in ICES CM77/Gen:3, but useI=TRUE has been done on occasion.

Value

Vector of strings containing ICES statistical rectangle codes or conversely, center coordinates of rectangles which codes were given as input.

Note

The statistical rectangles are coded as follows:

The latitudinal rows are 30’ high, numbered from ‘01’ at the southern boundary of the ICES statistical area (36°00’N, see ICES No. 2 chart) northwards to ‘99’. The northern limit of the statistical rectangle system is thus 85°30’N. The longitudinal columns are 1° wide, coded in an alphanumeric system which starts at the western boundary of the ICES statistical area (44°00’W, see ICES No.1 chart) with ‘A0’, continuing ‘A1’, ‘A2’, ‘A3’ to 40°W. East of 40°W the system continues ‘B0’, ‘B1’, ‘B2’, ..., ‘B9’, ‘C0’, ‘C1’, ‘C2’, ..., ‘M8’, using a different letter for each 10° block and covering the entire west-east extent of the ICES statistical area, thus:

‘A0’	44°00’W	–	40°00’W	‘A3’
‘B0’	40°00’W	–	30°00’W	‘B9’
‘C0’	30°00’W	–	20°00’W	‘C9’
‘D0’	20°00’W	–	10°00’W	‘D9’
‘E0’	10°00’W	–	0°00’	‘E9’
‘F0’	0°00’	–	10°00’E	‘F9’
‘G0’	10°00’E	–	20°00’E	‘G9’
‘H0’	20°00’E	–	30°00’E	‘H9’

'J0'	30°00'E	–	40°00'E	'J9'
'K0'	40°00'E	–	50°00'E	'K9'
'L0'	50°00'E	–	60°00'E	'L9'
'M0'	60°00'E	–	68°30'E	'M8'

Note that letter “I” is omitted, unless useI=TRUE.

When designating a statistical rectangle, the north coordinate is stated first. Thus, the rectangle of which the southwest corner is 54°00'N, 3°00'E is coded '37F3'.

Author(s)

Sigurdur Thor Jonsson with contributions from Asta Gudmundsdottir and Arni Magnusson.

References

ICES C.M. 1977/Gen:3.

See Also

[d2r](#) and [r2d](#) convert between geographic coordinates and Icelandic rectangles (a local coding system using the same rectangle size as the ICES coding system).

Examples

```
d2ir(54.25, 3.5)
d2ir(c(50,60), c(-20,-10))

ir2d(d2ir(54, 3))
## center positions for bottom left and approx top right rects
ir2d("01A0")
ir2d("99M7")
ir2d(c("01A0", "99M7"))
## note that ICES CM1977/Gen:3 indicates half-size rects on eastern margin!
```

deg2rect

Given position return rectangle code.

Description

Functions that convert positions in decimal degrees latitude and longitude to rectangle codes for statistical rectangles, their subrectangles or rectangle codes in systems of various resolutions as described below and relating to (see [rect2deg](#)).

Usage

```
d2r(lat, lon = NULL)

d2sr(lat, lon = NULL)

d2mr(lat, lon = NULL, dlat = 5, dlon = 10)

d2dr(lat, lon = NULL, dlat = 1, dlon = 2, startLat = 50)
```

Arguments

lat, lon	Position(s) as decimal degrees latitude and longitude. If lat is list its components lat\$lat and lat\$lon are used for lat and lon.
dlat, dlon	Rectangle height and width in degrees and minutes latitude and longitude for d2dr and d2mr respectively.
startLat	Starting latitude used in coding the rectangles.

Details

- r2d with a resolution of 30 min latitude and 1 deg longitude (the Icelandic numbering system, 'tilkynningaskyldureitir').
- sr2d with a resolution of 15 min latitude by 30 min longitude in the Icelandic numbering system for statistical rectangles which starts counting at 60 deg N latitude, with sub-rectangles of 30 min lat by 1 deg lon coded 1, 2, 3 and 4 for the NW, NA, SW and SA quadrants respectively

A small number (1e-06) is added to latitude and subtracted from longitude to ensure rectangle membership of positions on border are “logical” on the nw-hemisphere.

Value

Vector of rectangle codes in the chosen coding system.

Note

These functions could be made hemisphere-aware, and no attention has been given to making the functions work in the southern hemisphere, possibly with the option of specifying starting latitudes.

Author(s)

HB (d2r, d2sr, STJ (d2mr, d2dr).

See Also

[rect2deg](#)

Examples

```
## tally positions in rectangles in object \code{\link{island}} giving
## Iceland's coastline

data(island)
rects <- d2r(island)
table(rects)
```

depthloc	<i>Depth label locations.</i>
----------	-------------------------------

Description

Standard locations for the labels of depth contours given in [gbdypi](#).

Format

A data frame with 9 observations on the following 3 variables.

lat a numeric vector

lon a numeric vector

z a numeric vector

Elimcomp	<i>Parameters manipulation ?</i>
----------	----------------------------------

Description

Parameters manipulation?

Usage

```
Elimcomp(parlist)
```

Arguments

parlist List of parameters ?

Value

Manipulated (shortened ?) parameterlist

Note

Needs elaboration, selectedpar warps around this function, where parameter names in nonsetpar come from is a mystery.

See Also

Called by [geoplot](#), [init](#) and [selectedpar](#).

extract	<i>Extract a grid (?)</i>
---------	---------------------------

Description

Extract a grid (?).

Usage

```
extract(grd, z, maxn = 10000, limits = NULL, col.names = c("lon", "lat"))
```

Arguments

grd	Grid
z	Value
maxn	Max number
limits	Limits
col.names	Defaults to lat, lon

Value

Returns a list with components:

grd1	A grid
z	Values over the grid

Note

Internal to the geo-contour-functions, needs elaboration.

See Also

Called by [geocontour](#) and [geocontour.fill](#).

eyjar	<i>Icelandic islands</i>
-------	--------------------------

Description

Islands around Iceland with area greater than 0.25 square km

Format

A data frame with 3401 (including NAs) observations on the following 2 variables.

lat a numeric vector,

lon a numeric vector: latitude and longitude of coastlines of Icelandic islands given as decimal degrees.

Details

Islands around Iceland extracted from gshhs_f.b with area greater than 1/4 of a square kilometer.
Script available as <http://www.hafro.is/~sigurdur/gshhsIslands/newEyjar.R>.

Source

GSHTS - A Global Self-consistent, Hierarchical, High-resolution Shoreline Database, version 2.2.

Examples

```
data(eyjar)
# islands in the Breidafjordur region
geoplot(xlim = list(lat = c(64.85, 65.65), lon = c(-24.6, -21.7)),
        country = bisland, grid = FALSE)
geolines(eyjar, col = "magenta")
## maybe str(eyjar) ; plot(eyjar) ...
```

faeroes

Coastline of Faroe Islands

Description

Old version of the coastline of the Faroes, with legacy typo.

Format

A data frame with 2161 observations on the following 2 variables.

lat a numeric vector

lon a numeric vector

Source

Possibly from package mapdata.

See Also

faroes

fill.matrix	<i>Relace elements of a matrix</i>
-------------	------------------------------------

Description

Replace (fill) elements of a matrix (or data.frame) with a value for given pairs of row and column indices.

Usage

```
fill.matrix(outcome, x, rownr, dalknr)
```

Arguments

outcome	Input matrix/data.frame
x	Value or values to replace/fill with
rownr	Row index/indices
dalknr	Column index/indices

Value

Matrix or data.frame with given values replaced.

Note

Probably redundant, not called by any geo-function, the same effect could be achieved with an assignment to a matrix with an index-matrix of the values in rownr and dalknr: `mat[matrix(c(rownr, dalknr), ncol = 2)] <- x`

fill.outside.border	<i>Fills space outside the border of a plot.</i>
---------------------	--

Description

When programs like `geopolygon` are used they sometimes fills space outside the border of their plots, to refill that space white we use `fill.outside.border`.

Usage

```
fill.outside.border(col = 0, rat = 1)
```

Arguments

col	The color of the fill, default is 0 (usually white).
rat	Ratio between the size of the current plot and the size of the fill, if you want to allow the program to draw 5% outside the plot you set <code>rat = 1.05</code> . Default <code>rat = 1</code> .

Value

No Value.

Side Effects

Fill outside of the border of the current plot.

See Also

[geoworld](#), [geopolygon](#).

Examples

```
## Not run:
geoplot(xlim=c(-50,20),ylim=c(50,70))      # Initialize plot.
geoworld(fill=T,color=120)                 # Colour countries.
fill.outside.border()                      # Clear outside of border.
geoplot(xlim=c(-50,20),ylim=c(50,70),new=T) # Relabel.

## End(Not run)
```

fill.points	<i>Fill points (thicken)</i>
-------------	------------------------------

Description

Fill points (thicken) for drawing continuous lines in Lambert projection.

Usage

```
fill.points(x, y, nx, option = 1)
```

Arguments

x, y	Coordinates
nx	Thickening factor
option	Deals with NAs in the coordinates when not 1, the default

Value

List of thickened values with components:

x, y	of coordinates
------	----------------

Note

Internal, needs elaboration.

See Also

The function is called by [geopolygon](#), [geolines](#), [reitaplott](#) and [gridaxes.Lambert](#)

find.hnit	<i>Find coordinate(s) ??</i>
-----------	------------------------------

Description

Find coordinates with some sort of interpolation (??).

Usage

```
find.hnit(pt, poly)
```

Arguments

pt	Point(s) ??
poly	Polygon (??)

Value

Returns list with components:

x, y	of coordinates
------	----------------

Note

~~further notes~~ Needs elaboration.

See Also

Called by [findcut](#)

findcut	<i>Find intersection or complement</i>
---------	--

Description

Find intersection or compliment of two polygons.

Usage

```
findcut(x, xb, in.or.out)
```

Arguments

x	Polygon
xb	Polygon to intersect with/complement from
in.or.out	Whether to take intersect of x and xb (0) or complement of x in xb (1). Default 0.

Value

Returns a list of

x, y	Coordinate of intersect or compliment
nxr	Number/index of returned coordinates in xb (?)

Note

Needs elaboration.

See Also

Called by [cut_multipoly](#), [geointersect](#) and [reitaplott](#); calls [find.hnit](#) and [geoinside](#).

findline	<i>Finds a line (?)</i>
----------	-------------------------

Description

Finds a line (?).

Usage

```
findline(x, xb, plot = T)
```

Arguments

x	Coordinates 1 (?)
xb	Coordinates 2 (?)
plot	Plot or not, default TRUE

Value

If plot is TRUE, returns coordinates for plotting, if FALSE returns data.frame of with latitude and longitude.

Note

Needs further elaboration.

See Also

Called by [adjust.grd](#), [geolines](#), [geolines.with.arrows](#), [init](#) and [link{reitaplott}](#), calls [invProj](#), [prepare.line](#) and [Proj](#).

<code>fitspher.aut.1</code>	<i>Fits a super-smoother to a variogram (?)</i>
-----------------------------	---

Description

Fits a super-smoother to a variogram (?)

Usage

```
fitspher.aut.1(vagram, option, sill)
```

Arguments

<code>vagram</code>	Variogram
<code>option</code>	What to do ?
<code>sill</code>	Sill for the variogram (?)

Value

Returns a list with components of the returned variogram:

<code>nugget</code>	Nugget
<code>dist</code>	Distance
<code>range</code>	Range
<code>sill</code>	Sill
<code>error</code>	Does the function call return an error? Used in <code>variofit</code>

Note

Needs further elaboration.

See Also

Called by [variofit](#).

<code>frame2gpx</code>	<i>Convert latlon data frame to gpx file.</i>
------------------------	---

Description

Writes a gpx file of a dataframe with a call to `gpsbabel` in the shell

Usage

```
frame2gpx(data, filename = "tmp.gpx", type = "rte")
```

Arguments

data	Data frame with positions in columns lat and lon.
filename	Name of gpx-file, defaults to 'tmp.gpx'.
type	Type of gpx-file, one of wpt for waypoints, rte' for route or trk for track.

Note

Requires gpsbabel installation working from the command line.

See Also

gpsbabel.org

Examples

```
## Not run:  
# some positions  
pos <- rPeri(323)  
frame2gpx(pos)  
system("more tmp.gpx")  
system("rm tmp.gpx")  
  
## End(Not run)
```

gbdypi

Isobaths in the NE-Atlantic

Description

An list of isobaths from IBCAO and/or GEBCO for selected depths.

Format

The format is: List of 58 data frame of lat and lon giving isobaths for the same number of depths ranging from 50 to 6000 m at 100 m intervals in the range 200 to 5500 m.

Source

IBCAO and/or GEBCO

gbdypi.100

GEBCO 100 m isobath in the N-Atlantic.

Description

GEBCO 100 m isobath in the N-Atlantic.

Format

A data frame with 19190 observations on the following 2 variables.

lat a numeric vector

lon a numeric vector

Examples

```
data(gbdypi.100)
## maybe str(gbdypi.100) ; plot(gbdypi.100) ...
```

gbdypi.1000

GEBCO 1000 m isobath in the N-Atlantic.

Description

GEBCO 1000 m isobath in the N-Atlantic.

Format

A data frame with 14869 observations on the following 2 variables.

lat a numeric vector

lon a numeric vector

Examples

```
data(gbdypi.1000)
## maybe str(gbdypi.1000) ; plot(gbdypi.1000) ...
```

gbdypi.200	<i>GEBCO 200 m isobath in the N-Atlantic.</i>
------------	---

Description

GEBCO 200 m isobath in the N-Atlantic.

Format

A data frame with 29665 observations on the following 2 variables.

lat a numeric vector

lon a numeric vector

Examples

```
data(gbdypi.200)
## maybe str(gbdypi.200) ; plot(gbdypi.200) ...
```

gbdypi.400	<i>GEBCO 400 m isobath in the N-Atlantic.</i>
------------	---

Description

GEBCO 400 m isobath in the N-Atlantic.

Format

A data frame with 7201 observations on the following 2 variables.

lat a numeric vector

lon a numeric vector

gbdypi.500	<i>GEBCO 500 m isobath in the N-Atlantic.</i>
------------	---

Description

GEBCO 500 m isobath in the N-Atlantic.

Format

A data frame with 16679 observations on the following 2 variables.

lat a numeric vector

lon a numeric vector

Examples

```
data(gbdypi.500)
## maybe str(gbdypi.500) ; plot(gbdypi.500) ...
```

`gbdypi.800`*GEBCO 800 m isobath in the N-Atlantic.*

Description

GEBCO 800 m isobath in the N-Atlantic.

Format

A data frame with 3083 observations on the following 2 variables.

lat a numeric vector

lon a numeric vector

Examples

```
data(gbdypi.800)
## maybe str(gbdypi.800) ; plot(gbdypi.800) ...
```

`gbdypif.200`*GEBCO 200 m isobath around Iceland.*

Description

Given in one polygon.

Format

A data frame with 914 observations on the following 2 variables:

lat a numeric vector

lon a numeric vector

Examples

```
data(gbdypif.200)
## maybe str(gbdypif.200) ; plot(gbdypif.200) ...
```

`gbdypif.400`*GEBCO 400 m isobath around Iceland.*

Description

Given as 4 polygons.

Format

A list of 4 data.frames of the following 2 variables:

lat a numeric vector

lon a numeric vector

Examples

```
data(gbdypif.400)
## maybe str(gbdypif.400) ; plot(gbdypif.400) ...
```

`gbdypif.500`*GEBCO 500 m isobath around Iceland.*

Description

Given as 2 polygons.

Format

A list of 2 data.frames of the following 2 variables:

lat a numeric vector

lon a numeric vector

Examples

```
data(gbdypif.500)
## maybe str(gbdypif.500) ; plot(gbdypif.500) ...
```

`gbplot`*GEBCO plot. Plots equidepth lines.*

Description

Plots lines of equal depths using a database from GEBCO.

Usage

```
gbplot(depth, col, lty, lwd, depthlab, depthlabcex)
```

Arguments

<code>depth</code>	A vector of the depths which we want equidept lines plotted.
<code>col</code>	The colour of the lines, if the <code>col</code> vector is shorter than the <code>depth</code> vector it is repeated. Default is all lines black.
<code>lty</code>	Linetype, if the <code>lty</code> vector is shorter than the <code>depth</code> vector it is repeated. Default is all lines have linetype 1.
<code>lwd</code>	Linewidth, if the <code>lwd</code> vector is shorter than the <code>depth</code> vector it is repeated. Default is all lines have linewidth 1.
<code>depthlab</code>	A boolean variable determening whether labels should be printed on equidepth lines, default is false.
<code>depthlabcex</code>	The size of depthlabels.

Value

None

Side Effects

Plots equidepth lines on current plot.

See Also

[geoplot](#), [geolines](#).

Examples

```
geoplot() # Set up plot.  
  
gbplot(c(100,500,1000),depthlab=T,depthlabcex=0.2)  
# Plot depthlines for 100,500,1000,1500 m, showing the  
# depth on the line.
```

geo.Split.poly	<i>Split a polygon into a list</i>
----------------	------------------------------------

Description

Split a 'polygon with NAs' into a list of true polygons before calculating its area.

Usage

```
geo.Split.poly(data)
```

Arguments

data	A dataframe of coordinates in latitude and longitude
------	--

Value

Returns a list of true polygons as dataframes, each with coordinates

lat	Latitude
lon	Longitude

Note

Needs further elaboration.

See Also

Called by [geoarea](#).

geoarea	<i>Calculates the area of a given region.</i>
---------	---

Description

Calculates the area of a given region to a given precision.

Usage

```
geoarea(
  data,
  Projection = "Lambert",
  old.method = F,
  ngrdpts = 2000,
  robust = T
)
```

Arguments

data	The region, should contain lat and lon.
Projection	Which projection is beeing used.
old.method	If true an older version of this program is used. Default is False.
ngrdpts	The precision of the calculation.
robust	If true a more robust method is used, default is True.

Value

The area of the region given.

Side Effects

None

See Also

[geodefine](#), [geocator](#), [geoinside](#).

Examples

```

geoarea(island)          # Calculates the area of Iceland up to
                          # an with default precision.

geoarea(island,10000)    # Calculates the area of Iceland up to
                          # an adiquite precision.

#      geoarea(geodefine(),10) # Calculates the area of a region specified
                          # by the user.
```

geoarea.old

Old method for calculating geographical area

Description

Calculates the area inside a geographical region by splitting it into a grid, determining which grid-points are inside the region and then finds the area.

Usage

```
geoarea.old(reg, n, robust = T)
```

Arguments

reg	Region of interest, list with components lat and lon.
n	Numeber of gridpoints to use for calculating the area.
robust	Robust or not, defaults to TRUE.

Value

Geographical area in square nautical miles.

Note

Difference between old and new geoarea-method might be explained better, if indeed it is necessary to keep the old method.

See Also

[geoarea](#).

geoaxis	<i>Setup axes for a geoplots</i>
---------	----------------------------------

Description

Set up axes for a geoplots.

Usage

```
geoaxis(
  side,
  pos,
  dist,
  dlat = 0.5,
  dlon = 1,
  csi = 0.12,
  cex = 0.7,
  inside = T,
  r = 1,
  ...
)
```

Arguments

side	On which side should the axis be drawn?
pos	Positions of labels (?)
dist	Distance from plot of axes labels (?)
dlat	Latitude resolution of labels (?)
dlon	Longitude resolution of labels (?)
csi	Character size inches, default 0.12
cex	Character size expansion, default 0.7
inside	Whether or not stay inside ??????????
r	yet another setting ??????????
...	Additional arguments to geotext

Value

Adds labels to a geoplot, no value returned.

Note

Needs further elaboration.

See Also

Calls [geotext](#), called by [init](#).

geochull	<i>Convex hull of a set of positions</i>
----------	--

Description

Finds the convex hull of a set of positions in lat and lon.

Usage

```
geochull(lat, lon = NULL)
```

Arguments

lat, lon	Position(s) as decimal degrees latitude and longitude. If 'lat' is 'list' its components 'lat\$lat' and 'lat\$lon' are used for 'lat' and 'lon'.
----------	--

Value

List with components lat and lon of the convex hull of the input.

See Also

[Proj](#), [invProj](#), [chull](#)

Examples

```
# draws the convex hull of Iceland's coastline.
geoplot()
geolines(geochull(island))
```

 geocontour

Plots contour lines.

Description

The function plots contour lines. It is based on the Splus function "contour" with some changes and extensions. The main change is that coordinates are in lat, lon instead of x, y. There are two additions : 1. Possibility to have many colors. 2. Possibility to only plot lines within certain borders. 3. Possibility to have labels in a box on the plot. Note: to plot contour plots with fill between lines you use geocontour.fill

Usage

```
geocontour(
  grd,
  z,
  nlevels = 10,
  levels = NULL,
  labcex = 1,
  triangles = TRUE,
  reg = 0,
  fill = 1,
  colors = TRUE,
  col = 1,
  only.positive = FALSE,
  maxcol = 155,
  cex = 0.7,
  save = FALSE,
  plotit = TRUE,
  label.location = 0,
  lwd = 1,
  lty = 1,
  labels.only = FALSE,
  digits = 1,
  paint = FALSE,
  set = NA,
  col.names = c("lon", "lat"),
  csi = NULL,
  drawlabels = FALSE
)
```

Arguments

- | | |
|-----|--|
| grd | List with components lat and lon (or x,y) defining the grid values. Can be made for example by lat <- list(lat = seq(60,65,length = 100), lon = seq(-30,- 10,length = 200)). Also lat can be the output of the program grid and then the components lat\$grpt\$lat, lat\$grpt\$lon and lat\$reg are used. See geocontour.fill. |
| z | Matrix or vector of values. Length nlat*nlon except when lat is a list with components lat\$grpt and lat\$xgr. Then the length of z is the same as the length of lat\$xgr\$lat and lat\$xgr\$lon. |

nlevels	Number of contourlines. Used if the program has to determine the contourlines. Default value is 10. nlevels = 10 does not always mean 10 due to characteristic of the pretty command
levels	Ratio of textsize to current size. 1 means same size, 0.5 half size , 0 no text etc. Default value 1.
labcex	Character expansion of letters indicating the z value on the contour lines. Maybe used instead of a legend.
triangles	If TRUE the program makes 4 triangles from each element in the grid. In fact it makes the number of grid points approximately twice as many as before which means smoother contourlines. Default value TRUE. Triangles = TRUE is necessary if the area was plotted with geocontour.fill. It fits with geocontour.fill.
reg	List with two components, reg\$lat and reg\$lon. Region of interest. Contourlines are only plotted inside the region. Holes in the region of interest begin with NA. If the region consists only of holes then reg\$lat and reg\$lon begin with NA.
fill	If fill is one the matrix is filled with zeros. If two it is filled with mean(z). Default is fill = 1. (not an important parameter)
colors	If colors is TRUE the contour lines are plotted in many colors. Else only in one. The lines can also be in one color using many linetypes.
col	Number of color (colors) used to plot contourlines. Default value is 1 if colors is FALSE. If colors is TRUE the default value is found from the number of contour lines.
only.positive	Logical value. If FALSE then negative values are allowed else negative values are set to zero. Default value is FALSE.
maxcol	Maximum color number in the Splus graphics, default value 155.
cex	Character expansion of digits, default value is 0.7.
save	If true the contour lines are saved in a list so they can later be plotted with geolines. Default value is false.
plotit	If TRUE the lines are plotted on the graphics device else not. Default value is true.
label.location	List with components lat and lon (or x,y) Gives the lower left and upper right corner of the box where the labels are put. Default value is 0 that means no labels are put on the drawing (except when geopar\$cont = TRUE). 11 is best given by geolocator or directly by specifying label.location = "locator".
lwd	Line width. Default value is the value set when the program was called.
lty	line type. If lty is a vector of the same length of levels it specifies the linetype for each contour line. Default value is the same as when the program was called.
labels.only	If true only the labels are drawn. Default is false.
digits	Number of digits in labels. Default value is one.
paint	if true borders of regions will be painted.
set	Set something.
col.names	the names of the vectors containing the data in grd. Default is col.names = c("lon","lat").
csi	Size of character. This parameter can not be set in R but for compatibility with old Splus scripts the parameter cex is readjusted by $cex = cex * csi / 0.12$. Use of this parameter is not recommended. Default value is NULL i.e not used.
drawlabels	Draw labels on the contour lines? Default FALSE.

Value

No values returned.

Side Effects

No side effects.

See Also

[contour](#), [geocontour.fill](#), [geolocator](#), [geopolygon](#), [geotext](#), [geosymbols](#), [geogrid](#), [geopar](#), [geolines](#).

Examples

```
#####
# Example 1                                     #
#####

## Not run:
  geoplots(deg, cont = TRUE)                    # Plot initialized.
  geocontour(grd$lat,grd$lon,z,nlevels = 10,
             neg = FALSE,reg = reg,colors = TRUE) # Contour plot.
  geoplots(deg,pch = " ",cont = TRUE,new = TRUE) # Plot over contourplot.

## End(Not run)
#####
# Example 2 Sea Temperature.                     #
#####

# The following data names used are in Icelandic, stodvar means
# stations and botnhiti means temperature.

geoplots()
gbplot(500)
grd <- list(lat = seq(63,67,length = 30),
            lon = seq(-28,-10,length = 50))
labloc <- list(lat = c(63.95,65.4),lon = c(-19.8,-17.3))

grd1 <- geoexpand(grd)                         # Make grid.
grd2 <- geoinside(grd1,gbdpif.500)
grd2 <- geoinside(grd2,island,robust = FALSE,option = 2)
# Use only the points where depth < 500 and outside Iceland.

## Not run:
  xx <- loess(botnhiti~ lat*lon,degree = 2,spALSEn = 0.25,
             data = stodvar, na.action = na.omit)
  # Use loess for interpolating.

  grd2$temp <- predict(xx,grd2)
  #geocontour(grd2,z = grd2$temp,levels = c(0,1,2,3,4,5,6,7),
  #           label.location = labloc)

#####
# Example 3 example of gam() and indexes.         #
#####

stations<-data.frame(lat = stodvar$lat,lon = stodvar$lon,
                     temp = stodvar$botnhiti)
```

```

# Making a partial data.frame from a big one called stodvar,
# which means stations in Icelandic.

stations[1:5,]          # Show first 5 lines all columns
                        # in stations.
dim(stations)           # Length of (lines,columns).
dim(stations[!is.na(stations$temp),]) # Without NAs.
my.data <- stations[!is.na(stations$temp),]
my.data <- my.data[!is.na(my.data$lat),]
my.data <- my.data[!is.na(my.data$lon),]
# my.data is now same as stations but without NAs in lat,
# lon and temp.

pred.grid <- list(lat = seq(63.25,67.25,length = round((67.25-63.25+1)*8)),
                  lon = seq(-27,-11.5,length = round((27-11.5+1)*4)))
pred.grid <- geoexpand(pred.grid)
# Making a grid to fit our area of interest.
pred.grid <- geoinside(pred.grid,gbdpif.500)
# Points within 500m.
pred.grid <- geoinside(pred.grid,island,robust = FALSE,option = 2)
# Points outside of Iceland.

geoplot(grid = FALSE)
my.data <- geoinside(my.data,island,robust = FALSE,option = 2)
geopoints(my.data,pch = ".")

fit <- gam(temp~lo(lat,lon,span = 0.1),data = my.data)
# see help(gam)
# can also do:
# fit <- loess(temp~lon*lat,data = my.data,span = 0.1)
# fit <- gam(temp~ns(lon,df = 7)*ns(lat,df = 5),data = my.data)

pred.grid$pred.temp <- predict(fit,newdata = pred.grid)
geocontour(pred.grid,z = pred.grid$pred.temp,levels = 0:7,
            label.location = labloc)

## End(Not run)

```

geocontour.fill

geocontour.fill plots colored or black and white contours on a graph made by geoplot.

Description

The program accepts data on a rectangular grid. Irregular data can be interpolated on the grid in a number of ways, using kriging, the Splus function `interp`, etc. NA's are allowed in the matrix but they are changed to 0 or the average value of `z` early in the program. The `geo` package contains two different contour plot programs, `geocontour` and `geocontour.fill`. `Geocontour` draws contour lines and `geocontour.fill` fills in between the lines. Hardcopy of the plot can be done on a color postscript printer or on a black and white postscript printer. In all cases the "postscript" driver included with Splus has to be used. There is another postscript driver included with Splus called "pscript". That driver is used when a plot is on the screen and the print button is pressed. That can not be done when the postscript driver is used as described here after.

Usage

```

geocontour.fill(
  grd,
  z,
  levels = NULL,
  nlevels = 0,
  cex = 0.7,
  digits = 1,
  col = NULL,
  working.space = 0,
  labels = 1,
  ratio = 1000,
  only.positive = FALSE,
  fill = 0,
  maxcol = 155,
  white = FALSE,
  label.location = 0,
  labels.only = FALSE,
  bordercheck = FALSE,
  maxn = 10000,
  bcrat = 0.05,
  limits = NULL,
  col.names = c("lon", "lat"),
  minsym = "<",
  label.resolution = 0,
  labtxt = NULL,
  boxcol = 0,
  first.color.trans = TRUE,
  mai = c(0, 1, 0, 1),
  leftrat = 0.1,
  labbox = TRUE,
  csi = NULL
)

```

Arguments

grd	List with components lat and lon (or x, y) defining the grid values. Can be made for example by <code>lat <- list(lat = seq(60, 65, length = 100), lon = seq(-30, -10, length = 200))</code> . Also lat can be the outcome of the program grid and then the components <code>latgrptlat</code> , <code>latgrptlon</code> and <code>latreg</code> are used.
z	Matrix or vector of values. <code>Length(z) = nlat*nlon</code> except when lat is a list with components <code>latgrpt</code> and <code>latxgr</code> . Then the length of z is the same as the length of <code>latxgrlat</code> and <code>latxgrlon</code> . This is the case if the output of the grid program is used as input lat.
levels	Values at contourlines. Default value is zero. If levels is zero the program determines the contourlines from the data. If levels is of length 3 with levels = <code>c(0, 1, 2)</code> then there are 4 groups each with special color i.e. <code><0</code> , <code>0-1</code> , <code>1-2</code> and <code>>2</code> .
nlevels	Number of contourlines. Used if the program has to determine the contourlines. Default value is 10. <code>nlevels = 10</code> does not always mean 10 due to characteristic of the pretty command.

cex	Character size expansion. Size of letters in labels. Default value is 0.7.
digits	Number of digits in the labels. Default value is 1.
col	Color number for the contourlines. A vector one longer than the vector levels. Default is blue-green- yellow-red from lowest to the highest values. The program assumes certain setup of the splus colors that is described later. Colors should not be specified explicitly except the contour lines are.
working.space	Size of working space. The program determines it from data and prints on the screen. If mysterious errors occur then it is likely that the program has not reserved enough working space. The program writes the used work space on the screen for use in similar situations. (saves time.)
labels	Type of labels, either 1 or 2 <s-example> labels = 1 means type of label used with few contourlines <20. labels = 2 means type of label used with many contourlines >20. can also use labels = 3 means only labels, no contour plot. </s-example> Labels can be inserted in two ways, by calling geoplot with cont = TRUE or by specifying label.location. In the first case the left part of the plot is reserved for labels while in the latter case the label is put in a place specified by the user. The latter method is recommended in most cases.
ratio	Factor used to avoid numerical problems. Default value 100. Lower values decrease numerical problems but can introduce bias.
only.positive	Logical value. If TRUE then negative values are not allowed else negative values are set to zero. Default value is FALSE.
fill	Determines whether NA's should be replaced with zeros (fill = 1) or mean(z) (fill = 2). Default value is fill = 1. Used when lat is a list with components latxgrlat, latxgrlon, latgrptlat and latgrptlon. Only used in special cases.
maxcol	Number of colors used (excluding #0). Default value 155.
white	If true the the first class is represented with white (color 0). Default value is FALSE but white = TRUE is also often used.
label.location	List with components lat and lon. (or x, y) Gives the lower left and upper right corner of the box where the labels are put. Default value is 0 that means no labels are put on the drawing (except when geoparcont = TRUE). 11 is best given by geolocator or directly by specifying label.location = "locator".
labels.only	If labels.only is true no contours are drawn but only labels. Default value is false. Order of the commands could be: <s-example> > geoplot(deg, plot = FALSE) > geocontour.fill(lat, z = z, levels = lev, reg = area) # Draw contours. > geocontour(lat, z = z, levels = lev, colors = FALSE, reg = area) # Only used with black and white printers to make distinction # between different levels clearer. (not used with color # printers). > geoplot(deg, new = TRUE) # Refresh gridlines. > geocontour.fill(lat, z = z, levels = lev, labels.only = TRUE, label.location = 11) # Add labels. </s-example>
bordercheck	if true the program checks if plot is outside border and does not plot what is outside border.
maxn	a parameter determing the resolution of the plot, unimportant.
bcrat	bordercheck ratio, how much outside the border we will allow to be plotted, default is bcrat = 0.05 meaning that we will allow the plot to go 5% off the border.
limits	To be described.

col.names	the names of the columns in grid, the first argument will be plotted on the x-axes and the second on the y-axes. Default is col.names = c("lat", "lon").
minsym	minimum symbol, default is "<", meaning that if levels = c(1, 2), labels will be presented as < 1, 1-2, 2 <, but if minsym = " " labels will be presented as 1, 1-2, 2. See also labels.resolution.
label.resolution	the resolution (precision) of the label numbers, default is 0, meaning that if levels = c(1, 2), labels will be presented < 1, 1-2, 2 <, if label.resolution = 0.1 labels will be presented < 1, 1.1-2, 2.1<, see also minsym. If label.resolution = "none", the labels will present the lowest number of the interval with each color.
labtxt	To be described.
boxcol	Colour of box around labels (legend?).
first.color.trans	To be described.
mai	par argument 'margin in inches'?
leftrat	To be described.
labbox	should a box be drawn around labels (legend?).
csi	Size of character. This parameter can not be set in R but for compatibility with old Splus scripts the parameter cex is readjusted by $cex = cex * csi / 0.12$. Use of this parameter is not recommended. Default value is NULL i.e not used.

Details

<s-example>

The program is based on making triangles out of a matrix of data $n_x * n_y$. The number of triangles is $(n_x - 1) * (n_y - 1) * 4$ and linear interpolation is used inside each triangle. The factor 4 comes from the fact that the number of points is doubled by interpolation. </s-example> <s-example>

Most of the calculations in the program are done by a program written in C. The Splus part of the program prepares the data for the C program, reserves memory and the user interface is in Splus. The C program is loaded automatically. </s-example> <s-example>

The program is currently based on the following color setup in Splus. </s-example> <s-example>
 splus*color : white black blue 50 green 50 yellow 50 red </s-example> <s-example>

This gives 155 colors with color#0 white, #1 black, #2 blue, #53 green, #104 yellow, and #155 red. The parameter maxcol is set to 155 based on this setup but it can be changed if another setup is used. The vector postcol stores the mapping from the colors on the terminal to color postscript based on this setup. </s-example> <s-example>

If the openlook() or motif() windowmanager is used in Splus the colors can be changed inside Splus. In openlook it is best to make a colorscheme called geoplot: black blue 50 green 50 yellow 50 red. The background color, i.e. white is not in the definition here. </s-example> <s-example>

The maximum number of contourlines that can be used is currently 60. </s-example> <s-example>

The program requires that geoplot is called before to set up the drawing. geoplot is called by the parameter cont = TRUE. This is to reserve space for labels on the drawing. </s-example> <s-example>

If the drawing is written to a file geoplot should be called with plot = FALSE. Then it only sets up the drawing but does not plot anything so the size of the file will be reduced. </s-example> <s-example>

If the labels do not fit the relative space taken by labels and picture can be changed by the parameter `lcont` in `geoplot`. After `geocontour.fill` is called `geoplot` is called again with `new = TRUE` to get all kind of lines back but they were painted over by the program. `</s-example>` `<s-example>`

The program treats borders in a special way. It begins by making contour-lines over the hole area as if the borders did not exist. When that is finished the area outside the borders is painted white. `</s-example>` `<s-example>`

If good picture is needed `geocontour` should be used between levels to get sharper pictures. `</s-example>` `<s-example>`

If the matrix `z` is not full the program should be called by a list `latxgrlat`, `latxgrlon`, `latgrptlat` and `latgrptlon`. Then the length of the vectors `z`, `latxgrlat` and `latxgrlon` is the same. `latgrptlat` and `latgrptlon` is on the other hand the coordinates of the rows and columns of the matrix. The program grid makes a list with components with these names. `</s-example>` `<s-example>`

The functions `geolines`, `geopoint`, `geopolygon`, `geotext` & `geosymbols` can be used to add things to the contourplot. `</s-example>`

See Also

[geoplot](#), [geolines](#), [geopolygon](#), [geotext](#), [geosymbols](#), [geogrid](#), [geopar](#), [geolocator](#), [geocontour](#), [reitaplott](#), [geodefine](#).

Examples

```
## Not run: #####
# Example 1.                                     #
#####

# Need the data.frame botnv.2004 to be able to compile this
# example, if not attached use:
# >attach("/usr/local/reikn/Splus5/Aflaskyrslur/Data")

codgrd <-list(lat = seq(62, 68, by = 0.1), lon = seq(-30, -9, by = 0.25))
# A grid is made.
lab.loc<-list(lat = c(63.9, 65.6), lon = c(-20.75, -16.5))
# Location of the label.

tmp <- combine.rt(botnv.2004$lat, botnv.2004$lon,
                 botnv.2004$torskur, codgrd, fun = "sum", fill = TRUE)
# The data is read into the grid with combine.rt.

tmp$z <- tmp$z/(cos(tmp$lat*pi/180)*0.1*60*0.25*60)
# Data changed.(from being in
# kilos per box to kilos per square mile).

vg <- list(nugget = 0.1, sill = 1, range = 50)
# Parameters for the variogram

z <- pointkriging(tmp$lat, tmp$lon, tmp$z, codgrd, vg,
                 maxnumber = 80, maxdist = 30, set = -1)
# Dataset smoothened with pointkriging.

geoplot(lat = c(63, 67.5), lon = c(-27, -11), grid = FALSE, axlabels = TRUE, type = "n")
# Plot initialized
level = c(160, 200, 320, 500, 700, 1000, 2000, 3000, 4000, 5000, 6000)
# Levels for geocontour.fill
```

```

geocontour.fill(codgrd, z, levels = level, white = TRUE      # Plot the data.
                , working.space = 300000)

geopolygon(island)
# Contourlines inside Iceland overwritten.
geolines(island)
# Iceland redrawn with geolines.

geocontour.fill(codgrd, z, levels = level, white = TRUE,
                label.location = lab.loc, labels.only = TRUE)
# Call geocontour.fill again only to plot the labels.

#####
# Example 2.                                     #
#####

# Preperation for pointkriging
# th4.2002 is the data used here, dataframe [lon, lat, mat].
# >attach(????)

grd.smb <-list(lat = seq(62.8, 67.5, length = 80),   # Set up the grid.
               lon = seq(-28, -10, length = 130))
m.lev<-c(0, 0.1, 0.2, 0.3, 0.5)
# Levels for the geocontour.fill.
m.col<-c(0, 14, 59, 104, 119, 149)
# Colors for the levels.
lab.in.island<-list(lat = c(63.9, 65.6), lon = c(-20.75, -16.5))
# Location of the Label

vg <- list(nugget = 0.3, sill = 1, range = 50)
# Initialize variogram parameters.

zfj<-pointkriging(th4.2002$lat, th4.2002$lon, z = th4.2002$mat,
                  grd.smb, vg, maxnumber = 80, maxdist = 30, set = -1)
# Smooth the data with pointkriging.

#
# Plotting
#

par(mfrow = c(1, 1), mai = rep(0, 4))
# Set up graphic parameters.
geoplot(lat = c(63, 67.5), lon = c(-27, -11), grid = FALSE, axlabels = FALSE, type = "n")
# Draw a background with Iceland with geoplot.

geocontour.fill(zfj, levels = m.lev, col = m.col, working.space = 300000)
# Plot the data with geocontour.fill.

geopolygon(gbdypif.500, col = 0, exterior = TRUE, r = 0)
# Remove contours outside gbdypif.500.
geoplot(lat = c(63, 67.5), lon = c(-27, -11), grid = FALSE,      # Replot.
        axlabels = FALSE, type = "n", new = TRUE)

geopolygon(island, col = 43)

```

```
# Remove contours inside Iceland and color Iceland.
geocontour.fill(zfj, levels = m.lev, label.location = lab.in.island,
               labels.only = TRUE, csi = 0.1, col = m.col, working.space = 300000)
# Call geocontour.fill to plot labels.
geolines(island)
# Redraw the lines of Iceland.

## End(Not run)
```

geoconvert

Convert latitude and longitude

Description

Convert between different representations of latitude and longitude, namely degrees-minutes-decimal minutes and decimal degrees.

Usage

```
geoconvert(data, inverse = F, col.names = c("lat", "lon"))
```

Arguments

<code>data</code>	Dataframe with coordinates in two columns
<code>inverse</code>	Which conversion should be undertaken, default from degrees-minutes-decimal minutes (DDMMmm) to decimal degrees (DD.dd)
<code>col.names</code>	Colnames in data with coordinates to convert, default lat, lon

Value

Returns data with converted values in the coordinate columns.

Note

Functions calling `geoconvert` do so in a branch that is probably rarely used. Implement conversion from other representations of lat and lon in future?

See Also

Called by a number of functions, i.e. [d2mr](#), [mr2d](#), [geogrid](#), [geoidentify](#), [geolines](#), [geopoints](#), [geopolygon](#) and [geotext](#), perhaps unnecessarily in some. Wraps around functions [geoconvert.1](#) and [geoconvert.2](#), depending on which conversion to undertake.

geoconvert.1	<i>Convert to decimal degrees</i>
--------------	-----------------------------------

Description

Convert to decimal degrees.

Usage

geoconvert.1(x)

Arguments

x Vector of decimal-minute-decimal minutes

Value

Returns converted value in decimal degrees.

See Also

Called by [geoconvert](#)

geoconvert.2	<i>Convert from decimal degrees</i>
--------------	-------------------------------------

Description

Convert from decimal degrees to degrees, minutes and fractional minutes representation (DDM-Mmm) of lat or lon.

Usage

geoconvert.2(lat)

Arguments

lat Vector of latitude or longitudes

Value

Returns a vector of six digit values with degrees, minutes and fractions of minutes, with two decimal values, concatenated.

See Also

Called by [geoconvert](#), when inverse = TRUE.

geocurve

*Smooth curves and put arrows in the beginning and end.***Description**

Smooth curves and put arrows in the beginning and end. Useful for plotting closed areas, making smooth lines with arrows etc. The function needs dataframe with columns lat and lon. No NA values currently allowed. Uses the functions ns or ps to smooth the curve depending on if the curve is open or closed. Smooths separately $\text{lat} \sim \text{ns}(d, df)$, $\text{lon} \sim \text{ns}(d, df)$. d is here distance along the curve. If the same curve is smoothed many times the output is stored the first time and the function called later with `smooth=F`.

Usage

```
geocurve(
  data,
  df = nrow(data)/2,
  n = 10,
  open = T,
  arrow = "none",
  col = 1,
  lwd = 1,
  size = 0.2,
  angle = 15,
  smooth = T,
  plot = T,
  ...
)
```

Arguments

<code>data</code>	Dataframe that must have columns lat and lon.
<code>df</code>	Degrees of Freedom in smoothing the curve. Default is number of points/2. Maximum allowed df is <code>nrow(data)-1</code> . <code>df="all"</code> gives this value.
<code>n</code>	How much denser the output data is than the input data. <code>n=10</code> means that the value of the smoothing spline is predicted at 9 evenly spaced points between each pair of data points. Default is <code>n = 10</code> .
<code>open</code>	Is the curve open or closed. Default open.
<code>arrow</code>	If there is an arrow in the beginning, end or both of the line segment. Options are "none", "start", "end", "both".
<code>col</code>	Color of line.
<code>lwd</code>	Width of line segment. Default is 1. <code>lwd=2</code> seems to fit well with default size of arrows.
<code>size</code>	Size of arrow, default 0.2 inches.
<code>angle</code>	Angle of arrow opening, default 15 degrees.
<code>smooth</code>	Should the data be smoothed, default is T.
<code>plot</code>	Should the curve be plotted, default is T.
<code>...</code>	Other parameters to <code>geolines</code> and <code>geopolygons</code> .

Value

If smooth=F = the input data is returned, else smoothed inputdata.

Side Effects

Plots the line segment on the screen.

See Also

[geolines](#), [geopolygon](#), [ns](#).

Examples

```
## Not run:          # define curve store the result.
curve1 <- geocurve(geolocator(type="p"),arrow="end",lwd=2)
# use the result.
geocurve(curve1,smooth=F,arrow="start",lwd=2,col=150)

# define closed area and hatch it.
area1 <- geocurve(geodefine(),open=F)
geopolygon(area1,density=10,col=1)

# Make closed curve with big arrow and not store the result.
geocurve(geodefine(),open=F,arrow="end",lwd=2,size=0.5)

## End(Not run)
```

geodefine

Defines regions.

Description

locates points pointed out by users and defines regions.

Usage

```
geodefine(nholes = 0)
```

Arguments

nholes The number of holes in data, number of regions - 1.

Details

Draws the regions on the plot.

Value

A list of the points pointed out by the user with NA's between regions.

Side Effects

Draws the regions on the plot.

See Also

[geolocator](#).

Examples

```
## Push left mouse button to mark point, push middle button to  
## mark the end of a region.
```

geodezoom	<i>Restores a zoomed plot.</i>
-----------	--------------------------------

Description

Geodezoom restores a plot zoomed with geodezoom.

Usage

```
geodezoom()
```

Value

none

Side Effects

The limits of the current plot change back to its original size.

See Also

[geoplot](#), [geozoom](#).

Examples

```
## See examples in help(geozoom).
```

geoexpand	<i>expands a grid to a dataframe.</i>
-----------	---------------------------------------

Description

Expands a grid to a dataframe similar to `expand.grid`.

Usage

```
geoexpand(grid)
```

Arguments

grid The grid to be expanded, a list containing vectors lat and lon.

Value

A dataframe of the grid.

Side Effects

None.

See Also

[expand.grid.](#)

Examples

```
grd <- list(lat=seq(63,67,length=30),lon=seq(-28,-10,length=50))
# a list with length(lat) = 30 and length(lon) = 50

grd1<- geoexpand(grd)
# a 30 by 50 dataframe made.

# See also examples in geocontour.
```

geogrid

Plots a grid.

Description

Plots a grid defined by the vectors lat, lon. The grid is plotted on a graph initialized by geoplots. lon gives the meridians plotted and lat the parallels plotted.

Usage

```
geogrid(lat, lon = 0, col = 1, type = "l", lwd = 0, lty = 0, pch = "+", nx = 5)
```

Arguments

lat, lon	Latitude and longitude of data (or x and y coordinates), negative for southern latitudes and western longitudes. May be supplied as two vectors or as a dataframe lat (or x) including vectors lat\$lat and lat\$lon (x\$x and x\$y if projection = none).
col	Color number used, default value is 1 (black).
type	"l" means line and "p" points. Default is "l".
lwd	Linewidth. Default value is the value set when the program was called.
lty	Linetype. Default value is the value set when the program was called.
pch	Type of symbol at gridpoints default is "+".
nx	sets smoothness of curved Lambert parallels

Value

No values returned.

See Also

[geoplot](#), [geolines](#), [geopolygon](#), [geotext](#), [geosymbols](#), [geopar](#), [geolocator](#), [geocontour](#).

Examples

```
## Not run:          geogrid(latgr, longgr)

codgrd <- list(lat = seq(62, 68, by = 0.1), lon = seq(-28, -10, 0.25))
geogrid(codgrd)      # a fine grid of Iceland and neighbouring seas
geoplot(new = T)

## End(Not run)
```

geoidentify

Identifies points on plots using lat and lon coordinates.

Description

Works the same way as `identify` except that it also accepts coordinates as `lat` and `lon`. Identifies points on a plot identified by the user.

Usage

```
geoidentify(
  lat,
  lon = NULL,
  labels = 1,
  n = 0,
  plot = TRUE,
  atpen = TRUE,
  offset = 0.5,
  col = 1,
  cex = 1
)
```

Arguments

<code>lat, lon</code>	Coordinates of points. The coordinates can be given by two vectors or a <code>data.frame</code> containin vectors <code>lat</code> and <code>lon</code> .
<code>labels</code>	Vector giving labels for each of the points. If supplied, this must have the same length as <code>lat</code> and <code>lon</code> . As a default the vector indece number of the points will be used.
<code>n</code>	Maximum number of points to be identified.
<code>plot</code>	If true, <code>geoidentify</code> plots the labels if the points identified.
<code>atpen</code>	If true, plotted identification is relative to locator position when the point is identified; otherwise, plotting is relative to the identified <code>lat,lon</code> value. This can be useful when points are crowded. Default is true.

offset	Identification is plotted as a text string, moved offset character from the point. If the locator was left (right) of the nearest point, the label will be offset to the left (right) of the point.
col	The color of the labels.
cex	Character size expansion of label characters.

Details

Observations that have missing values in either lat or lon are treated as if they were not given. When using the X11 driver under the X Window System, a point is identified by positioning the cursor over the point and pressing the left button. To exit identify press the middle button (both buttons on a two button mouse) while the cursor is in the graphics window. The same procedure is used under the suntools driver. This function may also be used with the "tek" drivers.

Some devices that do not allow interaction prompt you for an x,y pair. The nearest point to the locator position is identified, but must be at most 0.5 inches away. In case of ties, the earliest point is identified.

Value

Indices (in lat and lon) corresponding to the identified points.

Side Effects

Labels are placed on the current plot if plot is true.

See Also

[identify](#), [geolocator](#), [geotext](#).

Examples

```
## Not run:      geoidentify(stations, labels = stations$temp)
                 # plots the temperature in the closest measuring point.

                 geoidentify(stations, atpen = FALSE)
                 # plots the indece number of the station closest to
                 # where pointed at the stations position.

## End(Not run)
```

geoinside

Finds a subset of a given set of data which is inside a given region.

Description

Finds a subset of a given set of data which is inside or outside a given region and returns a submatrix of those values, boolean vector of indexes or index vector.

Usage

```
geoinside(
  data,
  reg,
  option = 1,
  col.names = c("lat", "lon"),
  na.rm = T,
  robust = F
)
```

Arguments

data	a dataframe, should include vectors lat and lon, but will except other names, see col.names.
reg	The region we want to determine whether the data is inside of, should include vectors with same names as data.
option	Allows you to determine on what format you recieve the output: <s-example> option = 1: returns the submatrix of data which is inside reg. option = 2: returns the submatrix of data which is outside reg. option = 3: returns a boolean vector saying whether a given index is inside reg. option = 4: returns a boolean vector saying whether a give index is outside reg. option = 5: returns a vector of indexes to data, data[return[i],] is the i-th point in data inside reg. option = 6: returns a vector of indexes to data, data[return[i],] is the i-th point in data outside reg. </s-example>
col.names	Default col.names = c("lat","lon"), determines the names of the base vectors of the space we are viewing. May be replaced by for instance col.names = c("x","y")
na.rm	If true values where lat or lon are NA are removed. Default is false.
robust	If true a robust search is done, if false the function runs faster. Default is true. Robust = T will not work if the regions edges overlap each other, if region is sensibly defined this will not happen and robust =F should be used.

Value

Returns and output vector or matrix, see option.

Side Effects

None.

See Also

[geoplot](#), [geolocator](#).

Examples

```
## Not run:   seafishing <- geoinside(fishing,island,option=2)
# Removes those datapoints from fishing where fishing took
# place inside Iceland (misspells).

grd <- list(lat=c(64,64,63,63),lon=c(-23,-22,-22,-23))
ins.lat.64.63.lon.23.22 <- geoinside(fishing,grd,robust =T)
```

```

# Extracts those points from fishing where fishing
# took place inside the given box.

#####
# Example                                     #
#####

par(mfrow=c(2,1))
stations<-data.frame(lat=stodvar$lat,lon=stodvar$lon)
stations<-stations[!is.na(stations$lon),]
stations<-stations[!is.na(stations$lat),]

geoplot(grid=F)
geopoints(stations,pch=".",col=25)
title(main="Before geoinside")

sea.stations <- geoinside(stations,island,option=2)
# Removes those datapoints from stations where
# measurments took place inside Iceland (misspells).

geoplot(grid=F)
geopoints(sea.stations,pch=".",col=25)
title(main="After geoinside")

#####

## End(Not run)

```

geointersect

Find intersection of 2 polygons

Description

Find intersection of 2 polygons

Usage

```
geointersect(data, border, in.or.out)
```

Arguments

data	Polygon.
border	Polygon to intersect with/complement from.
in.or.out	Whether to take intersect of A and B (0) or complement of A in B (1). Default 0.

See Also

[findcut](#), [invProj](#), [Proj](#)

geolegend

Put a legend on a plot in the geo series.

Description

Adds a legend to current plot. The location can be specified with lat and lon. Allows all the same parameters as legend.

Usage

```
geolegend(pos, legend, ...)
```

Arguments

pos	The position of the text, should include lat and lon. If lat and lon are llength 1, they determine the top left corner of the rectangle; if they are length 2 vectors, they give opposite corners of the rectangular area. A list containing x and y values may be supplied.
legend	Vector of character strings to be associated with plot.
...	The function allows any optional argument to the legend function to be taken.

Details

See legend.

Value

None.

Side Effects

Draws a box at specified coordinates and puts inside (if possible) examples of lin, points, marks and/or shading, each identified with a user-specified text string.

See Also

[legend](#).

Examples

```
# The function is currently defined as
function(pos, legend, ...)
{
    oldpar <- par()
    par(geopar$gpar)
    on.exit(par(oldpar))
    xx <- Proj(pos$lat, pos$lon)
    legend(xx$x, xx$y, legend = legend, ...)
}
```

geolines

Add lines to current plot initialized by geoplot.

Description

Add lines to a plot initialized by geoplot. Data is stored as lat, lon or x,y. Lists are assumed to have the components \$x and \$y if projection in geoplot was "none", else \$lat,\$lon. The program transforms the data as specified in geoplot. Similar to the Splus function lies.

Usage

```
geolines(
  lat,
  lon = 0,
  col = 1,
  lwd = 0,
  lty = 0,
  nx = 1,
  outside = FALSE,
  return.data = FALSE
)
```

Arguments

lat	Latitude of data. (or x coordinate)
lon	Longitude of data. (or y coordinate) Negative values mean western longitudes. Default value is zero. If lon is zero then the data is stored as lat\$lat and lat\$lon. (or lat\$x and lat\$y)
col	Colour number used for plotting the lines, default value is 1.
lwd	Line width. Default is to use the width set when the program was called.
lty	Line type. Default is to use the width set when the program was called. See Splus manuals for numbers corresponding to different linetypes and linewidths.
nx	Parameter only used with Lambert transform when lines in lat,lon are curves in x,y. If nx > 1, nx-1 points are put between each two datapoints in lat, lon before projection is done. For example: geolines(c(66, 66), c(-30, -10), nx = 50) plots a line onto the 66 degree latitude from -30 to -10. The line is curved because it is made of 50 segments.
outside	Logical, should lines outside the plot region be drawn? Default FALSE.
return.data	Logical, should the data be returned? Default FALSE.

Value

No values returned.

Side Effects

The projection is stored in geoparprojection and parameters for the transform in geopar\$b0, geopar\$b1 and geopar\$l1.

See Also

[geoplot](#), [geopolygon](#), [geopoints](#), [geotext](#), [geosymbols](#), [geocontour.fill](#), [geogrid](#), [geocontour](#).

Examples

```
geolines(island)                # plot iceland.
geolines(island$lat, island$lon, col = 1) # same.

#####

geoplot(xlim=c(0, -50), ylim=c(60, 75), projection = "Lambert")
# Set up a Lambert plot.

geolines(c(66, 66), c(-30, -10), nx = 50, col = 155, lwd = 2)
# Draw a line with colour 155 and width 2.

geopolygon(island)
geolines(island, col = 3, lwd = 3)
geolines(eyjar, col = 40)
geolines(faeroes, col = 40)
geolines(greenland, col = 3, lwd = 3)
#   geolines(janmayen, col = 40)
# Plot some more countries using geolines.
```

`geolines.with.arrows` *Add arrowhead to plot*

Description

Adds arrowhead at the start or the end of a list of coordinates on a geoplot.

Usage

```
geolines.with.arrows(data, start = T, size = 0.2, ...)
```

Arguments

<code>data</code>	Dataframe with coordinates in columns <code>lat</code> , <code>lon</code> .
<code>start</code>	Start from beginning (i.e. arrow points at first coordinate), default <code>TRUE</code> .
<code>size</code>	Size of arrow
<code>...</code>	Additional arguments to <code>Arrow</code>

Value

Draws an Arrowhead at the start or the end of the coordinates given in `data`, returns invisibly the outline as four coordinate pairs in a dataframe.

Note

May need further elaboration and/or detail.

See Also

Called by [geocurve](#), calls [findline](#), [invProj](#) and [Arrow](#).

geolocator	<i>Locates points on a plot initialized by geoplot.</i>
------------	---

Description

The function locates points on a plot initialized by [geoplot](#) returning their latitude and longitude.

Usage

```
geolocator(type = "p", n = 0)
```

Arguments

type	Same parameter as in the locator function. Type = "l" draws line between points.
n	Number of points. Default value is zero, then the point coordinates are located till the mouse' middle button is clicked.

Value

A list with components \$lat and \$lon. or (\$x,\$y if `geopar$projection = "none"`)

See Also

[geoplot](#), [locator](#), [geodefine](#).

geopar	<i>Parameters for the geo plot functions.</i>
--------	---

Description

Parameters for the geo plot functions.

Format

A list of parameters.

<code>geopar.std</code>	<i>Parameters for the geo plot functions.</i>
-------------------------	---

Description

Parameters for the geo plot functions.

Format

The format is: List of 5 \$ xlim: num [1:2] -30 -10 \$ ylim: num [1:2] 62.2 67.9 \$ plt : num [1:4] 0.0816 0.9534 0.1218 0.9004 \$ fig : num [1:4] 0 1 0 1 \$ mex : num 1

geoplot

Plots lat and lon coordinates using Mercator or Lambert transformation.

Description

Creates a plot of given data, as the "plot" function but allows input data to have latitude and longitude coordinates. Coordinates are latitude and longitude in degrees (default) or x, y in m (or other units).

Usage

```
geoplot(
  lat = NULL,
  lon = 0,
  type = "p",
  pch = "*",
  xlim = c(0, 0),
  ylim = c(0, 0),
  b0 = 65,
  r = 1.05,
  country = "default",
  xlab = " ",
  ylab = " ",
  option = "cut",
  grid = TRUE,
  new = FALSE,
  cont = FALSE,
  cex = 0.9,
  col = 1,
  lcont = c(0.13, 0.21),
  plotit = TRUE,
  reitur = FALSE,
  smareitur = FALSE,
  reittext = FALSE,
  cexrt = 0.7,
  csirt = NULL,
  axratio = 1,
  lwd = 0,
  lwd1 = 0,
  locator = FALSE,
  axlabels = TRUE,
  projection = "Mercator",
  b1 = b0,
  dlat = 0,
  dlon = 0,
  jitter = 0,
  zoom,
  csi = NULL,
  xaxdist = 0.2,
  yaxdist = 0.3
```

)

Arguments

lat, lon	Latitude and longitude of data (or x and y coordinates), negative for southern latitudes and western longitudes. May be supplied as two vectors or as a list or dataframe lat (or x) including vectors latlat and latlon (xx and xy if projection = none). If xlim and ylim are given the arguments lat and lon are not required.
type	Options are the same as in the plot function i.e "l" for lines, "n" for not plotting the data, "p" for points etc. Default value is "p"
pch	Type of symbol drawn in points. Default is "*". Any other character or symbol can be used, see the help on points.
xlim	x limits of drawing (longitudinal direction). If not given the program finds the limits as the range of the data times the parameter r (1.05 default). xlim can be a list (dataframe) with components lat and lon (x and y if projection = "none").
ylim	y limits of drawing (latitudinal direction). If not given and xlim is not a list the program finds the ulimits as the range of the data times the parameter r (1.05).
b0	Base latitude for the Mercator or Lambert transform. Default value is 65 (typical for Iceland). If Mercator transform is used values close to the mean of the data are recommended except the data extend very far north, (> 75 th degree) then b0 should be close to the northern limits of the data).
r	Size of area which is plotted. r = 1.0 means exactly the range of the datapoints, but r = 1.5 means that the range is 1.5 times the range of data. Default value is 1.05. Does not matter if xlim and ylim are given.
country	Country that is plotted. Options are: 1. island 1300 points (iceland) 2. bisland 20000 points (iceland fine) 3. greenland 62000 points 4. faeroes 2100 points 5. eyjar 2200 points (islands around iceland) 6. none no country plotted Which map is used depends on the size of the area plotted. If small part of the coast is seen bisland is used, else Iceland. Islands can be added later by geolines(eyjar). Default is set by a variable with the name COUNTRY.DEFAULT. COUNTRY.DEFAULT <- "island" makes iceland the default country. The program geoworld can be used to add all coastlines that intersect the map and also to make new countries (see geoworld).
xlab	X-label. Default value is " "
ylab	Y-label. Default value is " "
option	Can be either "cut" or "nocut". If "nocut" the plot always fills the plotted area but if "cut" the plot does not fill it in the direction where the range of data is minimum. It has to be kept in mind that the program always keeps the same scale vertically and horizontally. Default value is "cut". Not effective when contourplots are plotted.
grid	If grid is TRUE meridians and parallels are plotted, else not. Default value is TRUE.
new	If new is FALSE the plot is added to the current plot otherwise a new plot is made. Default value is FALSE. Similar to the Splus command "par(new = TRUE)".
cont	A parameter to reserve space for legend for contours besides the plot. Default value is "FALSE". Rarely used at the legends are usually put somewhere in the graph.

cex	Relative size of character and symbols (see the help on the parameter cex). The size of plotted characters is cex time the parameter csi that can be seen by par(csi). In earlier versions of geoplot the parameter csi was set but csi is a parameter that can not be set in R.
col	The color number used to for the plot. Default value is 1 that is usually black.
lcont	Limits of area preserved for lables and contour plot as ration of plotting area. Default value is c(0.13, 0.21). That means that the labels take 13% of the plotting area but the figure 79%. (Labels to left, figure to left.) 0.08 to 0.1 is a reasonable difference. Only used when cont = TRUE which is seldom used as described erlier.
plotit	If FALSE plot is only initialized but not plotted. If used other programs are used to fill the plot (geolines, geocontour, geopolygon etc). Most often used in multiple plots. Used in connection with geocontour.fill to fewer files but the plot command is given again with new = TRUE when geocontour.fill is called. Plot = FALSE does not work if axeslabels = FALSE. Something seems to have to be on the graph for proper setup.
reitur	Reitur means statistical square in Icelandic. If true the division of the axes is idendical to the distribution of the ocean around Iceland in statistical squares. Means dlat = 0.5;dlon = 1.
smareitur	If true the division of the axes is idendical to the distribution of the ocean around Iceland in subsquares.dlat = 0.25;dlon = 0.5
reittext	If true the number of each square is written in the center of the square. Only meaningful for Icelandic waters.
cexrt	Relative size of reittext text in statistical squares (cexrt*csi). Default value is 0.7.
csirt	Size of reittext. Default value is 0.1. Only for compatibility with older Splus programs but cexrt should be used as the parameter csi can not be set in R:
axratio	Parameter usually not changed by the user.
lwd	Line width for plot (grid and axes). Default value is the value set when the program was called.(usually 1). Higher values correspond to wider lines.
lwd1	Line width for plot country. Default value is the value set when the program was called (usually 1). Higher values correspond to wider lines.
locator	Some kind of a simple zoom command. If locator is TRUE the user points with the mouse on the limits of the plot he wants to make. Needs a plot made by geoplot on the screen. Same as if zoom is used.
axlabels	If FALSE no numbers are plotted on the axes. Default value is TRUE.
projection	Projection used to make the plot. Options are "Mercator", "none" and "Lambert". Default value is "Mercator". If projection = "none" data is assumed to be x, y.
b1	Second latitude to define Lambert projection.
dlat	Defines the grid, to make a grid on the lat axis, 1 is a number on axis and a line at every deg. Not usualy set by user.
dlon	Same as dlat, but for lon.
jitter	useful if many datapoints have the same coordinates, points are jittered randomly to make common values look bigger. jitter = 0.016 is often about right but you may want to have jitter smaller or bigger varying on plot.

zoom	If TRUE the mouse is used to point to two points on the current map with those two new points becoming the new corner points on a new map. . Same as the parameter locator in older geoplot versions.
csi	Size of character. This parameter can not be set in R but for compatibility with old Splus scripts the parameter cex is readjusted by $cex = cex * csi / 0.12$. Use of this parameter is not recommended. Default value is NULL i.e not used.
xaxdist	Distance from plot to the labels on the xaxis (dist or r argument to geoaxis. Default value is 0.2 but higher value mean that axlabels is further away from the plot. Further flexibility with axes can be reached by calling geoplot with axlabels = FALSE and geoaxis afterwards.
yaxdist	Distance from plot to the labels on the yaxis (dist or r argument to geoaxis. Default value is 0.3 but higher value mean that axlabels is further away from the plot.

Details

The program performs the Mercator(default) or Lambert transformation of the data in degrees and plots it. The plot is scaled such that 1cm in vertical and horizontal directions correspond to the same distance. The program is used to initialize drawings to be used by programs, i.e. geolines, geopolygon, geotext, geosymbol, geogrid, geocontour.fill, geoimage and geocontour. The inputs to the program are decimal numbers negative for western longitudes e.g.lat = 65.75 lon = -25.8. Whether the program interprets data as lat, lon or x, y depends on the parameter projection. If projection is "none" data is interpreted as x, y else lat, lon. When the data is interpreted as x, y lists are assumed to have the components x and y, else lat and lon. Geoplot is often used with type = "n" so the datapoints used to set up the drawing are not seen. Calling geoplot with plotit = FALSE sets up the drawing without putting anything on the screen (or page).

Value

No values are returned. The graphical setup is stored in a global list called geopar. That list is accessed by other program that use the same setup.

Side Effects

There should be no side effects. The program changes a number of graphical parameters but the old parameters are restored before.

See Also

[geolines](#), [geopolygon](#), [geotext](#), [geosymbols](#), [geogrid](#), [geopar](#), [geocontour.fill](#), [geolocator](#), [geocontour](#), [reitaplott](#), [geodefine](#), [Proj](#).

Examples

Not run: Examples shown here below also include calls to the other functions in the geopackage. Further explanations of these functions can be found in the appropriate help files.

Contour plot of haddock catch in icelandic waters based on logbooks.
A color scheme
where color 0 is white, 1 black and 2-150 gradually changing from white to black is used. The data is for 6 years and is stored in a list hadcatch with 6 components. Circles showing haddock catch in

the Icelandic groundfish survey is added on top of the plot with the function `geosymbols` but `utbrteg` is a dataframe with information on all catch in the Icelandic groundfish survey (all names in Icelandic `ysa` means haddock and `ar` year). The function `bwps` is a call to the `postscript` function with the indicated color scheme. Designed for `Splus` and has to be changed for `R` as the color schemes there are quite different. This applies to all the examples below.

```
lev <- c(0.5, 1, 2, 4, 6)
col <- c(0, 30, 50, 70, 90, 150)
txt <- c(1993, 1995, 2000, 2002, 2005, 2006)
par(mfrow = c(3, 2));par(mex = 0.01)
bwps(file = "hadcatchutbr.ps", height = 6.8, width = 6.5, horizontal = FALSE)
for(i in 1:6) {
  SMB.std.background(grid = FALSE, axlabels = FALSE)
  geocontour.fill(hadcatch[[i]], levels = lev, col = col,
    white = TRUE, working.space = 2e6)
  gbplot(200)
  geotext(67.3, -27.6, txt[i], csi = 0.16, adj = 0)
  geopolygon(island, col = 0);geolines(island)
  tmp <- utbrteg[utbrteg$ar == txt[i], ] # select the year
  geosymbols(tmp, z = tmp$ysa.kg, circles = 0.2,
    sqrt = TRUE, lwd = 1)# amount of haddock
  geopoints(tmp, pch = 16, csi = 0.05)
}
dev.off()

# plot x, y data
geoplot(x$x, x$y, projection = "none", type = "n")

geoplot(x, projection = "none", type = "n")
# does the same thing.

# The packages maps and mapdata need to be installed
# worldHires is a very detailed database of coastlines from the
# package mapdata. Could be problematic if used with fill = TRUE)
# Allowed.size is the maximum allowed size of polygons.
library(map) # world coastlines and programs
library(mapdata) # more detailed coastlines
geoplot(xlim = c(20, 70), ylim = c(15, 34))
geoworld(database = "worldHires", fill = TRUE, col = 30, allowed.size = 30000)

geoplot(xlim = c(20, 70), ylim = c(15, 34), dlat = 10, dlon = 10)
geoworld(database = "world", fill = TRUE, col = 30) #

geoplot(xlim = c(-10, 70), ylim = c(71, 81), b0 = 80,
  dlat = 2, dlon = 10) # 0 must be high here else
geoworld(database = "world", fill = TRUE, col = 30) #the plot fails.

# Lambert projection,
geoplot(xlim = c(-10, 70), ylim = c(71, 81),
  dlat = 2, dlon = 10, projection = "Lambert")
geoworld(database = "world", fill = TRUE, col = 30)

# Lambert projection, get the axis closer with the mgp command
par(mgp = c(2, 0, 0))
geoplot(xlim = c(-10, 70), ylim = c(71, 81),
```

```

    dlat = 2, dlon = 10, projection = "Lambert", cex = 1.1)
geoworld(database = "world", fill = TRUE, col = 30)

# Example with capelin data. (lodna means capelin). lodna.2 is a
# data.frame with components lat, lon and z.
geoplot(lodna.2, type = "l")
geopoints(lodna.2)
geosymbols(lodna.2, z = lodna.2$z, colplot = TRUE,
  parbars = 0.05, levels = vor.levels, label.location = labloc)

geoplot(lodna.2, type = "l")
geosymbols(lodna.2, z = lodna.2$z, parbars = 0.1)

geoplot(lodna.2, type = "l")
geosymbols(lodna.2, z = log(1+lodna.2$z), parbars = 0.1)

limits <- list(lat = c(63, 68), lon = c(-30, -10))
geoplot(xlim = limits, type = "n", grid = FALSE, axlabels = TRUE, plot = FALSE)
tmp <- geoexpand(lodna.2.grd) # expand the grid
# has defined the area vor.area and data outside it are set to NA.
i <- geoinside(tmp, vor.area.new, option = 0)
zgr <- z.lodna.2; zgr[-i] <- NA
geocontour.fill(lodna.2.grd, z = zgr, white = TRUE,
  label.location = labloc, levels = vor.levels)
geoplot(xlim = limits, type = "n", grid = FALSE, axlabels = TRUE, new = TRUE)
#geolines(lodna.2, lwd = 1)
gbplot(c(200, 500)) # Depth contours.

# make a plot of number within a square, calculate the total number of
#cod (torskur) within a square (reitur) and put the text number of
#square and total number of cod in the center of the square (number of
#cod below number of square). Apply.shrink is similar to tapply
#returning the data in different form and is included with the geo library

geoplot(island, r = 1.2, type = "n", reitur = TRUE)
x <- apply.shrink(data$torskur.stk, data$reitur, sum,
  names = c("reitur", "torskur.stk"))
x1 <- r2d(x$reitur)
geotext(x1, z = paste(x$reitur, round(x$torskur.stk, 1), sep = "\n"))

# Plot filled circles. The color scheme used is the same as described
# color 0 white, 1 black and 2 - 155 white-black see bwps
# in geosymbols the argument color means size (in inches)
# when the fill.circles = TRUE. The data used AfliBySquareMonthYear have
# the columns year, month, square and catch.
# the function r2d changes square (reitur in Icelandic) to position
# Text is put in the middle of the circles where catch exceeds 1000
# tonnes.

my.colors = c(0.004, 0.04, 0.1, 0.15, 0.20, .25, 100)
lev <- c(0.2, 2, 7.5, 10, 20, 50)

yy <- c(1932:1939)
tmp4 <- AfliBySquareMonthYear
tmp4$catch <- tmp4$catch/1000

```

```

for (ar in yy) {
  bwps(file = paste(ar, ".ps", sep = ""))
  par(omi = c(0, 0, 0, 2))
  par(mfrow = c(4, 3))
  par(mex = 0.01)

  for(man in 1:12){
    tmp1 <- tmp4[tmp4$year == ar & tmp4$month == man, ]
    SMB.std.background(axlabels = FALSE, country = "none", plotit = FALSE)
    if(nrow(tmp1) > 0) {
      tmp2 <- apply.shrink(tmp1$catch, tmp1$square, sum,
        names = c("square", "catch"))
      tmp2 <- tmp2[order(-tmp2$catch), ]

      tmp3 <- data.frame(r2d(tmp2$square))
      geosymbols(tmp3, z = tmp2$catch, fill.circles = TRUE, col = 60,
        levels = lev, colors = my.colors, bordercol = 0, border = TRUE)
      geopolygon(island, col = 30)
      geolines(eyjar, lwd = 3, col = 30)
      j <- tmp2$catch > 1
      if(any(j)) {
        tmp3 <- tmp3[j, ]
        tmp2 <- tmp2[j, ]
        geotext(lat = tmp3$lat, lon = tmp3$lon, z = tmp2$catch,
          angle = 45, csi = 0.1)
      }
      geotext(lat = c(65.2), lon = (-18), z = paste(month.abb[man],
        round(sum(tmp2$catch, na.rm = TRUE)), sep = "\n"), csi = 0.2)
    }
    else {geotext(lat = c(65.2), lon = (-18),
      z = paste(month.abb[man], "0", sep = "\n"), csi = 0.2)}
  }
  geotext(63, -10, ar, adj = 1, csi = 0.18)
  dev.off()
}

## End(Not run)

```

geoplotbwpalette

Black and white palette

Description

Sets palette to 155 or so greys.

Usage

```
geoplotbwpalette()
```

Value

Sets the palette, no value.

Note

Redundant, deprecate?

See Also

Called by [bwps](#).

geoplotpalette

Set define a traditional geoplot-palette

Description

Set define a traditional geoplot-palette with colors from the matrix 'postcol'.

Usage

```
geoplotpalette()
```

Value

Sets the palette

See Also

[colps](#), [hsv](#)

geopoints

Adds points on plots initialized by geoplot.

Description

Plot points on a graph initialized by geoplot. Data is stored as lat, lon or x,y depending on the projection. The program plots the transformation of the data. Parameters for the projection are stored in the list geopar. Similar to the Splus function points.

Usage

```
geopoints(
  lat,
  lon = 0,
  pch = "*",
  cex = 0.7,
  col = 1,
  lwd = 0,
  outside = FALSE,
  jitter = NULL,
  mkh = NULL,
  csi = NULL
)
```

Arguments

lat, lon	Plot points on a graph initialized by <code>geoplot</code> . Data is stored as lat, lon or x,y depending on the projection. The program plots the transformation of the data. Parameters for the projection are stored in the list <code>geopar</code> . Similar to the <code>Splus</code> function <code>points</code> .
pch	Type of symbol used options are for example. " ", "*", "+", "." or anything else, letter, digit, or symbol. Default is "*".
cex	Relative size of character and symbols (see the help on the parameter <code>cex</code>). The size of plotted characters is <code>cex</code> time the parameter <code>csi</code> that can be seen by <code>par("csi")</code> . In earlier versions of <code>geoplot</code> the parameter <code>csi</code> was set but <code>csi</code> is a parameter that can not be set in R. The parameter <code>mkh</code> should probably be used for symbols instead of <code>cex</code> , see help on graphical parameters.
col	Colour number used. Default value is one.
lwd	Linewidth used. Default is the value set when the program was called.
outside	If TRUE <code>geopoints</code> will plot points outside the specified limits set by <code>geoplot()</code> . If FALSE, which is default, outside points will be skipped.
jitter	useful if many datapoints have the same coordinates, points are jittered randomly to make common values look bigger. <code>jitter=0.016</code> is often about right but you may want to have jitter smaller or bigger varying on plot.
mkh	Size of symbol in inches. If not given <code>cex</code> is used instead.
csi	Size of character. This parameter can not be set in R but for compatibility with old <code>Splus</code> scripts the parameter <code>cex</code> is readjusted by <code>cex = cex*csi/0.12</code> . Use of this parameter is not recommended. Default value is NULL i.e not used.

See Also

[geoplot](#), [geopolygon](#), [geolines](#), [points](#), [geotext](#), [geosymbols](#), [geocontour.fill](#), [geogrid](#), [geocontour](#).

Examples

```
## Not run:      geopoints(deg)                # Plots * in the points
                                                         # defined by deg$lat,deg$lon.

geopoints(deg$lat,deg$lon,pch="*",col=5) # Same but uses color 5.

geopoints(fd$x,fd$y)                # Points in x,y when
                                     # projection in geoplot
                                     # was "none".

## End(Not run)
```

`geopolygon`

Fill an area.

Description

The program fills an area. The program is similar to the `polygon` function except the data is in lat, lon and the transform of the data specified in `geoplot` is used. Also there are some additional parameters. The graph has to be initialized by `geoplot`.

Usage

```

geopolygon(
  lat,
  lon = NULL,
  col = "white",
  border = FALSE,
  exterior = FALSE,
  nx = 1,
  outside = FALSE,
  plot = TRUE,
  save = FALSE,
  rat = 0.005,
  density = -1,
  Projection = NULL,
  angle = 45,
  allowed.size = 80000,
  option = 1
)

```

Arguments

lat, lon	Latitude and longitude of data (or x and y coordinates), negative for southern latitudes and western longitudes. May be supplied as two vectors or as a list lat (or x) including vectors latlat and latlon (xx and xy if projection = none).
col	Color number used. Default value is 0 (often white).
border	If TRUE borders around the polygon are drawn. Default value is FALSE.
exterior	If TRUE everything that is outside the polygon is painted , else everything inside. Default value is FALSE. If exterior = TRUE axes and grid often need refreshing by calling geoplot again with new = TRUE.
nx	See geolines for further details.
outside	If TRUE what is outside of the polygon is colored else what is inside. Default value is TRUE.
plot	if TRUE the polygon is plotted. Default is TRUE.
save	if TRUE the points plotted are returned. Default is FALSE.
rat	the ratio of the plot to what is plotted. Default is 0.005 meaning that the plot is 0.5% bigger than what is plotted.
density	see polygon.
Projection	the projection to be used. Default is the one defined by current plot.
angle	see polygon.
allowed.size	printers are limited to printing polygons of certain size, when the polygons size is actually too big for your printer you can enter the tedious task of splitting up the polygon. Default is 4000.
option	Some option. Default 1.

Value

The points plotted are returned if save = TRUE.

See Also

[polygon](#), [geoplot](#), [geolines](#), [geopoints](#), [geotext](#), [geosymbols](#), [geocontour.fill](#), [geogrid](#), [geocontour](#).

Examples

```
## Not run:      geopolygon(island)          # Paint iceland with
                                     # color #0 (often white).

geopolygon(island, col = 0, exterior = TRUE)

geopolygon(geolocator(), col = 1) # Paints a region defined
                                   # by pointing on map black.

# Of the maps available island (iceland) is about the only that
# is correctly defined as closed polygon so it is the only one that
# can be painted by geopolygon.

geoplot(grid = FALSE, type = "n")
# Star by setting up the plot.
geopolygon(gbdypif.500, col = 4, exterior = FALSE, r = 0)
# Use geopolygon to draw the 500 m area.
geopolygon(gbdypif.100, col = 155, exterior = FALSE, r = 0)
# Draw 100 m are over the 500 m.
geolines(eyjar, col = 115)
# Add islands around Iceland.
gbplot(c(100, 500), depthlab = TRUE)
# Draw the depth lines, labels on lines.
geopolygon(island, col = 115, outside = TRUE, r = 0)
# Draw Iceland over.
geoplot(grid = FALSE, new = TRUE)
# Draw lines around Iceland, could also use geolines.

## End(Not run)
```

geoSR

Plot Squares in Different Colors

Description

Plot Icelandic statistical squares in different colors, based on specified levels.

Usage

```
geoSR(sr, z, levels, grid = FALSE, ...)
```

Arguments

sr	squares or subsquares.
z	values for each square.
levels	threshold levels for splitting the values into categories.
grid	passed to geoplot.
...	passed to geoplot.

Details

If `levels` is a named vector, the names are used as level-specific colors, ignoring the last element. The `levels` should go lower and higher than the range of `z` values.

Value

Invisible NULL.

Note

Convenience wrapper for the `reitaPlott` function.

The 3-digit Icelandic statistical squares are 0.5 degrees latitude and 1 degree longitude, defined in the range 60–70°N and 0–50°W.

The bottom-left corner of square 0 is at 60°N 0°. From there, the numbering system is best explained with a table:

	49°W	48°W	...	1°W	0°W
69.5°N	'999'	'998'	...	'951'	'950'
69.0°N	'949'	'948'	...	'901'	'900'
...	...	...	...	...	...
60.5°N	'099'	'098'	...	'051'	'050'
60.0°N	'049'	'048'	...	'001'	'000'

The 4-digit subsquares divide each square into quadrants (NW, NE, SW, SE), appending a number to indicate the quadrant:

```

***1'  ***2'
***3'  ***4'

```

Author(s)

Arni Magnusson.

See Also

[geoplot](#) and [reitaPlott](#) are the underlying drawing functions.
[colorRampPalette](#) can be used to create a ramp of level-specific colors.

Examples

```

# Use existing palette
geoSR(561:560, c(0.3,3), levels=c(0,1,10))

# Pass colors along with levels
lev <- c(0, 1, 10)
names(lev) <- c("brown", "orange", NA)
geoSR(561:560, c(0.3,3), lev)

# Subsquares
geoSR(5611:5612, c(0.3,3), lev)

```

```
# Color ramp
z <- (0:10) / 10
lev <- (0:10) / 10
ramp <- colorRampPalette(c("khaki1", "gold", "orange", "darkorange2", "red",
                           "darkred", "black"))
names(lev) <- ramp(length(lev))
geoSR(724:715, z, lev)
```

geosubplot

Adds a plot to an existing plot initialized by geoplot.

Description

Adds a plot, of any kind, to the current plot at a location specified by the user.

Usage

```
geosubplot(fun, pos, size = c(2, 2), fill, fillcol, ...)
```

Arguments

fun	a call to the function to be subplotted. Be sure to call geoplot with new = T.
pos	Position of the plot. Should include lat and lon. If length(pos\$lat) is 1 that point will determine the middle of the subplot, if length(pos\$lat) is 2 the points will determine opposite corners of the subplot position.
size	The width and the height of the plot if length(pos\$lat) is 1. Default is 2,2.
fill	If true the background of the subplot area will be filled. Default is F.
fillcol	fill colour, the colour of the background of the subplot area, default is 0 (usually white)
...	The program accepts any parameter that the subplot program will accept.

Details

The **geo** functions rely on the parameters in options("geopar") for plotting. Every time geoplot is called a new set of geopar is made and the current erased. If you want to change the parameters in the background plot after you call geosubplot you have to save the current geopar before calling geosubplot. When reassigning geopar, the command options(geopar=list(...)) must be used.

When using geosubplot it is important to save these parameters before the subplot is plotted if one wants to make changes to the large plot afterwards.

Value

none

Side Effects

A plot added to the current plot.

See Also

[subplot](#), [geoplot](#), [geopar](#).

Examples

```
## Not run:
#####
# Example 1                                     #
#####

geoplot()
pos<- list(lat = 63.5,lon=-11.75)
geosubplot(geoplot(faeroes,pch=" ",country =faeroes,new=T)
            ,pos,fill=T)
# Plots the Faeroes on a plot with Iceland. Be sure to
# use new = T if geoplot is called again.

#####
# Example 2                                     #
#####

geoplot()
large.geopar <- geopar                # Parameters saved.
pos <- list(lat=c(63,64),lon=c(-27,-24))
geosubplot(geoplot(island, new=T,grid=F,type="l"),pos)

geotext(65,-18,"subplot")             # Text on subplot.
small.geopar <- geopar # Parameters for subplot saved.

# geopar <- large.geopar             # Make big plot active.
# Unless you are working directly with the .Data dir of the
# geolibrary this assignment will not work, must use:
assign("geopar",large.geopar,where=0)

geotext(65,-18,"Big plot")            # Text on big plot.

# Another subplot.

pos <- list(lat=c(63,64),lon=c(-17,-14))
geosubplot(geoplot(island,new=T,grid=F,type="l"),pos,fill=T)
# Another subplot drawn.
geotext(65,-18,"subplot # 2")

small.geopar.2 <- geopar # parameters for subplot # 2 saved.
# geopar <- large.geopar # Big plot made active again
# Same as above, instead use:
assign("geopar",large.geopar,where=0)

# See also similar example in geopar.

## End(Not run)
```

Description

The function plots different kinds of symbols at the data points defined by lat, lon. There are four categories of symbols:

Usage

```
geosymbols(  
  lat,  
  lon = 0,  
  z,  
  levels = NULL,  
  refllevels = NULL,  
  labels.only = FALSE,  
  cex = 0.6,  
  chs = 0.8,  
  z1 = 0,  
  circles = 0,  
  squares = 0,  
  rectangles = c(0, 0),  
  vbars = 0,  
  hbars = 0,  
  perbars = 0,  
  parbars = 0,  
  sqrt = FALSE,  
  col = 1,  
  maxn = 0,  
  colplot = FALSE,  
  nlevels = 10,  
  colors = 0,  
  n = 25,  
  maxcol = 155,  
  only.positive = FALSE,  
  digits = 0,  
  white = FALSE,  
  lwd = 1,  
  label.location = NULL,  
  labels = 1,  
  fill.circles = FALSE,  
  density = 0,  
  angle = 45,  
  rotate = 0,  
  outside = FALSE,  
  minsym = "<",  
  boundcheck = 0,  
  na.rm = TRUE,  
  label.resolution = 0,  
  characters = FALSE,  
  pch,  
  marks,  
  charcol = 0,  
  open.circles = FALSE,  
  col.names = c("lat", "lon"),
```

```

border = FALSE,
bordercol = 0
)

```

Arguments

lat, lon	latitude and longitude of data or a dataframe containing latitude and longitude of data (or x and y coordinates), negative for southern latitudes and western longitudes. Expected to contain lat and lon if not otherwise specified in col.names.
z	Matrix containing values at datapoints.
levels	Values at contourlines. Default value is zero. If levels is zero the program determines the contourlines from the data. If squares, circles, hbars, vbars or perbars are being used levels corresponds to the levels where labels are given.
reflevels	i Some levels for reference?
labels.only	if true only labels are plotted. Default is false.
cex	Size expansion of digits.
chs	Something to do with characters?
z1	Value 2 at data points. Only used in connection with rectangles.
circles	Max size of circles plotted at data points. Default value used if value <-0 or >100. Size is either proportional to z or sqrt(z).
squares	Max size of squares plotted at data points. Default value used if value <-0 or >100. Size is either proportional to z or sqrt(z).
rectangles	Max size of rectangles plotted at datapoints in inches. The first number gives max height and the second number max width. Values < 0 or > 100 give default values.
vbars	Max size of vertical bars at data points in inches. Values >100 give default values. Value <0 gives bars below points.
hbars	Max size of horizontal bars at data points in inches. Values >100 give default values. Value <0 gives bars left of points.
perbars	Max size of bars perpendicular to transect lines in inches. Values >100 give default values. Value <0 gives different orientation.
parbars	Same as perbars, except the bars are now parallel.
sqrt	If sqrt = TRUE the size of symbol is proportional to sqrt(z) else to z. Default value if FALSE.
col	Color number used, default col = 0.
maxn	If nonzero maxn is the base for the size of symbols, else max(z) is used. Nonzero maxn is used if several plots are to be compared.
colplot	if true range of z is specified by a colour and not size.
nlevels	Number of contourlines. Used if the program has to determine the contourlines. Default value is 10.
colors	Color number for the contourlines. Runs from 0 to 155. A vector one longer than the vector levels. Default is blue-green-yellow- red from lowest to the highest values. Color 0 is white and 1 is black. On black and white plots higher color number means darker color and also when hatching. When hatching the useful range is 10 to 80. When plotting filled circles of different sizes colors means sizes in inches. When levels are not specified directly colors have to be found by the program because even though nlevels = 5 the length of levels can be 7 due to characteristics of the Splus pretty command.

n	Number of vertices in each circle, default value is 25, this parameter is rarely changed by the user.
maxcol	Number of colors used (excluding #0). Default value is 155
only.positive	Logical value. If FALSE then negative values are allowed else negative values are set to zero. Default value is FALSE.
digits	Number of digits used in labels. Default value is zero.
white	If true the first color is white.
lwd	Line width for symbols. Default value is the value when the program was called.
label.location	List with components lat and lon specifying opposite corners of a square where the label should be put. (or x, y) Gives the lower left and upper right corner of the box where the labels are put. Default value is 0 that means no labels are put on the drawing or if geoplot was initialized with cont = TRUE, then labels are put on the left side of the plot. label.location is best given by geolocator or directly by specifying label.location = "locator".
labels	Type of labels 1 or 2. One is default and is usually used except in color with very many colors (more than 10-20) .
fill.circles	If TRUE filled circles of different sizes are plotted. Size of the circles is given directly by the parameter color or found from the data. (maxcol corresponds to the size of the largest circle in this case and is 0.1 by default (changing maxcol to 0.4 makes all the circles 4 times larger.)
density	If density is 1 (or not zero) circles are hatched instead of having different color. Only available with circles. Color does in this case specify the density of hatching. Higher number means denser hatching. The range is from zero to maxcol (155). But the effective range is ca. 10 - 80.
angle	Angle of hatching, default is 45 degrees.
rotate	Rotation of hatching from one level to the next. Default value is 0 but 45 or 90 can be good to better distinguish between different levels.
outside	If TRUE geosymbols will plot outside the specified limits set by geoplot(). If FALSE, which is default, outside points will be skipped.
minsym	Minimum symbol, default is "<", meaning that if levels = c(1, 2), labels will be presented as < 1, 1-2, 2 <, but if minsym = " " labels will be presented as 1, 1-2, 2. See also labels.resolution.
boundcheck	If boundcheck != 0 those points which are out of bounds are returned to the user, if boundcheck = 2 the points are also not plotted, default is boundcheck = 0.
na.rm	If true NA's are removed, default is true.
label.resolution	the resolution (precision) of the label numbers, default is 0, meaning that if levels = c(1, 2), labels will be presented < 1, 1-2, 2 <, if label.resolution = 0.1 labels will be presented < 1, 1.1-2, 2.1<, see also minsym. If label.resolution = "none", the labels will present the lowest number of the interval with each color.
characters	A boolean variable determining whether characters are to be plotted.
pch	Type of symbols for each level.
marks	Type of symbols for each level. The difference between marks and pch is because when making points on a plot the user can either give pch = 17 or pch = "A". The former type is called marks here but the latter pch. Marks have to be given for each level and set to -1 where pch is to be used. The length of the vector
charcol	The color of the characters, default is the same as col.

<code>open.circles</code>	Should <code>open.circles</code> be plotted. Default FALSE. NEEDS CHECKING.
<code>col.names</code>	Column names with positions. Default <code>lat</code> , <code>lon</code> .
<code>border</code>	Should border be plotted? Default FALSE. NEEDS CHECKING.
<code>bordercol</code>	Color of border. Default 0. NEEDS CHECKING.

Details

Default Shapes whose size is proportional to `z` or `sqrt(z)`.

Categories Shapes where certain color, shading or size represents certain range of `z`. Similar to contour program.

Filled circles Certain size represents certain range of `z`, specified with `fill.circles = TRUE`.

Characters Characters or character strings represent the different ranges of `z`, specified with `characters = TRUE`.

There are seven types of shapes: circles, squares, rectangles, `vbars` (vertical bars), `hbars` (horizontal bars), `perbars` (perpendicular bars), `parbars` (parallel bars)

Value

No values are returned

See Also

[geoplot](#), [geopolygon](#), [points](#), [geotext](#), [geopoints](#), [geocontour.fill](#), [geogrid](#), [geocontour](#).

Examples

```
## Not run:      # lodna.2 composes of echo measurements for capelin on the
                  # norther- and easternshores of Iceland. [lat, lon, z]

# Show points.

geoplot(lodna.2, type = "l", grid = FALSE) # Begin by plotting Iceland.
geopoints(lodna.2, pch = "*", col = 150)  # See where the points are.

#####
# Example 1, color parbars plot. #
#####

geoplot(lodna.2, type = "l", grid = TRUE)    # Begin by plotting Iceland.
levels = c(0, 20, 50, 100, 500, 1000)

geosymbols(lodna.2, z = lodna.2$z, colplot = TRUE, colors = seven.col,
            levels = levels, parbars = 0.05, colors = seven.col,
            label.location = "locator")

# "locator" click twice on the map where you want the contour index.
# Indicate firstly the upper left corner position then lower right.

#####
# Example 2, black/white perbars plot.#
#####

geoplot(lodna.2, type = "l", grid = FALSE)
```

```

geosymbols(lodna.2, z = lodna.2$z, perbars = 0.1)

# Bars perpendicular to measurement direction

#####
# Example 3, Color Dots.          #
#####

# Set up data.
attach("/usr/local/reikn/SplusNamskeid")
i<-utbrteg$ar == 2004

# Set up the plot.
geoplot()
levels = c(10, 100, 500)
colors = c(13, 55, 111, 153)
labloc<-list(lat = c(63.95, 65.4), lon = c(-19.8, -17.3))

geosymbols(utbrteg[i, ], z = utbrteg[i, "torskur.kg"], circles = 0.05,
           sqrt = TRUE, colplot = TRUE, levels = levels, colors = colors,
           label.location = labloc)

#####
# Example 4, Rings around points.  #
#####

# Having done the set up data and plot in Example 3.

geoplot(utbrteg$lat, utbrteg$lon, pch = ".")
geosymbols(utbrteg[i, ], z = utbrteg[i, "torskur.kg"], circles = 0.2,
           sqrt = TRUE, label.location = labloc)

# Circles can be replaced with squares, rectangles, vbars, hbars or
# perbars or more than one used simultaneously.

## End(Not run)

```

geotext

Plots text on a drawing defined by geoplot.

Description

The function plots a text in each of the points defined by lat,lon. (or x,y) The value plotted at each point is defined by the numeric vector or text vector z. Similar to text but allows lat and lon and a choice of digits used.

Usage

```

geotext(
  lat,
  lon = 0,
  z,
  cex = 0.7,

```

```

adj = 0.5,
col = 1,
digits = 0,
pretext = "",
lwd = 0,
aftertext = "",
outside = F,
angle = 0,
jitter = NULL,
csi = NULL
)

```

Arguments

lat, lon	Latitude and longitude of data (or x and y coordinates), negative for southern latitudes and western longitudes. May be supplied as two vectors or as a list lat (or x) including vectors latlat and latlon (xx and xy if projection = none).
z	Vector with values that will be plotted at datapoints. Has to be of the same length as lat. If z is of mode character it is written directly on the screen.
cex	Relative size of characters (see the help on the parameter cex). The size of plotted characters is cex time the parameter csi that can be seen by par()csi. In earlier versions of geoplots the parameter csi was set but csi is a parameter that can not be set in R.
adj	Location of the text relative to the point, 0 means text right of point, 0.5 text centered at point and 1 text left of point. Default value is 0.5.
col	Color number used. Default value is 1.
digits	Number of digits used.
pretext	Text put in front of all the text. Default value is nothing.
lwd	Linewidth used.
aftertext	Text put after all the text. Default value is nothing.
outside	If outside is F no text is plotted outside the range defined by xlim,ylim. Else it is done. Default value is F.
angle	angle of text in degrees, default is 0.
jitter	see jitter in geoplots.
csi	Size of character. This parameter can not be set in R but for compatibility with old Splus scripts the parameter cex is readjusted by $cex = cex * csi / 0.12$. Use of this parameter is not recommended. Default value is NULL i.e not used.

Value

No values returned.

See Also

[geoplots](#), [geopolygons](#), [geolines](#), [geosymbols](#), [geogrid](#), [geopar](#), [geocontour.fill](#), [geolocator](#), [geocontour](#).

Examples

```
geoplot()
deg <- data.frame(lon = rnorm(10,-27,1.3),lat = rnorm(10,65,0.6))
z <- letters[1:10]
geotext(deg,z=z)      # plot text at points deg$lat,deg$lon

      geotext(deg$lat,deg$lon,z,csi=0.06) # Same, size of text 0.06".

x <- deg
names(x) <- c('y','x')
x$z <- z
      geotext(x$x,x$y,x$z,aftertext="km",pretext="distance")
      # If geopar$projection="none"

      geotext(x$x,x$y,z=x$z,aftertext=" km",pretext="distance",angle = 90)
      # Same text written vertically.

#####
# Example                                     #
#####

lon <- rnorm(10,-27,1.3)
lat <- rnorm(10,65,0.6)
# Make a normal dist. random set of 10 points.

geoplot(lat=lat,lon=lon,grid=FALSE,xlim=c(-22,-30),ylim=c(63,67))
# Plot the random data points.

geopolygon(island,col=115,exterior=TRUE)
geolines(island)
# Color Iceland. Use litir(number) to see colour scheme.
# Sharpen lines around Iceland.

num <- 1:10
lab <- paste("Nr.",num,sep="")
# Make string vector with "Nr.1".. "Nr.10" for geotext.

geopoints(lat,lon,pch="*",col=5)
# Redraw the data in a new color the * mark at points.

geotext(lon=lon,lat=lat,z=lab,col=155)
# With geotext we put one element from lab at each data point.
title(main="10 Random Data Point")
# Add title
```

geotows

Plot tows as line segments

Description

The function gets 4 arguments i.e position of begininning and end of segments. If the first argument is given col.names gives the column names in the data frame describing the position.

Usage

```
geotows(
  lat,
  lon,
  lat1,
  lon1,
  col = 1,
  col.names = c("kastad.n.breidd", "kastad.v.lengd", "hifft.n.breidd", "hifft.v.lengd"),
  ...
)
```

Arguments

<code>lat</code>	Vector of start position latitudes, dataframe of start positions given in columns <code>lat</code> and <code>lon</code> or dataframe with start and end positions given as columns <code>col.names</code> .
<code>lon</code>	When given, vector of start position longitudes.
<code>lat1</code>	When given, vector of end position latitudes or a data frame of end positions given in columns <code>lat</code> and <code>lon</code> .
<code>lon1</code>	When given, vector of end position longitudes.
<code>col</code>	Color of line segments.
<code>col.names</code>	Column names of start lat and lon and end lat and lon when <code>tows</code> are given in a single dataframe. Defaults to column names in the Hafro/MRI database table <code>fiskar.stodvar</code> .
<code>...</code>	Additional arguments to <code>geolines</code> .

See Also

[geolines](#)

geoworld

Plots rough outline of the world.

Description

Plots roughly the outlines of those countries which have borders intersecting the current plot initialized by `geoplot`. The countries can also be filled.

Usage

```
geoworld(
  regions = ".",
  exact = FALSE,
  boundary = TRUE,
  fill = FALSE,
  color = 1,
  lwd = 1,
  lty = 1,
  plot = TRUE,
  type = "l",
```

```

    pch = ".",
    database = "world",
    return.data = FALSE,
    outside = FALSE,
    allowed.size = 80000
  )

```

Arguments

regions	Allows plotting only a certain part of the world to speed up the function, such as regions = "iceland". Default is plotting what is inside the map.
exact	Draws a more exact plot.
boundary	If true country boundaries are drawn. Default is false.
fill	If true countries are filled with a color. Default is false.
color	Default is 1 (usually black).
lwd	Linewidth, default is 1.
lty	Linetype, default is 1.
plot	If false a plot is not drawn. Default is true.
type	If lines or points should be plotted, see type for geoplots.
pch	The character to be used for plotting, see par.
database	The database the world plot is to be taken from. Currently there are two possible databases world and worldHires. world is in the map library and worldHires in the mapdata library and is much more precise.
return.data	If true those points used to make the in the plot are return, default is FALSE. Can be used to save the coastlines of specified countries.
outside	If TRUE data are plotted outside the borders. Default is FALSE.
allowed.size	Maximum size of polygon that can be filled. Default is 80 000 which is rather large polygon but this value is rapidly changing by more powerful hardware.

Value

default none, see option return data.

Side Effects

the outlines of all countries who intersect current plot are drawn.

See Also

[geoplot](#), [fill.outside.border](#), [geopolygon](#), [par](#).

Examples

```

geoplot(xlim = c(0, -53), ylim = c(53, 75))
geoworld()

# Should plot in all countries who intersect the plot draw
# with geoplot.

# The packages maps and mapdata need to be installed

```

```
# worldHires is a very detailed database of coastlines from the
# package mapdata. Could be problematic if used with fill = TRUE)
# Allowed.size is the maximum allowed size of polygons.
library(maps) # world coastlines and programs
library(mapdata) # more detailed coastlines
geoplot(xlim = c(20, 70), ylim = c(15, 34))
geoworld(database = "worldHires", fill = TRUE, col = 30, allowed.size = 1e5)

geoplot(xlim = c(20, 70), ylim = c(15, 34), dlat = 10, dlon = 10)
geoworld(database = "world", fill = TRUE, col = 30) #

geoplot(xlim = c(-10, 70), ylim = c(71, 81), b0 = 80,
        dlat = 2, dlon = 10) # 0 must be high here else
geoworld(database = "world", fill = TRUE, col = 30) #the plot fails.

# Lambert projection,
geoplot(xlim = c(-10, 70), ylim = c(71, 81),
        dlat = 2, dlon = 10, projection = "Lambert")
geoworld(database = "world", fill = TRUE, col = 30)
```

geozoom

Zoom into plots.

Description

Zoom into current geoplot, redraws current geoplot with area defined by user. The current plot can be restored by using geodezoom.

Usage

```
geozoom()
```

Value

none.

Side Effects

The last call to geoplot is recalled with different borders for x and y.

See Also

[geoplot](#), [geodezoom](#).

Examples

```
geoplot()
geozoom()
# Click with mouse as when placing legend i.e. first place the
# mouse where the upper left corner is supposed to be and click
# once, then move to where the lower right corner is supposed to
# be and also press once.
geodezoom()
```

```
# Return to previous plot, here geoplot().
```

gepco500	<i>GEBCO 500 m isobath extending around the Faroes.</i>
----------	---

Description

GEBCO 500 m isobath extending around the Faroes.

Format

A data frame with 679 observations on the following 2 variables.

lat a numeric vector

lon a numeric vector

Examples

```
data(gepco500)
## maybe str(gepco500) ; plot(gepco500) ...
```

giveborder	<i>Define regions interactively</i>
------------	-------------------------------------

Description

Define regions interactively.

Usage

```
giveborder(nholes = 0)
```

Arguments

nholes The number of holes in the data, number of regions - 1

Value

Returns a composite list of the following compents:

reg	List with components number of components (?)
x, y	Coordinates when <code>geopar\$projection == "none"</code>
lat, lon	Geographical coordinates for other projections (Mercator and Lambert ?)
lxv	ID of region (?)

Note

Needs further elaboration.

See Also

Called by [geodef](#), calls [invProj](#).

glaciers

Outlines of Icelandic glaciers.

Description

Outlines of Icelandic glaciers.

Format

A data frame with 6469 observations on the following 2 variables.

lat a numeric vector

lon a numeric vector

Examples

```
data(glaciers)
## maybe str(glaciers) ; plot(glaciers) ...
```

greenland

Outline of Greenland.

Description

Outline of Greenland.

Format

A data frame with 62416 observations on the following 2 variables.

lat a numeric vector

lon a numeric vector

Examples

```
data(greenland)
## maybe str(greenland) ; plot(greenland) ...
```

gridaxes

*Set up axes for a geoplot***Description**

Set up axes for a geoplot.

Usage

```
gridaxes(
  limx,
  limy,
  scale,
  b0,
  xyratio,
  grid,
  col,
  reitur,
  smareitur,
  axratio,
  axlabels,
  b1,
  l1,
  projection,
  dlat,
  dlon
)
```

Arguments

limx	Longitude limits
limy	Latitude limits
scale	Scale to supply to Proj, invProj
b0	Base latitude for the Mercator projection. Default value is 65 (typical for Iceland)
xyratio	Argument that can be used to set the aspect ratio (?)
grid	If grid is TRUE meridians and parallels are plotted, else not. Default TRUE
col	Color of gridlines
reitur	Should the grid show statistical rectangles?
smareitur	Should the grid show statistical sub-rectangles?
axratio	Parameter usually not changed by the user (?)
axlabels	If FALSE no numbers are plotted on the axes. Default TRUE
b1	Second latitude to define Lambert projection (?, as there's a Lambert version of this function)
l1	The longitude defining the Lambert projection, default is the l1 defined in geopar, ditto ??
projection	Projection ?, other than Mercator ??)
dlat	Latitude axis increment between labels
dlon	Longitude axis increment between labels

Value

No value, useful because of its side effect of adding axes to a geoplot.

Note

May need further elaboration or simplification (referring to the general geoplot help). Coordinate with the geoaxes.Lambert and gridaxes helpfiles (supply as one??). Why are the dlat and dlim returned for this one?

See Also

Called by [init](#), calls [invProj](#), [Proj](#), [mercator](#) and [plot_nogrid](#).

gridaxes.Lambert

Set up axes for a Lambert projected geoplot

Description

Set up axes for a Lambert projected geoplot.

Usage

```
gridaxes.Lambert(
    limx,
    limy,
    scale,
    b0,
    xyratio,
    grid,
    col,
    reitur,
    smareitur,
    axratio,
    axlabels,
    b1,
    l1,
    projection,
    dlat,
    dlon,
    col1 = 1
)
```

Arguments

limx	Longitude limits
limy	Latitude limits
scale	Scale to supply to Proj, invProj
b0	Base latitude for the Lambert projection. Default value is 65 (typical for Iceland)
xyratio	Unused argument (?)

grid	If grid is TRUE meridians and parallels are plotted, else not. Default TRUE
col	Color of gridlines
reitur	Should the grid show statistical rectangles?
smareitur	Should the grid show statistical sub-rectangles?
axratio	Parameter usually not changed by the user (?)
axlabels	If FALSE no numbers are plotted on the axes. Default TRUE
b1	Second latitude to define Lambert projection
l1	The longitude defining the Lambert projection, default is the l1 defined in geopar
projection	Projection (but is the function meaningful for other than Lambert?)
dlat	Latitude axis increment between labels
dlon	Longitude axis increment between labels
col1	Color of axes and labels, default 1 (but might be col

Value

No value, useful because of its side effect of adding axes to a Lambert geo-plot.

Note

May need further elaboration or simplification (referring to the general geoplots help). Coordinate with the geoaxes and gridaxes helpfiles (supply as one??).

See Also

Called by [init](#), calls [adjust.grd](#), [cut_box.1](#), [cut_box.2](#), [fill.points](#), [invProj](#), [Proj](#).

gridpoints	<i>Produce gridpoints over an area</i>
------------	--

Description

Produce gridpoints over an area.

Usage

```
gridpoints(border, dx, grpkt, nx, n)
```

Arguments

border	Border of the area
dx	Resolution in each direction (?)
grpkt	???
nx	Number of gridpoints in each direction (?)
n	Total number of gridpoints (?)

Value

List with components:

xgr	List of gridpoints in components lat, lon or x, y depending on the projection.
xgra	List of those gridpoints within the area given in border

Note

Needs further elaboration, check use with find = TRUE in setgrid.

See Also

Called by [setgrid](#).

icelandrivers	<i>Icelandic rivers.</i>
---------------	--------------------------

Description

Icelandic rivers, from gshhg probably.

Format

A data frame with 1124 observations on the following 2 variables.

lat a numeric vector

lon a numeric vector

Details

~~ If necessary, more details than the __description__ above ~~

Examples

```
data(icelandrivers)
## maybe str(icelandrivers) ; plot(icelandrivers) ...
```

ices

*ICES Areas***Description**

Draw a map showing ICES areas 1–14.

Usage

```
ices(
  labels = "roman",
  diagrams = FALSE,
  col = "black",
  lwd = 3,
  col.lines = "grey80",
  font = 2,
  col.text = "orangered",
  cex = 1
)
```

Arguments

labels	is how to annotate areas: "roman" (I–XIV), FALSE (no labels), vector (custom), otherwise 1–14.
diagrams	is whether to show diagrams on console.
col	is passed to map.
lwd	is passed to lines.
col.lines	is passed to lines.
font	is passed to text.
col.text	is passed to text.
cex	is passed to text.

Value

Invisible data frame with coordinates.

Note

The coordinates were inferred from official ICES maps.

Author(s)

Arni Magnusson.

Examples

```
ices()
```

init	<i>Initiate a geoplot (??)</i>
------	--------------------------------

Description

Initiate a geoplot (??).

Usage

```
init(  
  lat,  
  lon = 0,  
  type = "p",  
  pch = "*",  
  xlim = c(0, 0),  
  ylim = c(0, 0),  
  b0 = 65,  
  r = 1.05,  
  xlab = "Longitude",  
  ylab = "Latitude",  
  option = "cut",  
  grid = T,  
  new = F,  
  cont = F,  
  cex = 0.7,  
  col = 1,  
  lcont = c(0.13, 0.21),  
  plotit = T,  
  reitur = F,  
  smareitur = F,  
  reitttext = F,  
  axratio = 1,  
  lwd = 0,  
  axlabels = T,  
  oldpar,  
  projection = "Mercator",  
  b1 = 65,  
  dlat = 0,  
  dlon = 0,  
  command = 0,  
  jitter = 0,  
  xaxdist,  
  yaxdist  
)
```

Arguments

lat, lon	Latitude and longitude
type	Plot method
pch	Plot character

xlim, ylim	Plot limits
b0	Base latitude
r	Plot ratio, multiplier on <code>diff(x_or_ylim)</code>
xlab, ylab	Labels for x- and y-axes, default "Longitude", "Latitude"
option	Method of determining plot extent, default "cut"
grid	Should a grid be drawn, default TRUE
new	Plot control, default FALSE adds plot to current plot
cont	For use with contours: should space be reserved for contour labels? Default FALSE
cex	Character size expansion
col	Color, default 1, usually black
lcont	Contour label space allocation, default <code>c(0.13, 0.21)</code>
plotit	If FALSE plot is only initialized but not plotted. If used other programs are used to fill the plot (geolines, geocontour, geopolygon etc). Most often used in multiple plots.
reitur	Should the grid be that of statistical rectangles?
smareitur	Should the grid be that of statistical sub-rectangles?
reittext	Should the rectangles be labelled?
axratio	Parameter usually not changed by the user.
lwd	Line width
axlabels	If FALSE no numbers are plotted on the axes. Default value is TRUE.
oldpar	The old par-list, from the parent <code>geoplot</code> -call
projection	Projection, default Mercator
b1	Second latitude for Lambert projection
dlat, dlon	Defines the grid, to make a grid on the axes, 1 is a number on axis and a line at every deg. Not usually set by user.
command	The parent <code>geoplot</code> command is included, although for what purpose isn't quite clear??
jitter	Random jitter to accentuate repeated values. Default no jitter (<code>jitter = 0</code>)
xaxdist, yaxdist	Distance from plot to the labels on axes (<code>dist</code> or <code>r</code> argument to <code>geoaxis</code>). Default values 0.2, 0.3 but higher values mean that <code>axlabels</code> are further away from the plot. Further flexibility with axes can be reached by calling <code>geoplot</code> with <code>axlabels = FALSE</code> and <code>geoaxis</code> afterwards.

Value

No value, side effect plotted.

Note

Needs further elaboration, alternatively hide the function.

See Also

Called by `geoplot` calls `Elimcomp`, `findline`, `geoaxis`, `gridaxes`, `gridaxes.Lambert`, `invProj`, `Proj`.

inside	<i>Finds a subset of data inside (or outside) a region</i>
--------	--

Description

Finds a subset of data within or without a region, returned as submatrix, boolean vector or indices of the original data.

Usage

```
inside(lat, lon = 0, reg, option = 1, projection = "Mercator")
```

Arguments

lat, lon	Latitude and longitude of the data, if lat = 0, lat must have components lat, lon or x, y in case projection is "none".
reg	Region we want to check wheter or not includes the data.
option	How should the results be returned: 1 Submatrix of data inside reg 2 Submatrix of data outside reg 3 Boolean vector, TRUE for data inside reg 4 Boolean vector, TRUE for data outside reg 5 Indices of data inside reg 6 Indices of data outside reg
projection	Projection, default "none"

Value

Submatrix, boolean vector or index to data as described for argument option.

Note

Needs elaboration, why is this needed in addition to geoinside?

See Also

Called by [combine.rt](#), [setgrid](#), calls [adapt](#).

inside.reg.bc	<i>Determine which bormicon (or gadget) region data belong to.</i>
---------------	--

Description

Determine which bormicon (or approximately gadget) region data belong to.

Usage

```
inside.reg.bc(data, only.sv.1to10 = F, ignore.latlon = F, ignore.area0 = T)
```

Arguments

data	Data set with coordinates in components lat, lon.
only.sv.1to10	Label only data belonging to areas 1 to 10 (depth < 500 m). Default FALSE
ignore.latlon	Use rectangle resolution ???
ignore.area0	Action for data not belonging to bc-areas, not used???

Value

Returns original data with area code as an added component area.

Note

Needs elaboration, esp. when based on rectangles, merge doc with inside.reg.bc1.

See Also

Calls [inside.reg.bc1](#), [Reitur2Svaedi1to10](#), [sr2d](#). Data set [reg.bc](#) with bormicon-area outlines is used.

inside.reg.bc1	<i>Determine which bormicon (or gadget) region data belong to.</i>
----------------	--

Description

Determine which bormicon (or approximately gadget) region data belong to.

Usage

```
inside.reg.bc1(data)
```

Arguments

data	Data set with coordinates in components lat, lon.
------	---

Value

Returns original data with area code as an added component area.

Note

Needs further elaboration ?

See Also

Calls [geoinside](#), called by [inside.reg.bc](#) and [Reitur2Svaedi1to10](#). Data set [reg.bc](#) with bormicon-area outlines is used.

inside.reg.lump	<i>Inside region in lumpsucker fishery</i>
-----------------	--

Description

Finds regulation area from a position given in dataframe

Usage

```
inside.reg.lump(data)
```

Arguments

data	Data frame with columns lat and lon givin postions
------	--

Details

to be added later

Value

Original data with column area added, with index to list of regulation areas in [reg.lump](#).

Note

Could be repeated for other area divisions.

Author(s)

STJ

References

Borrows from [inside.reg.bc](#).

See Also

[inside.reg.bc](#), [reg.lump](#).

Examples

```
##---- Should be DIRECTLY executable !! ----
##-- ==> Define data, use random,
##-- or do help(data=index) for the standard data sets.

## The function is currently defined as
function (data)
{
  if (nrow(data) > 1)
    tmpdata <- data[, c("lat", "lon")]
  else tmpdata <- as.data.frame(data[, c("lat", "lon")])
  tmpdata$area <- rep(0, nrow(tmpdata))
  i <- 1
  ind <- geoinside(tmpdata, reg = reg.lump[[i]], option = 0,
    robust = FALSE)
  if (length(ind) > 0)
    tmpdata[ind, "area"] <- i
  i <- 2
  j <- tmpdata$area == 0
  j1 <- c(1:length(j))
  j1 <- j1[j == T]
  if (length(j1) > 0) {
    ind <- geoinside(tmpdata[j1, ], reg = reg.lump[[i]],
      option = 0, robust = FALSE)
    if (length(ind) > 0)
      tmpdata[j1[ind], "area"] <- i
  }
  i <- 3
  j <- tmpdata$area == 0
  j1 <- c(1:length(j))
  j1 <- j1[j == T]
  if (length(j1) > 0) {
    ind <- geoinside(tmpdata[j1, ], reg = reg.lump[[i]],
      option = 0, robust = FALSE)
    if (length(ind) > 0)
      tmpdata[j1[ind], "area"] <- i
  }
  i <- 4
  j <- tmpdata$area == 0
  j1 <- c(1:length(j))
  j1 <- j1[j == T]
  if (length(j1) > 0) {
    ind <- geoinside(tmpdata[j1, ], reg = reg.lump[[i]],
      option = 0, robust = FALSE)
    if (length(ind) > 0)
      tmpdata[j1[ind], "area"] <- i
  }
  i <- 5
  j <- tmpdata$area == 0
  j1 <- c(1:length(j))
  j1 <- j1[j == T]
  if (length(j1) > 0) {
    ind <- geoinside(tmpdata[j1, ], reg = reg.lump[[i]],
      option = 0, robust = FALSE)
    if (length(ind) > 0)
      tmpdata[j1[ind], "area"] <- i
  }
}
```

```

}
i <- 6
j <- tmpdata$area == 0
j1 <- c(1:length(j))
j1 <- j1[j == T]
if (length(j1) > 0) {
  ind <- geoinside(tmpdata[j1, ], reg = reg.lump[[i]],
    option = 0, robust = FALSE)
  if (length(ind) > 0)
    tmpdata[j1[ind], "area"] <- i
}
i <- 7
j <- tmpdata$area == 0
j1 <- c(1:length(j))
j1 <- j1[j == T]
if (length(j1) > 0) {
  ind <- geoinside(tmpdata[j1, ], reg = reg.lump[[i]],
    option = 0, robust = FALSE)
  if (length(ind) > 0)
    tmpdata[j1[ind], "area"] <- i
}
i <- 8
j <- tmpdata$area == 0
j1 <- c(1:length(j))
j1 <- j1[j == T]
if (length(j1) > 0) {
  ind <- geoinside(tmpdata[j1, ], reg = reg.lump[[i]],
    option = 0, robust = FALSE)
  if (length(ind) > 0)
    tmpdata[j1[ind], "area"] <- i
}
data$area <- tmpdata$area
return(data)
}

```

intra.point.dist	<i>Intra point/position distance</i>
------------------	--------------------------------------

Description

For a data frame of positions return the vector of intra point distances

Usage

```
intra.point.dist(x)
```

Arguments

x List of positions with components lat and lon.

Value

Vector of distances between the points in x.

See Also[arcdist](#)**Examples**

```
# distances along the perimeter of a statistical rectangle
pos <- rPeri(323)
intra.point.dist(pos)
sum(intra.point.dist(pos))
```

invlambert

*Inverse Lambert projection***Description**

Inverse Lambert projection.

Usage

```
invlambert(x, y, lat0, lon0, lat1, scale = "km", old = F)
```

Arguments

x, y	The input data to be inversely projected as two vectors
lat0	First latitude defining of the projection
lon0	Longitude defining the projection
lat1	Second latitude defining the projection
scale	Scale of the input data, "km" default, redundant ??
old	Old method, seldom used???

Value

List of components:

lat, lon	Coordinates in latitude and longitude
x, y	Input coordinates (projected)
scale	Scale
projection	Projection (stated redunantly)
lat0, lon0, lat1	Defining lats and lon

is returned invisibly.

Note

Needs elaboration and perhaps documenting with lambert in the same doc-file.

See Also

Called by [invProj](#).

invmerc	<i>Inverse Mercator Projection</i>
---------	------------------------------------

Description

Inverse Mercator Projection.

Usage

```
invmerc(x, y, scale = "km", b0 = 65)
```

Arguments

x, y	The input data to be inversely projected as two vectors
scale	Scale of the input data, "km" default, all other values imply nautical miles.
b0	Latitude defining the projection.

Value

List of components:

lat, lon	Coordinates in latitude and longitude
x, y	Input coordinates (projected)
scale	Scale
projection	Projection (stated redunantly)
b0, L0	Defining lat and a null value ???

is returned invisibly.

Note

Needs elaboration and perhaps documenting with mercator in the same doc-file.

See Also

Called by [invProj](#).

invProj

*Performs the inverse Mercator or Lambert projection of data.***Description**

Accepts data in spherical coordinates with center latitude 0 and longitude 0 and performs an inverse Mercator, Lambert or no transformation.

Usage

```
invProj(
  x,
  y = NULL,
  scale = getOption("geopar")$scale,
  b0 = getOption("geopar")$b0,
  b1 = getOption("geopar")$b1,
  l1 = getOption("geopar")$l1,
  projection = getOption("geopar")$projection
)
```

Arguments

x, y	The input data to be inversely projected, may be given as two vectors or as list attributes (x and y).
scale	The scale of the input data (m, km or miles), default is the scale defined in geopar (the scale defined when the plot is initialized).
b0	if projection = Mercator b0 is the center of the Mercator projection. If projection = Lambert b0, b1 are the latitudes defining the Lambert projection. Default are the b0 and b1 defined in geopar.
b1	Second defining latitude for Lambert projection.
l1	The longitude defining the Lambert projection, default is the l1 defined in geopar.
projection	The projection to be inverted, legal projections are "mercator", "Lambert" and "none". Default is the projection defined in geopar.

Value

The function returns a list containing if projection = "none" x and y, if projection is mercator or Lambert it includes the projection (projection), the scale (scale), lat and lon and x and y.

See Also

[invProj](#), [geopar](#), [geoplot](#).

island	<i>Icelandic coastline.</i>
--------	-----------------------------

Description

Icelandic coastline from unknown source.

Format

A data frame with 1323 observations on the following 2 variables.

lat a numeric vector

lon a numeric vector

Source

Unknown.

labels1	<i>Label plots</i>
---------	--------------------

Description

Label plots with categories.

Usage

```
labels1(  
  cont,  
  digits,  
  colors,  
  xlim = c(0, 1),  
  ylim = c(0, 1),  
  fill = F,  
  minsym = "<",  
  label.resolution = 0,  
  labtxt = NULL,  
  first.color.trans = T,  
  mai = c(0, 1, 0, 1),  
  leftrat = 0.1  
)
```

Arguments

cont	Contour ?
digits	Number of digits to use in labels
colors	Colors ?
xlim, ylim	Limits ?
fill	Fill with colors?

minsymb	Minimum symbol (for lowest category?)?
label.resolution	Label resolution ?
labtxt	Label text ?
first.color.trans	Should first color be transparent? Default TRUE
mai	Margins in inches?
leftrat	Left ratio (giving space for labels??)??

Value

No value, labels added to current plot.

Note

Needs elaboration, merge documentation with labels2, and possibly others?

See Also

Called by [colsymbol](#), [geocontour.fill](#) and [reitaplott](#).

labels2	<i>Label plots</i>
---------	--------------------

Description

Label plots with categories.

Usage

```
labels2(cont, digits, colors, xlim = c(0, 1), ylim = c(0, 1), nx = 4, fill = F)
```

Arguments

cont	Contour ?
digits	Number of digits to use in labels
colors	Colors ?
xlim, ylim	Limits ?
nx	?. Default 4
fill	Fill ?

Value

No value, labels added to current plot.

Note

Needs elaboration, merge documentation with labels1, and possibly others?

See Also

alled by [colsymbol](#) and [geocontour.fill](#).

labels_line	<i>Labels line ?</i>
-------------	----------------------

Description

Labels line ?.

Usage

```
labels_line(  
  cont,  
  digits,  
  colors,  
  lty,  
  lwd,  
  xlim = c(0, 1),  
  ylim = c(0, 1),  
  linewidth = F  
)
```

Arguments

cont	Contours?
digits	Number of digits
colors	Colors
lty	Line types
lwd	Line widths
xlim, ylim	Limit
linewidth	Linewidth for some reason not lwd?

Value

No value, labels added to current plot.

Note

Needs elaboration, possibly merge documentation with others label functions?

See Also

Called by [geocontour](#).

labels_size	<i>Label symbols of given size</i>
-------------	------------------------------------

Description

Labels of given size.

Usage

```
labels_size(
  cont,
  digits,
  sizes,
  xlim = c(0, 1),
  ylim = c(0, 1),
  fill = F,
  n,
  rat,
  minsym = "<",
  label.resolution = 0,
  open = F,
  lwd = 1,
  col = 1
)
```

Arguments

cont	Contours
digits	Number of digits to use in labels
sizes	Sizes (of what?)?
xlim, ylim	Limits
fill	Fill? Default FALSE
n	Number of ??
rat	Ratio of ??
minsym	Minimum symbol for lowest category in labels
label.resolution	Label resolution ?
open	Open legend/label, default FALSE
lwd	Line width
col	Color

Value

No value, labels added to current plot.

Note

Needs further elaboration, document with other labelling functions??

See Also

Called by [colsymbol](#).

lambert	<i>Lambert projection</i>
---------	---------------------------

Description

Lambert projection.

Usage

```
lambert(lat, lon, lat0, lon0, lat1, scale = "km", old = F)
```

Arguments

lat, lon	Coordinates as latitude and longitude vectors
lat0	First latitude defining of the projection
lon0	Longitude defining the projection
lat1	Second latitude defining the projection
scale	Scale, default "km", redundant ??
old	Old method ?

Value

List of components:

lat, lon	Coordinates in latitude and longitude
x, y	Input coordinates (projected)
scale	Scale
projection	Projection (stated redunantly)
lat0, lon0, lat1	Defining lats and lon

is returned invisibly.

Note

Needs elaboration, might be merged with docs for other proj-functions.

See Also

Called by [geoarea](#), [orthproj](#) and [Proj](#).

litir	<i>Display palette in effect</i>
-------	----------------------------------

Description

A grid of colors is plotted.

Usage

```
litir(n)
```

Arguments

n	Number of columns and rows of colors to display
---	---

Details

Palette in effect gets repeated for n greater than length(palette()).

Value

No value returned, plots an n by n grid of colors.

Note

Simpler version of [Rlitir](#), one of those should do, possibly with a name change.

See Also

[Rlitir](#)

locdist	<i>Distance between two locations</i>
---------	---------------------------------------

Description

Distance between two locations set out by clicking on a geo-plot.

Usage

```
locdist(scale = "nmi", type = "p")
```

Arguments

scale	Unit of returned distance, default "nmi" for nautical miles, all other values return kilometers
type	Display points "p" or lines "l" between clicks, "n" for nothing

Value

Returns distance between two point clicks on a geoplots.

Note

Rather limited functionality, could be built further?

See Also

Calls [arcdist](#) and [geolocator](#).

mercator	<i>Mercator projection</i>
----------	----------------------------

Description

Mercator projection.

Usage

```
mercator(lat, lon, scale = "km", b0 = 65)
```

Arguments

lat, lon	Coordinates as latitude and longitude vectors
scale	Scale of the output, "km" default, all other values imply nautical miles.
b0	Latitude defining the projection.

Value

List of components:

lat, lon	Coordinates in latitude and longitude
x, y	Input coordinates (projected)
scale	Scale
projection	Projection (stated redundantly)
b0, L0	Defining lat and a null value ???

is returned invisibly.

Note

Needs elaboration, could/should (?) be documented with other projection functions.

See Also

Called by [geoarea](#), [gridaxes](#) and [Proj](#).

nonsetpar	<i>Some parameter settings.</i>
-----------	---------------------------------

Description

Needs elaboration or to be removed.

Format

The format is: chr [1:8] "omd" "oma" "omi" "fin" "fig" "mfc" "mfrow" ...

Examples

```
data(nonsetpar)
## maybe str(nonsetpar) ; plot(nonsetpar) ...
```

Open.curve	<i>Open smooth curve through positions</i>
------------	--

Description

A smooth curve is fitted to the positions with natural spline as implemented in function ns.

Usage

```
Open.curve(gogn, df = nrow(gogn)/2, n = 10)
```

Arguments

gogn	Data frame of positions to be smoothed
df	Degrees of freedom
n	Number of points returned per data point

Value

Data frame of positions with components	
lat, lon	in decimal degrees

Note

Function ns is missing, assume it is some sort of natural spline.

orthproj	<i>Performs an orthogonal projection to a curve.</i>
----------	--

Description

Finds curve coordinates of points, given points and a curve it returns the length of the points orthogonal projection and the distance traveled to the projection from a given origin on the curve.

Usage

```
orthproj(pts, curve)
```

Arguments

pts	A list containing the points as lat and lon, you may also use geolocator.
curve	The curve to be used for projection.

Value

Returns a dataframe with vectors pardist (length of orthogonal projection) and perdist (how far traveled along the curve).

Side Effects

None.

See Also

[geocurve](#), [geolocator](#).

Examples

```
## Not run:      geoplots(my.curve)      # Plot curve and initialize plot.
                geocurve(geolocator(),my.curve)      # Mark points.

## End(Not run)
```

paint.window	<i>Paint window for label?</i>
--------------	--------------------------------

Description

Paint window for label based on lat and lon?.

Usage

```
paint.window(listi, col = 0, border = T, poly = T, col.names = c("lon", "lat"))
```

Arguments

listi	Label location ?
col	Color, not used ?
border	Should border be drawn?
poly	Should label be opaque ?
col.names	Column names containing label coordinates ?

Value

No value, lines and/or polygon added to current geoplot.

Note

Needs elaboration and possibly merging with paint.window doc-file. Argument col has no effect.

See Also

Called by [colsymbol](#), [geocontour](#), [geocontour.fill](#), [geosymbols](#) and [reitaplott](#); calls [Proj](#).

paint.window.x

Paint window for label ?

Description

Paint window for label based on projected coordinates?.

Usage

```
paint.window.x(listi, col = 0, border = T, poly = T)
```

Arguments

listi	Label location ?
col	Color, not used ?
border	Should border be drawn?
poly	Should label be opaque ?

Value

No value, lines and/or polygon for label added to current geoplot.

Note

Needs elaboration and possibly merging with paint.window doc-file. Argument col has no effect.

See Also

Called by [geocontour](#).

pdist	<i>Spherical (?) distance</i>
-------	-------------------------------

Description

Spherical (?) distance.

Usage

```
pdist(lat, lon, lat1, lon1)
```

Arguments

lat, lon	Coordinate vectors of starting positions in lat/lon
lat1, lon1	Coordinate vectors of end positions in lat/lon

Value

Distance in kilometers.

Note

Why not use `arcdist`? Distance functions could be documented in one file.

See Also

Called by [pointkriging](#) and [variogram](#).

pdistx	<i>Euclidian distance</i>
--------	---------------------------

Description

Euclidian distance.

Usage

```
pdistx(y, x, y1, x1)
```

Arguments

y, x	Coordinate vectors of starting positions
y1, x1	Coordinate vectors of end positions

Value

Euclidian distance

Note

Check order of coordinates (?), document distance functions together?

See Also

Called by [variogram](#).

plot_nogrid	<i>Grid control ?</i>
-------------	-----------------------

Description

Grid or border control of some sort?

Usage

```
plot_nogrid(o, xgr, ygr, col)
```

Arguments

o	o, some outer boundary?
xgr	xgr ?
ygr	ygr ?
col	Color

Value

No value, some lines added to current geoplot.

Note

Needs elaboration.

See Also

Called by [gridaxes](#).

plot_reitnr	<i>Plots rectangle codes</i>
-------------	------------------------------

Description

Adds codes for statistical rectangles to the current plot

Usage

```
plot_reitnr(cexrt, lwd = 0)
```

Arguments

cexrt	Character expansion of the codes on the plot.
lwd	Grid line width

Value

No value, adds the codes for statistical rectangles to the current geoplots. The codes are added for all rectangles visible on the plot.

Note

Might be extended to plotting 'smareitur' codes? Grid line setting affects bounding box of the plot could be fixed/extended as well(?).

See Also

Called by [geoplots](#), calls [d2r](#), [geotext](#) and [invProj](#).

pltgrid	<i>Plots grid lines</i>
---------	-------------------------

Description

Plots grid lines.

Usage

```
pltgrid(xgrid = NULL, ygrid = NULL, xpos, ypos, ...)
```

Arguments

xgrid	Should x-grid be drawn, default NULL?
ygrid	Should y-grid be drawn, default NULL?
xpos	xpos ?, if missing taken from par("xaxp")
ypos	ypos ?, if missing taken from par("yaxp")
...	optional parameters to be sent to lines

Value

No value, draws grid lines on current plot.

Note

Needs elaboration. Prints ypos ??

See Also

Neither called by nor calls any function in package geo.

plvar	<i>Plots a variogram estimated by variofit.</i>
-------	---

Description

Plot a variogram and a spherical model fit to it.

Usage

```
plvar(vagram, n = 4, fit = T, type = "p")
```

Arguments

vagram	List from the program variogram with the following components. <code><s-example></code> range: range of data. nugget: nugget effect. sill: sill. vario: z values of the variogram. xh: mean distance for each class. <code></s-example></code>
n	The plot covers n times the range. Default value is all the data.
fit	Also plot lines and fitted values (TRUE, the default, or plot variogram only as points (FALSE).
type	Plot type, given to plot, defaults to points.

Value

No values returned.

See Also

[pointkriging](#), [grid](#).

pointkriging	<i>interpolates regularly spaced data on a grid.</i>
--------------	--

Description

The function interpolates regularly spaced data on a grid. The program uses inverse distance method that takes clustering into account. Under certain conditions this method can be called kriging. For each gridpoint the program looks for neighbourhood points according to certain criteria. The program has a possibility to prevent smearing data out from one area to another where it is not wanted like between two fjords. The program has the possibility of universal kriging with the drift in lat,lon or due to an external variable.

Usage

```

pointkriging(
  lat,
  lon,
  z,
  xgr,
  vagram,
  maxnumber = 16,
  scale = "km",
  option = 1,
  maxdist = 0,
  rat = 3,
  nb = 8,
  set = 0,
  areas = 0,
  varcalc = F,
  sill = 0,
  minnumber = 2,
  suboption = 1,
  outside = T,
  degree = 0,
  lognormal = F,
  zeroset = F
)

```

Arguments

lat	Latitude of datapoints.
lon	Longitude of datapoints.
z	Values at datapoints.
xgr	Description of the grid. Can be output from program grid or just a list with components lat and lon.
vagram	Components of the variogram, a list with a least 3 components, sill, nugget & rang1.
maxnumber	Number of neighbourhood points used , default is 16. In some cases 16 points are not found.
scale	Scale "km" or "miles", default is "miles".
option	Option used for selecting neighbourhood points. Allowed values 1,2,3 and 4. Default value is one. For further information see below.
maxdist	If option = 4 all points within maxdist are used. If option = 4 the default value of maxdist is the range of the variogram. maxdist has also meaning when option = 1,2 or 3. In those cases points further away from any datapoint than maxdist are set to zero, mean z or NA.
rat	The number of points in the first step of the search is rat*maxnumber.
nb	Parameter describing the extent of the area where neighbourhood points are searched in the first round. Default value is 8 which means that and area of 16x16 gridpoints is searched.
set	Points outside region or further away than maxdist from any datapoint are set to either zero (set=0) or mean(z) set=1 or NA set =-1.

areas	A list defining a number of areas. NA is between areas. Points in different areas are treated as independent. Two neighbourhood fjords could be defined as different areas so the program does not interpolate between them.
varcalc	If varcalc is true the estimation variance at each datapoint is calculated. Default value is F.
sill	Sill in variance calculations. Default value is the sill of the variogram.
minnumber	If number of neighbourhood points found is less than or equal to minnumber the point is considered outside the areas covered by the datapoints and set to NA, 0 or mean(z)
suboption	If option = 4 and more than maxnumber points are found in the area the program switches to option 1, 2 or 3 in the point search. Default value is 1.
outside	If outside is T points further away from the area than nb*dx are excluded. dx is the grid interval. Too many points outside of the area can problems in the search because the grid is used to divide the area in squares and everything left and below the first gridpoint is for example one square.
degree	Degree of drift polynomial for universal kriging. 0, 1 or 2. Default 0.
lognormal	To be described.
zeroset	To be described.

Value

A vector with the calculated values at the gridpoints.

Side Effects

The program is partly written in C so if it crashes Splus is exited.

See Also

[variogram](#), [variofit](#), [grid](#), [geocontour.fill](#).

Examples

```
## See geocontour.fill
```

postcol

Old palette.

Description

Soon to be deprecated, still referred to by `geopalette` and `geocontour.fill`.

Format

The format is: `num [1:169, 1:3] 0.667 0.667 0.660 0.654 0.647 ... - attr(*, "dimnames")=List of 2 ..$ : chr [1:169] "1" "2" "3" "4" ... ..$ : chr [1:3] "H" "S" "V"`

Examples

```
data(postcol)
## maybe str(postcol) ; plot(postcol) ...
```

pre2s

*Read prelude files***Description**

Read data files in prelude format, which has 2 line headers and column names seperated from data with dashes.

Usage

```
pre2s(skr, rownames = F, dots.in.text = T)
```

Arguments

skr	Prelude file name
rownames	Should first column be used as row names? Default FALSE
dots.in.text	Should underscores in column names be replaced with "."? Default TRUE

Value

A data frame ([data.frame](#)) containing a representation of the data in the file.

Note

Call to `skipta.texta` could be replaced with a call to [chartr](#) (as in ROracleUI sql).

See Also

Calls [skipta.texta](#)

prepare.line

*Prepare a line ?***Description**

Prepare a line for some action ?

Usage

```
prepare.line(x)
```

Arguments

x	Line vector ?
---	---------------

Value

Returns a list with components:

lx1	lx1 ?
lx2	lx2 ?
nlx	nlx ?

Note

Needs elaboration.

See Also

Called by [cut_multipoly](#) and [findline](#).

Proj	<i>Performs Mercator or Lambert projection of data.</i>
------	---

Description

Performs Mercator, Lambert or no projection of data, as a default it will perform the projection used in current plot.

Usage

```
Proj(  
  a,  
  b = 0,  
  scale = getOption("geopar")$scale,  
  b0 = getOption("geopar")$b0,  
  b1 = getOption("geopar")$b1,  
  l1 = getOption("geopar")$l1,  
  projection = getOption("geopar")$projection,  
  col.names = c("lon", "lat")  
)
```

Arguments

a, b	The input data to be projected, may be given as two vectors or as list attributes, lat and lon (x and y if projection = none).
scale	The scale used for the projection, (m, km or miles). Default is the scale defined in geopar (the scale defined when the plot is initialized).
b0	if projection = mercator b0 is the center of the mercator projection. If projection = "Lambert" b0 and b1 are latitudes defining the Lambert projection. Default are the b0 and b1 defined in geopar.
b1	Second defining latitude for Lambert projection
l1	The longitude defining the Lambert projection, default is the l1 defined in geopar.
projection	The projection of the data, legal projections are "Mercator", "Lambert" and "none".
col.names	This has to be set to the default value of c("lon", "lat"), otherwise projection will be set to "none".

Value

The function returns a list containing if projection = "none" x and y, if projection is "Mercator" or "Lambert" it includes the projection (projection), the scale (scale), lat and lon and x and y (the distance in scale from point (0,0) in spherical coordinates).

See Also

[invProj](#), [geopar](#), [geoplot](#).

Examples

```
# For an example of use for this function see i.e. init() where
# it is called:
## Not run:
xgr <- Proj(lat, lon, scale, b0, b1, l1, projection)

## End(Not run)
```

ps

Periodic spline

Description

Fits periodic spline to data.

Usage

```
ps(x, df = NULL, knots = NULL, period = c(0, 2 * pi))
```

Arguments

x	Data ?
df	Degrees of freedom ?
knots	Knots ?
period	Period, default c(0, 2*pi) ?

Value

basis	Basis of periodic spline ?
-------	----------------------------

Note

Needs elaboration.

See Also

[Closed.curve](#), [spline.des](#).

rect2deg

*Given rectangle code return its center position.***Description**

Functions that convert statistical rectangle codes under: 1) a traditional Icelandic system ('tilkynningaskyldurreitakerfid') (see [d2r](#) and [d2sr](#)) and 2) set up in systems based on minutes and degrees (see [d2mr](#) and [d2dr](#)) to decimal representation of rectangles center positions in degrees latitude and longitude.

Usage

```
r2d(r)
```

```
sr2d(sr)
```

```
mr2d(mr, dlat = 5, dlon = 10)
```

```
dr2d(dr, dlat = 1, dlon = 2, startLat = 50)
```

Arguments

r	Rectangle code r in the 'tilkynningaskyldu-system', e.g from deg2rect .
sr	Rectangle code sr for subrectangle in 'tilkynningaskyldu-system', e.g. from deg2rect .
mr	Rectangle code mr based on minutes, e.g. from deg2rect .
dlat	Rectangle height in minutes or degrees latitude for mr2d and dr2d respectively.
dlon	As dlat except now width in longitude.
dr	Rectangle code dr based on degrees, e.g. from deg2rect .
startLat	Starting latitude for coding the rectangles.

Details

- r2d with a resolution of 30 min latitue and 1 deg longitude (the Icelandic numbering system, 'tilkynningaskyldureitir').
- sr2d with a resolution of 15 min latitude by 30 min longitude in the Icelandic numbering system for statistical rectangles which starts counting at 60 deg N latitude, with sub-rectangles of 30 min lat by 1 deg lon coded 1, 2, 3 and 4 for the NW, NA, SW and SA quadrants respectively,
- mr2d with resolution given in dlat by dlon minutes lat and lon,
- dr2d for rectangles with resolution given in dlat by dlon degrees lat and lon, code system starting at latitude startLat.

Value

dataframe of center positions (latitude lat and longitude lon) of rectangles in one of the coding systems

Note

Mostly used for plotting.

Author(s)

HB (for r, sr), STJ (for mr, dr).

See Also

[deg2rect](#)

Examples

```
r2d(d2r(lat = 65 + 1/4, lon = -19 - 1/2))
d2r(r2d(519))
```

rectArea	<i>Given rectangle code return area in square kilometers or nautical miles</i>
----------	--

Description

Rectangle area is returned through a call to [rectPeri](#)-functions and [geoarea](#).

Usage

```
rA(r, scale = "nmi")

srA(sr, scale = "nmi")

mrA(mr, dlat = 5, dlon = 10, scale = "nmi")

drA(dr, dlat = 1, dlon = 2, scale = "km")
```

Arguments

r, sr, mr, dr	rectangle code, as in rect2deg and deg2rect .
scale	nmi, km, default nmi returns values in square nautical miles for all except drA returns area in square kilometers.
dlat, dlon	Dimensions of latitude and longitude given in minutes and degrees for mrPeri and drPeri, respectively.

Value

Rectangle area in square nautical miles or kilometers.

Note

Unit nmi is used for historical/acoustical (sA) reasons.

See Also

[rectPeri](#), [deg2rect](#), [geoarea](#).

Examples

```
srA(7121)
srA(7121, "km")
srA(7121, "km")/1.852^2
srA(7121, "km")
rA(712)
srA(7121) + srA(7122) + srA(7123) + srA(7124)
```

rectPeri	<i>Given rectangle code return perimeter as a polygon in lat lon</i>
----------	--

Description

The outline/boundary of a statistical rectangle is returned as 5 positions, the first and last of which are the same.

Usage

```
rPeri(r)

srPeri(sr)

mrPeri(mr, dlat = 5, dlon = 10)

drPeri(dr, dlat = 1, dlon = 2)
```

Arguments

- r, sr, mr, dr Rectangle codes.
- dlat, dlon Dimensions of latitude and longitude given in minutes and degrees for mrPeri and drPeri, respectively.

Value

Rectangle outline as 5 positions.

Note

Should perhaps be extended to give a list or dataframe of polygons for more than one r , sr , mr or dr.

See Also

[deg2rect](#), [rectArea](#), [geoarea](#).

Examples

```
geoplot(island, type = "n", grid = FALSE)
geolines(rPeri(468))
geolines(srPeri(4681))
```

reg.bc

Bormicon regions

Description

A list of the Bormicon regions

Format

The format is: List of 16 \$ Vestur : 'data.frame': 1221 obs. of 2 variables: ..\$ lat: num [1:1221] 65.5 65.5 65.5 65.5 65.5\$ lon: num [1:1221] -24.4 -24.4 -24.3 -24.2 -24.1 ... \$ Vestfirðir : 'data.frame': 664 obs. of 2 variables: ..\$ lat: num [1:664] 65.5 65.5 65.4 65.4 65.4\$ lon: num [1:664] -21 -21 -21 -20.9 -21 ... \$ Mid.Nordur : 'data.frame': 560 obs. of 2 variables: ..\$ lat: num [1:560] 67.2 67.2 67.2 67.2 67.2\$ lon: num [1:560] -24 -24 -24 -24 -24 ... \$ Grunnslod.nordur : 'data.frame': 146 obs. of 2 variables: ..\$ lat: num [1:146] 66.2 66.2 66.2 66.2 66.2\$ lon: num [1:146] -17 -17 -17.1 -17.1 -17.2 ... \$ Nordaustur : 'data.frame': 561 obs. of 2 variables: ..\$ lat: num [1:561] 66.3 66.3 66.3 66.3 66.3\$ lon: num [1:561] -15 -15.1 -15.1 -15.2 -15.3 ... \$ Austur : 'data.frame': 707 obs. of 2 variables: ..\$ lat: num [1:707] 64.5 64.5 64.6 64.6 64.5\$ lon: num [1:707] -14.5 -14.5 -14.4 -14.4 -14.5 ... \$ Rosagardur : 'data.frame': 329 obs. of 2 variables: ..\$ lat: num [1:329] 64.5 64.5 64.5 64.5 64.5\$ lon: num [1:329] -10.4 -10.4 -10.4 -10.4 -10.4 ... \$ Sudaustur : 'data.frame': 19 obs. of 2 variables: ..\$ lat: num [1:19] 64.2 64.2 64.2 64.3 64.3\$ lon: num [1:19] -15 -15 -15 -14.9 -14.9 ... \$ Sudur.Sudaustur : 'data.frame': 788 obs. of 2 variables: ..\$ lat: num [1:788] 63.4 63.4 63.4 63.4 63.4\$ lon: num [1:788] -19 -19 -18.9 -18.9 -18.8 ... \$ Sudur.Sudvestur : 'data.frame': 546 obs. of 2 variables: ..\$ lat: num [1:546] 64 64 64 64 64\$ lon: num [1:546] -22.7 -22.7 -22.7 -22.7 -22.7 ... \$ Kantur.NW : 'data.frame': 557 obs. of 2 variables: ..\$ lat: num [1:557] 67.2 67.2 67.2 67.2 67.2\$ lon: num [1:557] -24 -24 -24 -24 -24 ... \$ Kantur.NE : 'data.frame': 642 obs. of 2 variables: ..\$ lat: num [1:642] 63.5 63.5 63.5\$ lon: num [1:642] -10.2 -10.2 -10.2 -10.2 -10.2 ... \$ Sudur.Dypi : 'data.frame': 1283 obs. of 2 variables: ..\$ lat: num [1:1283] 64 64 64 64 64\$ lon: num [1:1283] -13.2 -13.2 -13.2 -13.2 -13.2 ... \$ Reykjaneshr : 'data.frame': 188 obs. of 2 variables: ..\$ lat: num [1:188] 63 63 63 63 63\$ lon: num [1:188] -24 -24 -24 -24 -24.1 ... \$ Vestur.Dypi : 'data.frame': 1010 obs. of 2 variables: ..\$ lat: num [1:1010] 63 63 63 63 63\$ lon: num [1:1010] -25 -25 -25 -25 -25 ... \$ Nordur.Dypi : 'data.frame': 7 obs. of 2 variables: ..\$ lat: num [1:7] 68.5 70 72.5 72.5 70 68.5 68.5 ..\$ lon: num [1:7] -8 -4 -4 -21 -21 -27 -8 - attr(*, "area")= num [1:16] 50437 31953 22803 10501 16024 ... - attr(*, "xtloc")=List of 2 ..\$ lat: num [1:16] 64.6 66.6 67 66.3 67\$ lon: num [1:16] -25.6 -24.2 -19.2 -19.4 -15.6 ...

Details

Used in [inside.reg.bc](#) to assign region numbers to data.

Examples

```
data(reg.bc)
## maybe str(reg.bc) ; plot(reg.bc) ...
```

reg.lump

Lumpfish regions

Description

List of areas demarkated by the lumpfish regulation and the main part of 500 m isobath around Iceland (with a simple transverse line over the Iceland-Faroe Ridge).

Format

The format is: List of 8 \$ Faxaflói : 'data.frame': 214 obs. of 2 variables: ..\$ lat: num [1:214] 64.8 64.7 64.7 64.7 64.7\$ lon: num [1:214] -23.9 -23.9 -23.8 -23.8 -23.7 ... \$ Breidafjörður outer : 'data.frame': 53 obs. of 2 variables: ..\$ lat: num [1:53] 65.5 65.5 65.5 65.5 65.4\$ lon: num [1:53] -24.5 -24.4 -24.2 -24.1 -24 ... \$ Breidafjörður inner: 'data.frame': 212 obs. of 2 variables: ..\$ lat: num [1:212] 65.5 65.5 65.5 65.6 65.6\$ lon: num [1:212] -23.2 -23.2 -23.2 -23.2 -23.2 ... \$ Vestfirjörður : 'data.frame': 254 obs. of 2 variables: ..\$ lat: num [1:254] 66.4 66.5 66.4 66.4 66.4\$ lon: num [1:254] -22.4 -22.5 -22.5 -22.5 -22.6 ... \$ Húnaflói : 'data.frame': 149 obs. of 2 variables: ..\$ lat: num [1:149] 66.1 66.1 66.1 66.1 66\$ lon: num [1:149] -20.1 -20.2 -20.3 -20.4 -20.4 ... \$ Norðurland : 'data.frame': 261 obs. of 2 variables: ..\$ lat: num [1:261] 66.4 66.4 66.4 66.4 66.3\$ lon: num [1:261] -14.6 -14.6 -14.8 -14.9 -15 ... \$ Eastfirjörður : 'data.frame': 298 obs. of 2 variables: ..\$ lat: num [1:298] 64.4 64.4 64.5 64.5 64.5\$ lon: num [1:298] -14.5 -14.5 -14.5 -14.5 -14.5 ... \$ Suðurland : 'data.frame': 402 obs. of 2 variables: ..\$ lat: num [1:402] 64.1 64.1 64 64 64\$ lon: num [1:402] -22.7 -22.7 -22.7 -22.7 -22.7 ...

reitaplott

Function that fills rectangles or sub-rectangles according to the amount of fish calculated in them.

Description

Function that fills rectangles or sub-rectangles according to the amount of fish calculated in them, or some other quantity.

Usage

```
reitaplott(
  reitur,
  smareitur = NULL,
  z,
  levels = NULL,
  colors = NULL,
  density = 1,
  maxcol = 155,
  nlevels = 6,
  white = FALSE,
  border = FALSE,
  angle = 45,
  rotate = 0,
  digits = 0,
```

```

    cex = 0,
    label.location = 0,
    labels.only = FALSE,
    col = 1,
    outside = TRUE,
    mincol = NULL
)

```

Arguments

reitur	Number of square. If number of square >1000 the last digit means subsquare. For example 5604 means square 560, subsquare 4.
smareitur	Number of subsquare if they are used. Default is that subsquare is not used.
z	Values for each square and subsquare.
levels	Values between colors or different density of lines. Default is to let the program find it. A small example shows how levels are given. <code><s-example> levels <- c(0, 100, 1000, 2000, 4000, 5000, 8000) </s-example></code> Then the groups are <100, 100-1000, 2000-4000, 4000-5000 & >5000, i.e. the program more or less ignores the first and the last values given.
colors	Number of color or shading pattern for each level. Colours can go from 0 thru 155. They go from white = 0, black = 1, blue = 2, green, yello, red = 155. Density goes from 10 thru 130. Everything above 130 is completely black. If colors is not given the program determines it from the data. Default maximum density is set to 70 but everything above that is very black.
density	If density is 0 coloured plot is made else different desity of lines is used.
maxcol	Maximum color (or density) Default value is 155 for colors 70 for shading. All density of lines > 130 are black.
nlevels	Number of levels. Default value is 6.
white	If white = TRUE the lowest color or density is zero. Default value is white.
border	If border of polygon is to be drawn. Default value is FALSE.
angle	Angle of lines used for hatching. Default value is 45 degrees.
rotate	Rotation of hatcing lines between each level. Default value is zero or no rotation but rotate = 90 is very useful.
digits	Number of digits in labels. Default value is zero.
cex	Charcter expansion of digits in labels. Default value is the current size when the program is called.
label.location	location of labels, default is no labels are plotted.
labels.only	if true only labels are plotted.
col	the color of lines, default is 1 (black).
outside	if true squares outside plot area are plotted, default is true.
mincol	the color number of the first color to be used, colors are taken from the range mincol to maxcol.

Details

Geoplot has to be called before to set the parameter `geopar`. If `geoplot` is called with the parameter `cont = TRUE` labels are plotted on the left hand side of the plot. If a color plot is to be made `litaps` has to be called before.

Value

No values returned.

Side Effects

No side effects.

See Also

[geoplot](#).

Examples

```
geoplot(reitur = TRUE)
reitur <- c(560, 560, 560, 560, 561, 561, 561, 561)
smareitur <- c(1, 2, 3, 4, 1, 2, 3, 4)
lab.loc <- list(lat = c(63.95, 65.4), lon = c(-19.8, -17.3))
z <- c(100, 200, 300, 222, 345, 453, 232, 212)
geopolygon(island); geolines(island)
reitaplott(reitur, smareitur, z, rotate = 90, label.location = lab.loc)
# All subsuares in square 560 and 561.
```

Reitur2Svaedi1to10	<i>Allocate statistical rectangles and subrectangles to Bormicon-areas 1–10.</i>
--------------------	--

Description

Routine for allocating rectangles to areas with some in-built decisions for rectangles on the borders of areas.

Usage

```
Reitur2Svaedi1to10(reitur, smareitur, Totalreitir, Dypisreitir)
```

Arguments

reitur	Rectangle in the Icelandic statistical rectangle system.
smareitur	Sub-rectangle in the Icelandic statistical rectangle system.
Totalreitir	Projection between a list of all Icelandic rectangles to a list of Bormicon areas.
Dypisreitir	Not documented.

Value

Returns vector of bormicon areas of the rectangles.

Note

Objects Totalreitir containing a list of all rectangles completely contained in bc-areas and Dypisreitir with something are necessary for the function to work.

See Also

Bormicon-area allocation functions [inside.reg.bc](#) and [inside.reg.bc1](#), [r2d](#)

rGrid	<i>Produce a grid of rectangles on a plot, filled with colors if desired.</i>
-------	---

Description

Functions that plot a line grid or one filled with colors for rectangle codes in systems of various resolutions with call to [geolines](#) or [geopolygon](#) and perimeter utility functions (see [rectPeri](#)).

Usage

```
rgrid(r, fill = FALSE, ...)

srgid(sr, fill = FALSE, ...)

mrgrid(mr, dlat = 5, dlon = 10, fill = FALSE, ...)

drgrid(dr, dlat = 1, dlon = 2, fill = FALSE, ...)
```

Arguments

r, sr, dr, mr	Codes of rectangle to be outlined or filled. In different resolutions, r, sr with their own system (see deg2rect), mr dimensions based on minutes, dr on degrees.
fill	Logical, whether or not to fill the plotted rectangles.
...	other arguments to geopolygon or geolines as appropriate, notably col.
dlat, dlon	Dimensions of latitude and longitude given in minutes and degrees for mrgrid and drgrid, respectively

Details

The functions simply outline or fill the rectangles they are given. whereas [reitaplott](#) and [geoSR](#) assume levels and are more hi-level.

Value

No values returned, used for side-effects.

Note

Functions [reitaplott](#) and [geoSR](#) have more in-built functionality to deal with level-plots of rectangles.

Author(s)

STJ

See Also

[deg2rect](#), [geolines](#), [geopolygon](#), [rectPeri](#), [reitaplott](#), [geoSR](#).

Examples

```
geoplot(grid = FALSE)
tmp <- island
tmp$sr <- d2sr(island)
srects <- aggregate(. ~ sr, tmp, length)
names(srects)[2] <- "count"
srects$lev <- cut(srects$count, c(0, 1, 5, 10, 20, 50, 100))
mycol <- heat.colors(length(unique(srects$lev)))
srgrid(srects$sr, fill = TRUE, col = mycol[srects$lev])
geolines(island)
```

Rlitir

Display colors.

Description

A grid of colors is plotted.

Usage

```
Rlitir(n, col)
```

Arguments

n	Number of columns and rows of colors to display
col	A vector of colors

Value

No value returned, plots an n by n grid of colors.

See Also

[colorRampPalette](#)

Examples

```
# simple, perhaps not so useful application with default palette:

Rlitir(12, 1:144)

# Define a palette with some colors:

ramp <- colorRampPalette(c("khaki1", "gold", "orange",
  "darkorange2", "red", "darkred", "black"))
```

```
# number of columns and rows to display

n <- 10

Rlitir(n, ramp(n^2))
```

s2pre

Writes out data.frame or matrix to a prelude-file.

Description

Data.frame or matrix object is written to a prelude-file, that inherits names/dimnames attributes from the object.

Usage

```
s2pre(data, file = "splus.pre", na.replace = "")
```

Arguments

data	Data.frame or matrix object.
file	Name of the output file ("Splus.pre" by default).
na.replace	A character to replace NA with in the output file (" " by default).

Value

A prelude-file representation of the data-object is written to a file.

Side Effects

No warning is given if the filename "file" already exists – it is simply over-written.

See Also

[cat](#), [write](#).

Examples

```
## Not run: Within Splus:
> tmp.test.frame
      tolur1      tolur2 stafir1
1      11 0.04625551      a
2      12 0.04845815      a
3      13 0.05066079     NA
4      14 0.05286344      a
5      15 0.05506608      a
6      16 0.05726872      b
7      17 0.05947137      b
8      18          NA      b
9      19 0.06387665      b
10     20 0.06607930      b
> s2pre(tmp.test.frame, file="prufa.pre", na.replace="-1")
```

```
>

From UNIX:

hafbitur/home/reikn/gardar/Papers/Methods95 [435] cat prufa.pre
linu_nofn      tolur1  tolur2  stafir1
-----
1      11      0.04625551      a
2      12      0.04845815      a
3      13      0.05066079     -1
4      14      0.05286344      a
5      15      0.05506608      a
6      16      0.05726872      b
7      17      0.05947137      b
8      18      -1      b
9      19      0.06387665      b
10     20      0.06607930      b
hafbitur/home/reikn/gardar/Papers/Methods95 [436]

## End(Not run)
```

SegmentWithArrow	<i>Plot line segment with arrow at the end.</i>
------------------	---

Description

Plot line segment with arrow at the end.

Usage

```
SegmentWithArrow(pos, angle = 15, size = 0.2, col = "blue", lwd = 2)
```

Arguments

pos	Segment positions.
angle	(Half-)Angle (< 90) determining arrow sharpness.
size	Size of arrow.
col	Color of arrow.
lwd	Line width of segment.

Note

Needs further checking and elaboration.

selectedpar	<i>Parameters manipulation ?</i>
-------------	----------------------------------

Description

Parameters manipulation ?

Usage

```
selectedpar()
```

Value

Returns the names of some selected parameters.

Note

Needs elaboration,

See Also

Called by the majority of the graphical functions in geo, calls [Elimcomp](#).

selpos	<i>Geographical point selection.</i>
--------	--------------------------------------

Description

Select geographical data points by arbitrary criterion.

Usage

```
selpos(lat, lon = NULL, ind)
```

Arguments

lat	Latitude of points or list containing lat,lon.
lon	Longitude of data points. If missing, this must be part of the lat argument.
ind	Selection criterion.

Details

The normal way of working with geographical data is to store positions as a list with names lat and lon. This is easier for most applications, except selection of subsets, where it is essential to access individual elements. The purpose of this routine is merely to ease the selection process.

Value

Returns list with elements lat,lon which satisfy the criterion.

See Also

[geoplot](#),

Examples

```
## Not run:
      subs<-selpos(pos,,z>6)# Select positions where z>6

      The Function is trivially defined as
      function(lat, lon = NULL, ind)
      {
          if(is.null(lon)) {
              lon <- lat$lon
              lat <- lat$lat
          }
          lat <- lat[ind]
          lon <- lon[ind]
          return(lat, lon)
      }

      ## End(Not run)
```

Set.grd.and.z

Manipulate grid and z-values for contouring.

Description

Manipulate grid and z-values for contouring.

Usage

```
Set.grd.and.z(grd, z, mask, set = NA, col.names = c("lon", "lat"))
```

Arguments

grid	grid
z	z-value
mask	mask
set	set
col.names	column names, defaults to lat and lon

Value

Returns a list of:

grid	grid
z	value over grid

Note

Used in geocontour-functions.

setgrid

*Produce a grid over an area***Description**

Produce a grid over an area, possibly interactively.

Usage

```
setgrid(
  lat,
  lon = 0,
  type = "p",
  pch = "*",
  xlim = c(0, 0),
  ylim = c(0, 0),
  b0 = 65,
  r = 1.1,
  country = geo::island,
  xlab = "default",
  ylab = "default",
  option = "cut",
  reg = 0,
  dx = c(0, 0),
  nx = c(0, 0),
  grpkt = 0,
  scale = "km",
  find = F,
  new = F,
  grid = T,
  projection = "Mercator",
  n = 2500,
  b1 = b0,
  nholes = 0
)
```

Arguments

lat	Latitude
lon	Longitude, if not included in lat
type	Plot type
pch	Plot character
xlim, ylim	Limits for plot
b0	Base latitude
r	Range expansion
country	Country plotted
xlab, ylab	Labels for x and y axes
option	Method for determining plotted area extent, default "cut" (to the range of the data)

reg	Region to be gridded, can be set interactively
dx	Resolution in each direction (?)
nx	Number of gridpoints in each direction (?)
grpkt	Gridpoints can also be supplied to the function for plotting (?????????)
scale	Projection scale (general geo default is "km")
find	Should the gridpoints within reg be determined?
new	Plot control argument ?
grid	Draw grid (which grid?)
projection	Projection to use
n	Number of gridpoints (?)
b1	Second latitude for the Lambert projection
nholes	number of holes to be sent to geodefine when setting out the region to be gridded

Value

List of components:

grpt	Gridpoints
reg	Region over which the grid was laid
find	Was find = TRUE?
xgr	If find = TRUE the gridpoints within region reg is also returned.

Note

Needs elaboration, check use of find = TRUE

See Also

Calls a number of functions, i.e. [geodefine](#), [geoplot](#), [geopoints](#), [gridpoints](#), [inside](#), [selectedpar](#)

shadeborder	<i>Shade border ?</i>
-------------	-----------------------

Description

Shade border ?

Usage

shadeborder(reg, lat, lon, col = 0, col.names = c("lon", "lat"))

Arguments

reg	Region
lat	Latitude ?
lon	Longitude ?
col	Color, not used ?
col.names	Column names containing coordinates, default lat and lon.

Value

No value, adds shadedborder (with call to [lines](#)) to contoured geoplot.

Note

Needs elaboration. Color argument not used, but fixed values given to 2 calls to lines.

See Also

Called by [geocontour](#), calls [Proj](#).

shading1	<i>Shading of geoplots?</i>
----------	-----------------------------

Description

Shading of geoplots?

Usage

```
shading1(  
  cont,  
  digits,  
  colors,  
  xlim = c(0, 1),  
  ylim = c(0, 1),  
  fill = F,  
  angle,  
  rotate,  
  cex,  
  rat,  
  minsym = "<"  
)
```

Arguments

cont	Contours?
digits	Number of digits?
colors	Colors?
xlim,ylim	Limits?
fill	Fill?
angle	Angle?
rotate	Rotate?
cex	Character expansion?
rat	Ratio?
minsym	Minimum symbol on label?

Value

No value, shades current geoplots (label?) in some way?

Note

Needs elaboration.

See Also

Called by [colsymbol](#) and [reitaplott](#).

skipta.texta	<i>Translate characters in column names</i>
--------------	---

Description

Translate characters in column names

Usage

```
skipta.texta(txt, char = "_", replchar = ".")
```

Arguments

txt	Character vector of inputs
char	Character to be replaced
replchar	Character to replace with

Value

A character vector of the same length as txt.

Note

Could be deprecated, use [chartr](#) instead.

See Also

Called by [pre2s](#).

SMB.std.background	<i>Standard background for the SMB.</i>
--------------------	---

Description

Plots standard background for the SMB survey, aka Icelandic Groundfish Survey.

Usage

```
SMB.std.background(  
  depth,  
  depthcol = 1,  
  depthlty = 1,  
  depthlwd = 1,  
  eyjar,  
  depthlab,  
  depthlabcsi = 0.12,  
  ...  
)
```

Arguments

depth	Vector of isobaths to draw, default none
depthcol	Colors for isobaths
depthlty	Line types for isobaths
depthlwd	Line widths for isobaths
eyjar	Additional data to plot, e.g. islands
depthlab	Label depths? Defaults to FALSE
depthlabcsi	Depthlabel size
...	Additional arguemnts to geoplot and geolines

Value

No value returned, plots standard layout SMB/IGFS plot.

Note

Depth labelling etc might be cleaned up, use of eyjar argument could change, calls data object depthloc with a few positions for isobath labels.

See Also

[geoplot](#), [gbplot](#), [geolines](#), [eyjar](#)

spherical	<i>Spherical??</i>
-----------	--------------------

Description

'Spherical' of a variogram?

Usage

```
spherical(rang1, sill, nugget, x)
```

Arguments

rang1, sill, nugget	Parameters of a variogram.
x	The data.

Value

Fitted values?

Note

Needs elaboration, if this is a necessary function.

See Also

Some variogram stuff.

subplot	<i>Add subplot</i>
---------	--------------------

Description

Add subplot.

Usage

```
subplot(fun, x, y, size = c(1, 1), vadj = 0.5, hadj = 0.5, pars)
```

Arguments

fun	Graphical function to call
x, y	Coordinates
size	Subplot size (inches?)
vadj	Vertical adjustment
hadj	Horizontal adjustment
pars	Parameters to set?

Value

Returns parameter list invisibly, side effect subplot added to current plot.

Note

Needs elaboration.

See Also

Called by [geosubplot](#), calls [selectedpar](#).

twohmls	<i>The Icelandic EEZ.</i>
---------	---------------------------

Description

Outline of the Icelandic EEZ established somehow.

Format

A data frame with 88 observations on the following 2 variables.

lat a numeric vector

lon a numeric vector

Note

Needs checking, e.g. has recent agreement with Faroes been implemented. Compare with what's on www.marineregions.org.

variofit	<i>Function that fits a model to a variogram.</i>
----------	---

Description

Function that fits a model to a variogram. The fitting occurs either automatically or interactively. The function is called after the function `variogram` which calculates the variogram to which the model is fitted. Currently only spherical model is supported. Later other models will be added.

Usage

```
variofit(vagram, model = 1, option = 2, interactivt = F, sill = 0)
```

Arguments

vagram	List with the calculated variogram. Components of the list are: dist mean distance of the interval. <code><s-example></code> vario calculated value of the variogram. number number of datapoints in the interval. <code></s-example></code> In nearly all cases vagram will be the output from the program variogram.
model	Type of model. Default is spherical. It is currently the only model supported.
option	Method to use in automatic fitting. Allowed values are 1,2,3 and 4. Default value is 2. For further information see below.
interactivt	If T the fitting is done interactively by plotting the variogram on the screen and asking the user to select sill, range and nugget by the locator function.
sill	Sill of the variogram, or: Limit of the variogram tending to infinity lag distances (wikipedia).

Value

`<s-example>` A list with the following components. nugget : Estimated nugget effect sill : Estimated sill range : Estimated range dist : mean distance of the interval. vario : calculated value of the variogram. number : number of datapoints in the interval. `</s-example>`

See Also

[variogram](#), [pointkriging](#).

variogram	<i>Calculates the distance between each pair of datapoints.</i>
-----------	---

Description

The program calculates the distance between each pair of datapoints. The distances are grouped in groups such that even number of pairs is in each group. Then the estimated variogram for each group is calculated either by taking the mean or by a method from Cressie and Hawkins (1980). The latter method does in essence take the sum of the values^{0.25}. Only pair of points with distance less than certain distance are used. Zero - Zero pairs are not used if zzp is F.

Usage

```
variogram(  
  lat,  
  lon = 0,  
  z,  
  nbins = 100,  
  maxdist = 0,  
  Hawk = T,  
  throwout = F,  
  scale = "km",  
  evennumber = T,  
  zzp = F,  
  minnumber = 0,  
  col.names = c("lat", "lon")  
)
```

Arguments

lat	Latitude of datapoints. If lon=0 lat is a list with components lat and lon.
lon	longitude of datapoints.
z	Values at datapoints.
nbins	Number of distance intervals used.
maxdist	maximum distance of interest. Default value is range of data*0.7.
Hawk	If true the method from Cressie and Hawkins (1980) is used, else the mean. Default value is T
throwout	If T datapoints with value zero are not used at all. Default value is F which means that they are used. In all cases zero-zero pairs are not used when the variogram is estimated.
scale	"km" or "nmi". Default is "km".
evennumber	If T distance classes are chosen so approximately the same number of points is in each distance class. Else even distance increments are used.
zzp	If true zero-zero pairs are used else not. Default value is F.
minnumber	Distance intervals with minnumber or less pairs are not included. Default is zero.
col.names	if lat is a dataframe col.names should contain the names of the vectors containing the x and y coordinates, default is c("lat","lon")

Value

A list with the following components: <s-example>

number: Number of pair in each distance class. dist: Mean distance in each distance class. vario: variogram for each distance class. </s-example> The list is suitable for the program variofit. The variogram can also be plotted by plvar(vagram,fit=F)

See Also

[variofit](#), [pointkriging](#), [plvar](#).

Index

- * <
 - Put
 - colps, 12
 - combine.rt, 15
 - fill.outside.border, 26
- * **aplot**
 - adjust.grd, 5
 - Arrow, 8
 - colsymbol, 13
 - currentarrows, 17
 - geoaxis, 39
 - geolines, 61
 - geolines.with.arrows, 62
 - geosymbols, 77
 - geotows, 84
 - gridaxes, 90
 - gridaxes.Lambert, 91
 - labels1, 105
 - labels2, 106
 - labels_line, 107
 - labels_size, 108
 - paint.window, 113
 - paint.window.x, 114
 - plot_nogrid, 116
 - plot_reitnr, 116
 - pltgrid, 117
 - SegmentWithArrow, 134
 - setgrid, 137
 - shadeborder, 138
 - shading1, 139
 - subplot, 142
- * **arith**
 - d2ir, 20
 - deg2rect, 21
 - geoarea.old, 38
 - intra.point.dist, 101
 - rect2deg, 124
 - rectArea, 125
 - rectPeri, 126
 - spherical, 142
- * **colors**
 - litir, 110
- * **color**
 - geoplotbwpalette, 70
 - Rlitir, 132
- * **datasets**
 - bisland, 9
 - COUNTRY.DEFAULT, 17
 - depthloc, 23
 - eyjar, 24
 - faeroes, 25
 - gbdpi, 31
 - gbdpi.100, 32
 - gbdpi.1000, 32
 - gbdpi.200, 33
 - gbdpi.400, 33
 - gbdpi.500, 33
 - gbdpi.800, 34
 - gbdpif.200, 34
 - gbdpif.400, 35
 - gbdpif.500, 35
 - geopar, 63
 - gepc500, 88
 - glaciers, 89
 - greenland, 89
 - icelandrivers, 93
 - island, 105
 - nonsetpar, 112
 - postcol, 120
 - reg.lump, 128
- * **devices**
 - bwps, 10
- * **device**
 - Elimcomp, 23
 - selectedpar, 135
- * **here->**
 - colps, 12
 - combine.rt, 15
 - fill.outside.border, 26
- * **hplot**
 - circle.one.point, 11
 - geoSR, 74
 - ices, 94
 - init, 95
 - rGrid, 131
 - SMB.std.background, 141

- * **iplot**
 - giveborder, 88
 - locdist, 110
 - setgrid, 137
- * **logic**
 - findcut, 28
 - geointersect, 59
- * **manip**
 - adapt, 5
 - adjust.grd, 5
 - apply.shrink, 6
 - apply.shrink.dataframe, 7
 - bua, 9
 - Closed.curve, 12
 - cut_box.1, 18
 - cut_box.2, 18
 - cut_multipoly, 19
 - d2ir, 20
 - deg2rect, 21
 - fill.matrix, 26
 - fill.points, 27
 - find.hnit, 28
 - findcut, 28
 - findline, 29
 - frame2gpx, 30
 - geo.Split.poly, 37
 - geochull, 40
 - geoconvert, 50
 - geoconvert.1, 51
 - geoconvert.2, 51
 - geointersect, 59
 - gridpoints, 92
 - inside, 97
 - inside.reg.bc, 98
 - inside.reg.bc1, 98
 - inside.reg.lump, 99
 - invlambert, 102
 - invmerc, 103
 - lambert, 109
 - mercator, 111
 - Open.curve, 112
 - pdist, 115
 - pdistx, 115
 - prepare.line, 121
 - rect2deg, 124
 - rectArea, 125
 - rectPeri, 126
 - Reitur2Svaedi1to10, 130
 - Set.grd.and.z, 136
 - skipta.texta, 140
- * **more**
 - colps, 12
 - combine.rt, 15
 - fill.outside.border, 26
- * **one**
 - colps, 12
 - combine.rt, 15
 - fill.outside.border, 26
- * **or**
 - colps, 12
 - combine.rt, 15
 - fill.outside.border, 26
- * **s-keyword**
 - colps, 12
 - combine.rt, 15
 - fill.outside.border, 26
- * **smooth**
 - fitspher.aut.1, 30
 - ps, 123
- * **spatial**
 - geoSR, 74
 - ices, 94
 - rGrid, 131
- * **tags**
 - colps, 12
 - combine.rt, 15
 - fill.outside.border, 26
- * **utilities**
 - ices, 94
- adapt, 5, 97
- adjust.grd, 5, 29, 92
- apply.shrink, 6, 7
- apply.shrink.dataframe, 6, 7
- arcdist, 8, 102, 111
- Arrow, 8, 63
- bisland, 9
- bua, 9
- bwps, 10, 71
- cat, 133
- chartr, 121, 140
- chull, 40
- circle.one.point, 11
- Closed.curve, 12, 123
- colorRampPalette, 75, 132
- colps, 12, 71
- colsymbol, 13, 106, 109, 114, 140
- combine.rt, 15, 97
- contour, 43
- COUNTRY.DEFAULT, 17
- currentarrows, 17
- cut_box.1, 18, 92
- cut_box.2, 18, 92

- cut_multipoly, [19](#), [29](#), [122](#)
- d2dr, [124](#)
- d2dr (deg2rect), [21](#)
- d2ir, [20](#)
- d2mr, [50](#), [124](#)
- d2mr (deg2rect), [21](#)
- d2r, [21](#), [117](#), [124](#)
- d2r (deg2rect), [21](#)
- d2sr, [124](#)
- d2sr (deg2rect), [21](#)
- data.frame, [121](#)
- deg2rect, [21](#), [124–126](#), [131](#), [132](#)
- depthloc, [23](#)
- dr2d (rect2deg), [124](#)
- drA (rectArea), [125](#)
- drgrid (rGrid), [131](#)
- drPeri (rectPeri), [126](#)
- Elimcomp, [23](#), [96](#), [135](#)
- expand.grid, [55](#)
- extract, [24](#)
- eyjar, [24](#), [141](#)
- faeroes, [25](#)
- fill.matrix, [26](#)
- fill.outside.border, [26](#), [86](#)
- fill.points, [27](#), [92](#)
- find.hnit, [28](#), [29](#)
- findcut, [19](#), [28](#), [28](#), [59](#)
- findline, [6](#), [29](#), [63](#), [96](#), [122](#)
- fitspher.aut.1, [30](#)
- frame2gpx, [30](#)
- gbdypi, [23](#), [31](#)
- gbdypi.100, [32](#)
- gbdypi.1000, [32](#)
- gbdypi.200, [33](#)
- gbdypi.400, [33](#)
- gbdypi.500, [33](#)
- gbdypi.800, [34](#)
- gbdypif.200, [34](#)
- gbdypif.400, [35](#)
- gbdypif.500, [35](#)
- gbplot, [36](#), [141](#)
- geo (geo-package), [4](#)
- geo-package, [4](#)
- geo.Split.poly, [37](#)
- geoarea, [37](#), [37](#), [39](#), [109](#), [111](#), [125](#), [126](#)
- geoarea.old, [38](#)
- geoaxis, [39](#), [96](#)
- geochull, [40](#)
- geocontour, [24](#), [41](#), [48](#), [56](#), [62](#), [67](#), [72](#), [74](#), [81](#), [83](#), [107](#), [114](#), [139](#)
- geocontour.fill, [17](#), [24](#), [43](#), [44](#), [62](#), [67](#), [72](#), [74](#), [81](#), [83](#), [106](#), [114](#), [120](#)
- geoconvert, [50](#), [51](#)
- geoconvert.1, [50](#), [51](#)
- geoconvert.2, [50](#), [51](#)
- geocurve, [52](#), [63](#), [113](#)
- geodefine, [38](#), [48](#), [53](#), [63](#), [67](#), [88](#), [138](#)
- geodezoom, [54](#), [87](#)
- geoexpand, [54](#)
- geogrid, [43](#), [48](#), [50](#), [55](#), [62](#), [67](#), [72](#), [74](#), [81](#), [83](#)
- geoidentify, [50](#), [56](#)
- geoinside, [29](#), [38](#), [57](#), [99](#)
- geointersect, [29](#), [59](#)
- geolegend, [60](#)
- geolines, [27](#), [29](#), [36](#), [43](#), [48](#), [50](#), [53](#), [56](#), [61](#), [67](#), [72](#), [74](#), [83](#), [85](#), [131](#), [132](#), [141](#)
- geolines.with.arrows, [29](#), [62](#)
- geolocator, [15](#), [38](#), [43](#), [48](#), [54](#), [56–58](#), [63](#), [67](#), [83](#), [111](#), [113](#)
- geopar, [43](#), [48](#), [56](#), [63](#), [67](#), [77](#), [83](#), [104](#), [123](#)
- geopar.std, [63](#)
- geoplot, [8](#), [23](#), [36](#), [48](#), [54](#), [56](#), [58](#), [62](#), [63](#), [64](#), [72](#), [74](#), [75](#), [77](#), [81](#), [83](#), [86](#), [87](#), [96](#), [104](#), [117](#), [123](#), [130](#), [136](#), [138](#), [141](#)
- geoplotbwpalette, [70](#)
- geoplotpalette, [71](#)
- geopoints, [50](#), [62](#), [71](#), [74](#), [81](#), [138](#)
- geopolygon, [9](#), [19](#), [27](#), [43](#), [48](#), [50](#), [53](#), [56](#), [62](#), [67](#), [72](#), [72](#), [81](#), [83](#), [86](#), [131](#), [132](#)
- geoSR, [74](#), [131](#), [132](#)
- geosubplot, [76](#), [143](#)
- geosymbols, [15](#), [43](#), [48](#), [56](#), [62](#), [67](#), [72](#), [74](#), [77](#), [83](#), [114](#)
- geotext, [8](#), [40](#), [43](#), [48](#), [50](#), [56](#), [57](#), [62](#), [67](#), [72](#), [74](#), [81](#), [82](#), [117](#)
- geotows, [84](#)
- geoworld, [27](#), [85](#)
- geozoom, [54](#), [87](#)
- gepco500, [88](#)
- giveborder, [88](#)
- glaciers, [89](#)
- greenland, [89](#)
- grid, [17](#), [118](#), [120](#)
- gridaxes, [90](#), [96](#), [111](#), [116](#)
- gridaxes.Lambert, [6](#), [18](#), [19](#), [27](#), [91](#), [96](#)
- gridpoints, [92](#), [138](#)
- hsv, [71](#)
- icelanddrivers, [93](#)
- ices, [94](#)
- identify, [57](#)
- init, [23](#), [29](#), [40](#), [91](#), [92](#), [95](#)

- inside, [97](#), [138](#)
- inside.reg.bc, [98](#), [99](#), [127](#), [131](#)
- inside.reg.bc1, [98](#), [98](#), [131](#)
- inside.reg.lump, [99](#)
- intra.point.dist, [101](#)
- invlambert, [102](#)
- invmerc, [103](#)
- invProj, [9](#), [29](#), [40](#), [59](#), [63](#), [88](#), [91](#), [92](#), [96](#),
[102–104](#), [104](#), [117](#), [123](#)
- ir2d(d2ir), [20](#)
- island, [105](#)
- labels1, [15](#), [105](#)
- labels2, [15](#), [106](#)
- labels_line, [107](#)
- labels_size, [15](#), [108](#)
- lambert, [109](#)
- legend, [60](#)
- lines, [139](#)
- litir, [110](#)
- locator, [63](#)
- locdist, [110](#)
- mercator, [91](#), [111](#)
- merge, [6](#), [7](#)
- mr2d, [50](#)
- mr2d(rect2deg), [124](#)
- mrA(rectArea), [125](#)
- mrgrid(rGrid), [131](#)
- mrPeri(rectPeri), [126](#)
- nonsetpar, [112](#)
- ns, [53](#)
- Open.curve, [112](#)
- orthproj, [109](#), [113](#)
- paint.window, [15](#), [113](#)
- paint.window.x, [114](#)
- par, [86](#)
- pdist, [115](#)
- pdistx, [115](#)
- plot_nogrid, [91](#), [116](#)
- plot_reitnr, [116](#)
- pltgrid, [117](#)
- plvar, [118](#), [145](#)
- pointkriging, [10](#), [17](#), [115](#), [118](#), [118](#), [144](#), [145](#)
- points, [72](#), [81](#)
- polygon, [74](#)
- postcol, [120](#)
- pre2s, [121](#), [140](#)
- prepare.line, [19](#), [29](#), [121](#)
- Proj, [15](#), [29](#), [40](#), [59](#), [67](#), [91](#), [92](#), [96](#), [109](#), [111](#),
[114](#), [122](#), [139](#)
- ps, [12](#), [123](#)
- r2d, [21](#), [131](#)
- r2d(rect2deg), [124](#)
- rA(rectArea), [125](#)
- rect2deg, [21](#), [22](#), [124](#), [125](#)
- rectArea, [125](#), [126](#)
- rectPeri, [125](#), [126](#), [126](#), [131](#), [132](#)
- reg.bc, [98](#), [99](#), [127](#)
- reg.lump, [99](#), [128](#)
- reitaplott, [27](#), [29](#), [48](#), [67](#), [75](#), [106](#), [114](#), [128](#),
[131](#), [132](#), [140](#)
- Reitur2Svaedi1to10, [98](#), [99](#), [130](#)
- rGrid, [131](#)
- rgrid(rGrid), [131](#)
- Rlitir, [110](#), [132](#)
- rPeri(rectPeri), [126](#)
- s2pre, [133](#)
- SegmentWithArrow, [134](#)
- selectedpar, [23](#), [135](#), [138](#), [143](#)
- selpos, [8](#), [135](#)
- Set.grd.and.z, [136](#)
- setgrid, [93](#), [97](#), [137](#)
- shadeborder, [138](#)
- shading1, [15](#), [139](#)
- skipta.texta, [121](#), [140](#)
- SMB.std.background, [141](#)
- spherical, [142](#)
- spline.des, [123](#)
- sr2d, [98](#)
- sr2d(rect2deg), [124](#)
- srA(rectArea), [125](#)
- srgrid(rGrid), [131](#)
- srPeri(rectPeri), [126](#)
- subplot, [77](#), [142](#)
- twohmls, [143](#)
- variofit, [17](#), [30](#), [120](#), [143](#), [145](#)
- variogram, [17](#), [115](#), [116](#), [120](#), [144](#), [144](#)
- write, [133](#)